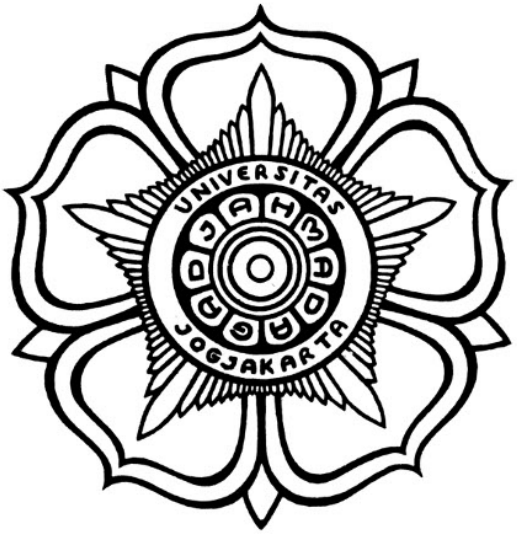
# RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

SKRIPSI




***THE SUSTAINABLE DEVELOPMENT GOALS***
***Decent Work and Economic Growth***
***Industry, Innovation and Infrastructure***

**Disusun oleh:**

**Muhammad Fachrizal Giffari**
**22/504570/TK/55192**

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI**
**DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA**
**YOGYAKARTA**
**2026**

# HALAMAN PENGESAHAN

# RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

**SKRIPSI**

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

**Muhammad Fachrizal Giffari**
**22/504570/TK/55192**

Telah disetujui dan disahkan

Pada tanggal . . . . . .

| Dosen Pembimbing I | Dosen Pembimbing II |
|---|---|
| **Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D.** | **Dr. Ir. Guntur Dharma Putra, S.T., M.Sc.** |
| **NIP. 196909201995121001** | **NIP. 111199104201802102** |

# PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

| | |
|---|---|
| Nama | : Muhammad Fachrizal Giffari |
| NIM | : 22/504570/TK/55192 |
| Tahun terdaftar | : 2022 |
| Program | : Sarjana |
| Program Studi | : Teknologi Infomrasi |
| Fakultas | : Teknik Universitas Gadjah Mada |

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 01 Juli 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Fachrizal Giffari
22/504570/TK/55192

# HALAMAN PERSEMBAHAN

*Bismillaahirrahmaanirrahiim.*
Skripsi ini kupersembahkan kepada kedua orang tuaku, saudaraku, dan juga teman-temanku
Skripsi ini juga kupersembahkan kepada sahabatku yaitu almarhum Bagas Amar Hakiki
Biarlah halaman ini menjadi saksi bisu bahwa kamu adalah manusia yang baik.

# KATA PENGANTAR

Puja dan puji syukur ke hadirat Allah Subhaanahu wa ta'aala atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi dengan judul "Rancang Bangun Rupiah Digital Ritel Berbasis Hyperledger Fabric" ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini, penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, Ph.D., IPM, SMIEEE., selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Ir. Ahmad Nasikun, S.T., M.Sc., IPM., selaku Ketua Program Sarjana Program Studi Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
3. Bapak Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D., selaku Dosen Pembimbing I yang senantiasa memberikan bimbingan dan masukan yang terbaik dalam pelaksanaan serta penulisan laporan tugas akhir.
4. Bapak Dr. Ir. Guntur Dharma Putra, S.T., M.Sc., selaku Dosen Pembimbing II yang telah memberikan arahan dan masukan teknis yang terbaik dalam pelaksanaan tugas akhir.
5. Ibu Dr. Eng. Silmi Fauziati, S.T., M.T., selaku Dosen Pembimbing Akademik yang senantiasa membantu penulis perihal dinamika perkuliahan.
6. Seluruh dosen dan tenaga pendidik Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada yang telah memberikan bekal ilmu kepada penulis selama menjalani masa perkuliahan.
7. Bapak Sujono, S.E., Ibu Mulyati, S.E., dr. Puji Arifianti Ramadhany, dan Muhammad Feizal Syafaat, S.T., selaku keluarga penulis yang senantiasa memanjatkan doa kepada penulis serta memberikan dukungan, semangat, dan motivasi kepada penulis dalam pengerjaan tugas akhir.
8. UKM UGM eSports 2024 yang telah memberikan penulis pandangan baru terkait dinamika eSports, dinamika UGM, hingga dinamika negara.
9. Cornelius Arden Satwika Hermawan, S.T., dan Mahsa Quereda Bahjah, S.T., selaku rekan kelompok mata kuliah Pemrograman Berorientasi Obyek yang terus berlanjut hingga Teknisiade, PPK Ormawa, Bengkel Inovasi, Kerja Praktik. Semoga kita dapat menjadi rekan kerja lagi di kesempatan lain.
10. WS yang telah menemani selama 24 jam dan membuat hidup penulis menjadi warna warni. Terimakasih sudah membuat kehidupan kuliah menjadi seperti tujuh belas lagi seperti yang dideskripsikan oleh Tulus, "Muda jiwa selamanya muda, Kisah kita abadi selamanya".
11. Abid, Ale, Caca, Ervina, Lintang, Nabila, Niko, Sanie, Vivian, Yajna, Abang Efrem, Bapa Poli, Abang Rinto, Abang Rino, Mama Fani, dan semuanya yang tidak bisa penulis sebutkan. Terimakasih sudah menjadi keluarga untuk penulis di Desa Iso, Kecamatan Kei Kecil Timur. Terimakasih juga sudah merayakan ulang tahun penulis dengan cara yang sangat berkesan.

12. Tim Jelajah Kei Kecil 2025, khususnya untuk Almarhum Bagas Amar Hakiki. Terimakasih sudah menjadi sahabat yang selalu menemani penulis bahkan saat berada di fase terendah.
13. Teman-teman Program Studi Teknologi Informasi Departemen Teknik Elektro dan Teknologi Informasi Universitas Gadjah Mada Angkatan 2022 yang telah menemani dan membantu penulis selama menjalani masa perkuliahan.

Tiada gading yang tak retak. Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan segala bentuk masukan, saran, dan kritik yang membangun. Akhir kata, penulis berharap semoga tulisan ini dapat memberikan manfaat dan wawasan tambahan bagi kita semua, aamiin

Yogyakarta, Juni 2026

Muhammad Fachrizal Giffari

# DAFTAR ISI


HALAMAN PENGESAHAN ..... ii
PERNYATAAN BEBAS PLAGIASI ..... iii
HALAMAN PERSEMBAHAN ..... iv
KATA PENGANTAR ..... vi
DAFTAR ISI ..... vii
DAFTAR TABEL ..... x
DAFTAR GAMBAR ..... xi
DAFTAR SINGKATAN ..... xiii
INTISARI ..... xiv
ABSTRACT ..... xv
BAB I Pendahuluan ..... 1
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 3
1.3 Tujuan Penelitian ..... 3
1.4 Batasan Penelitian ..... 4
1.5 Manfaat Penelitian ..... 6
1.6 Sistematika Penulisan ..... 6
BAB II Tinjauan Pustaka dan Dasar Teori ..... 7
2.1 Tinjauan Pustaka ..... 7
2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC ..... 7
2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric ..... 8
2.1.3 Posisi dan Celah Penelitian ..... 8
2.2 Dasar Teori ..... 10
2.2.1 Central Bank Digital Currency ..... 10
2.2.1.1 Model Pembayaran Berbasis Akun dan Token ..... 12
2.2.1.2 Arsitektur Penerbitan CBDC ..... 14
2.2.2 Blockchain dan Distributed Ledger Technology ..... 14
2.2.2.1 Struktur Blockchain ..... 14
2.2.2.2 Algoritme Konsensus ..... 15
2.2.2.3 Tipe Blockchain ..... 16
2.2.3 Hyperledger Fabric ..... 16
2.2.3.1 Arsitektur Hyperledger Fabric ..... 16
2.2.3.2 Hyperledger Fabric Ledger ..... 17
2.2.3.3 Alur Transaksi Hyperledger Fabric ..... 18
2.2.4 Hyperledger Caliper ..... 19
2.2.5 Transfer Ritel Langsung ..... 19

2.2.6 Application Programming Interface ..... 20
2.2.7 Bahasa Pemrograman Go ..... 20
2.3 Analisis Perbandingan Metode ..... 20
2.3.1 Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain ..... 20
2.3.2 Pemilihan Algoritme Konsensus Hyperledger Fabric ..... 23
2.3.3 Pemilihan Alat Pengujian Blockchain ..... 24
2.3.4 Pemilihan Arsitektur CBDC Dua Jenjang ..... 24
2.3.5 Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal ... 25
BAB III Metode Penelitian ..... 28
3.1 Metode yang Digunakan ..... 28
3.1.1 Metode Perancangan ..... 28
3.1.1.1 Analisis Kebutuhan Sistem ..... 28
3.1.1.2 Spesifikasi Kebutuhan Perangkat Lunak ..... 29
3.1.1.3 Perancangan Activity Diagram ..... 31
3.1.1.4 Perancangan Arsitektur Sistem Blockchain ..... 37
3.1.1.5 Perancangan Basis Data Luring ..... 44
3.1.2 Metode Implementasi ..... 45
3.1.2.1 Software Development Life Cycle ..... 45
3.1.2.2 Kerangka Kerja Blockchain ..... 45
3.1.2.3 Mekanisme Konsensus ..... 45
3.1.2.4 State Database ..... 45
3.1.2.5 Basis Data Luring ..... 46
3.1.2.6 Bahasa Pemrograman ..... 48
3.1.3 Metode Pengujian ..... 48
3.2 Alur Tugas Akhir ..... 54
3.2.1 Mempelajari Proses Bisnis & Sistem Blockchain ..... 54
3.2.2 Melakukan Studi Literatur ..... 54
3.2.3 Menentukan Metode Penelitian ..... 55
3.2.4 Pengembangan Sistem Blockchain bi-coin ..... 55
3.2.4.1 Instalasi Kerangka Kerja Hyperledger Fabric ..... 55
3.2.4.2 Konfigurasi Jaringan Blockchain ..... 55
3.2.4.3 Deploy Chaincode ..... 55
3.2.4.4 Pengembangan Fabric Gateway ..... 57
3.2.4.5 Pengembangan Backend ..... 57
3.2.5 Pengujian Sistem Blockchain bi-coin ..... 58
3.2.5.1 Unit Test Chaincode dan Backend ..... 59
3.2.5.2 Konfigurasi Hyperledger Caliper ..... 59
3.2.5.3 Menjalankan Pengujian Hyperledger Caliper ..... 60

3.2.5.4 Eksperimen Kontrol pada Lingkungan Terisolasi ..... 60
BAB IV Hasil dan Pembahasan ..... 68
4.1 Hasil Implementasi Sistem ..... 68
4.1.1 Jaringan Hyperledger Fabric ..... 68
4.1.2 Chaincode Digital Rupiah ..... 68
4.1.3 Backend API ..... 70
4.1.4 Hasil Pengujian Fungsional ..... 70
4.2 Hasil Evaluasi Kinerja Final ..... 72
4.3 Perbandingan Hasil Penelitian dengan Hasil Terdahulu ..... 76
4.4 Hasil Uji Kontrol pada Lingkungan Terisolasi ..... 77
BAB V Kesimpulan dan Saran ..... 82
5.1 Kesimpulan ..... 82
5.2 Saran ..... 83
DAFTAR PUSTAKA ..... 84
LAMPIRAN ..... L-1
L.1 Source Code Otorisasi MSP *Chaincode* ..... L-2
L.2 Source Code Pengawasan dan Rekonsiliasi ..... L-2
L.3 Source Code Konfigurasi Jaringan Caliper ..... L-5
L.4 Source Code Suite Konfigurasi *Benchmark* Caliper ..... L-6
L.5 Source Code Skrip Pembangkit Aset Jaringan Caliper ..... L-12

# DAFTAR TABEL


Tabel 2.1 Perbandingan penelitian terdahulu dengan penelitian ini ..... 9
Tabel 2.2 Perbandingan sistem terpusat, *permissionless blockchain*, dan *permissioned blockchain* untuk CBDC ..... 21
Tabel 2.3 Perbandingan platform DLT *permissioned* untuk sistem CBDC ..... 22
Tabel 2.4 Perbandingan algoritme konsensus untuk *blockchain permissioned* .. 23
Tabel 2.5 Perbandingan Hyperledger Caliper dan BlockBench berdasarkan [1, 2]. ..... 24
Tabel 2.6 Perbandingan arsitektur CBDC satu jenjang dan dua jenjang ..... 25
Tabel 2.7 Pemetaan purwarupa terhadap dimensi desain CBDC optimal [3] ... 26

Tabel 3.1 Kebutuhan fungsional purwarupa bi-coin ..... 29
Tabel 3.2 Kebutuhan nonfungsional purwarupa bi-coin ..... 30
Tabel 3.3 Keterlacakan kebutuhan terhadap metode verifikasi ..... 31
Tabel 3.4 Organisasi pada jaringan baseline Rupiah Digital ..... 42
Tabel 3.5 Objek status utama pada ledger ..... 44
Tabel 3.6 Perbandingan basis data status Hyperledger Fabric berdasarkan dokumentasi resmi [4]. ..... 46
Tabel 3.7 Perbandingan basis data kandidat untuk penyimpanan luring. ..... 48
Tabel 3.8 Tahap pencarian dan konfirmasi kapasitas ..... 50
Tabel 3.9 Cakupan 20 skenario fungsional dan assertion yang dijalankan ..... 51
Tabel 3.10 Lingkungan pengujian studi kontrol ..... 60

Tabel 4.1 Kelompok fungsi chaincode digital-rupiah ..... 69
Tabel 4.2 Peran akses pada backend API ..... 70
Tabel 4.3 Lingkungan pengujian kinerja final ..... 72
Tabel 4.4 Beban berkelanjutan pada pengujian primer ..... 73
Tabel 4.5 Kategori kegagalan pada pengujian primer pengganti ..... 73
Tabel 4.6 Statistik pengulangan pada beban berkelanjutan ..... 74
Tabel 4.7 Demonstrasi terpisah PostgreSQL/KYC ..... 74
Tabel 4.8 Hasil konformitas semantik Caliper final ..... 75
Tabel 4.9 Metrik sumber daya yang dikumpulkan pada pengujian primer ..... 75
Tabel 4.10 Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024). ..... 76
Tabel 4.11 Batas bawah laju lulus terkonfirmasi tiga *chaincode* studi pada satu *worker* (ops/s) ..... 79
Tabel 4.12 Konsumsi sumber daya komponen pada jendela konfirmasi 30 detik (31 beban kerja) ..... 80

# DAFTAR GAMBAR


Gambar 2.1 Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia ..... 10
Gambar 2.2 Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [5]. ..... 11
Gambar 2.3 Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6] ..... 12
Gambar 2.4 Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5]. ..... 13
Gambar 2.5 Perbandingan arsitektur penerbitan CBDC: tidak langsung (*indirect*/dua jenjang), langsung (*direct*/satu jenjang), dan hibrida (*hybrid*). Diadaptasi dari Auer dan Böhme [6] ..... 14
Gambar 2.6 Struktur rantai blok: setiap blok memuat *hash* kriptografis dari blok sebelumnya ..... 15
Gambar 2.7 Pohon Merkle: *hash* transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok ..... 15
Gambar 2.8 Diagram transisi *state* algoritme konsensus Raft: setiap simpul berstatus *Follower*, *Candidate*, atau *Leader* ..... 16
Gambar 2.9 Arsitektur komponen Hyperledger Fabric: klien mengirim proposal ke *peer endorser*, mengirim transaksi terendorsement ke *orderer*, lalu menerima komitmen blok pada semua *peer* dalam *channel*. ..... 17
Gambar 2.10 Dualitas komponen *ledger* Hyperledger Fabric: *blockchain* sebagai catatan transaksi historis tambah-saja yang perubahan retrospektifnya dapat dideteksi, dan *world state* sebagai representasi status terkini pada CouchDB ..... 18
Gambar 2.11 Siklus transaksi Hyperledger Fabric pada topologi uji dengan satu *consenter* Raft tanpa toleransi kegagalan *orderer*. ..... 19
Gambar 2.12 Perbandingan *throughput* transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [7]. ..... 23
Gambar 3.13 Siklus SDLC dengan metode *prototyping*: empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja .. 28
Gambar 3.14 Activity diagram pendaftaran peserta ..... 32
Gambar 3.15 Activity diagram penerbitan Digital Rupiah ..... 33
Gambar 3.16 Activity diagram distribusi likuiditas ke PJP ..... 34
Gambar 3.17 Activity diagram transfer saldo ritel ..... 35
Gambar 3.18 Activity diagram pembekuan dan pencairan peserta oleh Bank Indonesia ..... 36
Gambar 3.19 Activity diagram laporan dan pengawasan supervisi ..... 37
Gambar 3.20 Arsitektur distribusi dua jenjang pada purwarupa Rupiah Digital ritel. ..... 38

Gambar 3.21 Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5]. . . . . 39
Gambar 3.22 Arsitektur baseline kinerja `digital-rupiah` v3.0: lapisan *client*, *backend API* (Go REST + JWT/RBAC), basis data PostgreSQL luring, *Fabric Gateway SDK*, serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi *peer*, CouchDB, CA per organisasi, dan *orderer* Raft . . . . . 41
Gambar 3.23 Arsitektur jaringan baseline kinerja `digital-rupiah` v3.0 dengan lima organisasi. . . . . 42
Gambar 3.24 Diagram use case keseluruhan sistem Rupiah Digital. . . . . 43
Gambar 3.25 Diagram use case operasi institusional. . . . . 43
Gambar 3.26 Diagram use case operasi retail . . . . . 44
Gambar 3.27 Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian . . . . . 54
Gambar 3.28 Implementasi fungsi `Mint` pada *chaincode*: memverifikasi otorisasi dan status beku sebelum menambah saldo dompet. . . . . 56
Gambar 3.29 Pemeriksaan otorisasi, status dompet, dan kecukupan saldo sebelum transfer. . . . . 57
Gambar 3.30 Pengelompokan rute REST *backend* berdasarkan peran akses . . . . 58
Gambar 3.31 Arsitektur lingkungan studi kontrol dengan empat MSP, tiga *chaincode*, identitas per pengguna, dan basis data PostgreSQL terisolasi. . . . . 61
Gambar 3.32 Diagram kelas layanan backend dan tiga *chaincode* pada studi kontrol. . . . . 63
Gambar 3.33 Activity diagram alur onboarding subjek, persetujuan, pendanaan, dan transfer pada studi kontrol. . . . . 63
Gambar 3.34 Diagram use case fungsi empat organisasi pada studi kontrol. . . . . 64
Gambar 3.35 Urutan pembatasan subjek dan pelepasan pembatasan dengan bukti otoritas dan dua identitas staf berbeda. . . . . 65
Gambar 3.36 Urutan konversi *wholesale-to-retail* dari cadangan distributor ke *wallet* ritel dalam satu transaksi kanal. . . . . 66
Gambar 3.37 Urutan transfer ritel dengan pemeriksaan kewenangan, kelayakan, batas, saldo, dan receipt. . . . . 67
Gambar 4.38 Siklus hidup peserta pada *chaincode*: dari *pending* (menunggu persetujuan) menjadi *active*, kemudian dapat dibekukan (*frozen*) dan dicairkan kembali. Penolakan menghapus peserta yang masih *pending*, sedangkan `OffboardParticipant` menyimpan status *offboarded*, membekukan dompet terkait, dan mensyaratkan saldo agregat nol. . . . . 69

# DAFTAR SINGKATAN

| | | |
|---|---|---|
| API | = | *Application Programming Interface* |
| BI | = | Bank Indonesia |
| BSPI | = | Blueprint Sistem Pembayaran Indonesia |
| CBDC | = | *Central Bank Digital Currency* |
| CA | = | *Certificate Authority* |
| CFT | = | *Crash Fault Tolerance* |
| DLT | = | *Distributed Ledger Technology* |
| MSP | = | *Membership Service Provider* |
| OJK | = | Otoritas Jasa Keuangan |
| PJP | = | Penyedia Jasa Pembayaran |
| REST | = | *Representational State Transfer* |
| RTGS | = | *Real-Time Gross Settlement* |
| TEE | = | *Trusted Execution Environment* |
| TLS | = | *Transport Layer Security* |
| TPS | = | *Transactions Per Second* |
| UTXO | = | *Unspent Transaction Output* |

# INTISARI

Penelitian ini mengimplementasikan dan menguji kontrol KYC dan AML/CFT terpilih pada purwarupa Rupiah Digital ritel berbasis Hyperledger Fabric 2.5. Lingkungan studi kontrol menggunakan jaringan terisolasi dengan empat MSP: `CentralBankMSP`, `DistributorMSP`, `ValidatorMSP`, dan `EndUserMSP`. Tiga *chaincode* Go memisahkan operasi moneter, kontrol pelanggan, dan pembayaran. Identitas X.509 yang dikelola *backend* memisahkan penanda tangan, subjek pelanggan, dan kustodian dompet.

Uji unit sintetis dan uji integrasi *chaincode* lulus untuk keputusan KYC berisiko tinggi dengan peninjau dan pejabat senior yang berbeda, pembatasan dan pembukaan pembatasan berbukti, pendanaan cadangan, pembayaran ritel, pembacaan oleh `ValidatorMSP`, serta penolakan transaksi dengan *endorsement* yang tidak mencukupi. Hasil ini hanya menunjukkan perilaku pada skenario sintetis yang diuji; penelitian tidak membuktikan keaslian identitas, efektivitas penyaringan, pengiriman laporan kepada PPATK, atau kepatuhan institusional.

Sebagai bukti kinerja sekunder, Hyperledger Caliper menguji baseline terpisah berisi lima organisasi dengan `digital-rupiah` v3.0. Transfer ritel pada kontensi rendah mencapai 20 TPS, sedangkan uji berpasangan pada kontensi tinggi mencapai 2,01 TPS. Pengukuran tersebut tidak menguji tiga *chaincode* lingkungan kontrol dan tidak menetapkan kapasitas produksi atau reproduktibilitas lintas mesin.

Kata kunci: Rupiah Digital, CBDC ritel, KYC, AML/CFT, Hyperledger Fabric, *endorsement*, distribusi dua jenjang

# ABSTRACT

*This thesis implements and tests selected KYC and AML/CFT controls in a retail Digital Rupiah prototype using Hyperledger Fabric 2.5. Its controls-study extension uses a fresh four-MSP network—CentralBankMSP, DistributorMSP, ValidatorMSP, and EndUserMSP—and three Go chaincodes for monetary operations, customer controls, and payments. Server-managed X.509 identities distinguish the signing actor, customer subject, and wallet custodian.*

*Synthetic unit tests and a chaincode integration smoke test pass high-risk KYC decisions with distinct reviewer and senior approver, restriction and evidence-based release, reserve funding, retail payment, read-only ValidatorMSP access, and expected endorsement failures. These results establish behavior only for the tested synthetic scenarios; they do not establish identity authenticity, screening effectiveness, actual PPATK reporting, or institutional compliance.*

*As secondary performance evidence, Hyperledger Caliper tests the separate five-organization* `digital-rupiah` *v3.0 baseline. Low-contention retail transfer reaches 20 TPS, while the paired high-contention test reaches 2.01 TPS. Those measurements do not benchmark the three-chaincode extension and do not establish production capacity or cross-machine reproducibility.*

**Keywords**: Digital Rupiah, retail CBDC, KYC, AML/CFT, Hyperledger Fabric, endorsement, two-tier distribution

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Transformasi digital pada sektor sistem pembayaran Indonesia berlangsung sangat cepat dalam satu dekade terakhir. Penetrasi telepon pintar dan pertumbuhan ekonomi digital mendorong masyarakat beralih dari uang tunai menuju instrumen pembayaran nontunai. Bank Indonesia melalui Blueprint Sistem Pembayaran Indonesia (BSPI) menempatkan digitalisasi sebagai agenda utama untuk menjaga kedaulatan moneter, mendorong inklusi keuangan, serta memperkuat integrasi ekonomi dan keuangan digital nasional [8]. Dalam konteks tersebut, kehadiran mata uang digital bank sentral atau *Central Bank Digital Currency* (CBDC) menjadi salah satu inisiatif strategis untuk melengkapi uang kartal dan uang elektronik yang telah ada.

Pertumbuhan ekosistem digital tersebut, bagaimanapun, juga membawa risiko baru bagi stabilitas sistem keuangan. Pada tahun 2022, penetrasi internet Indonesia mencapai 74% dengan 204,7 juta pengguna, sementara pelanggan seluler mencapai 370,1 juta atau setara 133% populasi. Pandemi COVID-19 menambah 21 juta konsumen digital baru yang sebelumnya belum terlayani secara daring, sebagaimana dirangkum dalam Tabel 1 White Paper CBDC Bank Indonesia [5]. Pada periode yang sama, kapitalisasi pasar aset kripto global melonjak hingga hampir 3 triliun dolar AS, tumbuh 1.393% antara 2019 dan 2021, sementara investor aset kripto di Indonesia mencapai 16,3 juta per September 2022 dengan laju pertumbuhan 81,6% *year-over-year* [5]. Fenomena ini memunculkan *cryptoization*, yaitu kondisi di mana adopsi aset kripto oleh penduduk menyebar lebih cepat dibandingkan mata uang resmi [5]. Disrupsi digital tidak lagi terbatas pada isu *shadow banking*, melainkan telah merambah pada fenomena *shadow currency* dan *shadow central banking* yang mengikis efektivitas kebijakan bank sentral [5]. *Stablecoin* dan aset kripto tanpa aset dasar memperbesar risiko makrofinansial berupa melemahnya kendali otoritas atas stabilitas moneter dan sistem keuangan [5]. Mata uang digital bank sentral dirancang untuk menjaga kepercayaan publik terhadap uang dan kedaulatan moneter, sekaligus mempertahankan peran bank sentral sebagai jangkar stabilitas dalam ekonomi digital [5, 9].

CBDC merupakan representasi digital dari uang yang diterbitkan dan menjadi kewajiban langsung bank sentral, sehingga memiliki kualitas bebas risiko yang setara dengan uang tunai [10]. Berbeda dengan uang elektronik yang diterbitkan oleh penyelenggara swasta dan merupakan kewajiban penerbitnya, CBDC menempatkan bank sentral sebagai penjamin nilai akhir. Bagi Indonesia, Bank Indonesia menamai inisiatif CBDC sebagai Rupiah Digital melalui Proyek Garuda, yang menempuh

pendekatan bertahap dimulai dari Rupiah Digital *wholesale* (w-Digital Rupiah) untuk transaksi antarlembaga, kemudian diperluas menuju Rupiah Digital *retail* (r-Digital Rupiah) bagi masyarakat luas [5, 11]. Desain Rupiah Digital mengadopsi model dua jenjang (*two-tier*), yaitu Bank Indonesia menerbitkan dan mengelola Rupiah Digital pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui pihak perantara seperti perbankan dan penyedia jasa pembayaran [12].

Salah satu pilihan teknologi yang banyak dikaji untuk menopang CBDC adalah *Distributed Ledger Technology* (DLT) berbasis *blockchain*. Teknologi ini menawarkan keunggulan berupa integritas data yang terjaga melalui catatan yang bersifat *append-only*, jejak audit yang transparan, serta kemampuan otomasi aturan bisnis melalui *smart contract* [13, 14]. Untuk kebutuhan bank sentral, jenis *blockchain* yang sesuai adalah *permissioned blockchain*, yaitu jaringan dengan keanggotaan terbatas dan teridentifikasi, sehingga bank sentral tetap memegang kendali atas tata kelola jaringan [15]. Hyperledger Fabric merupakan salah satu kerangka kerja *permissioned blockchain* paling matang yang banyak digunakan pada ranah keuangan. Fabric mendukung arsitektur modular, identitas berbasis *Membership Service Provider* (MSP), pemisahan transaksi melalui kanal (*channel*), serta model eksekusi *execute-order-validate* yang mendukung pemrosesan paralel tanpa bergantung pada mekanisme *proof-of-work* yang boros energi [16].

Meskipun kajian mengenai CBDC berkembang pesat, fokus literaturnya masih timpang. Sebagian besar penelitian menelaah CBDC dari sisi makroekonomi, kebijakan moneter, dan stabilitas keuangan [9, 10, 17], atau dari sisi taksonomi desain tingkat tinggi seperti pilihan *token-based* versus *account-based* dan model satu jenjang versus dua jenjang [6, 18–20]. Sebaliknya, artefak teknis yang utuh, terbuka, dan dapat diuji secara empiris untuk konteks CBDC ritel masih jarang tersedia.

Kajian teknis yang ditinjau masih membahas komponen CBDC secara terpisah. Privasi dan auditabilitas dibahas dalam [21–23], keamanan dan pemodelan ancaman dalam [24], serta arsitektur jaringan dalam [15, 25]. Kelayakan sistem CBDC ritel, sebaliknya, bergantung pada keterpaduan komponen tersebut. Dalam korpus literatur yang digunakan pada penelitian ini, penerbitan dan distribusi likuiditas berjenjang, transfer ritel, serta pengawasan belum dievaluasi sebagai satu artefak Fabric yang kinerjanya dapat diukur.

Untuk konteks Indonesia, Bank Indonesia telah menetapkan arah desain Rupiah Digital berupa model dua jenjang dan interoperabilitas sistem pembayaran [5, 12]. Desain CBDC bagi perekonomian Indonesia juga telah dirumuskan dalam [26–28]. Realisasi teknis Proyek Garuda yang terdokumentasi secara publik baru mencakup lapisan *wholesale* [11]. Rohmalia dan Nugraha [29] telah merancang dan mengimplementasikan purwarupa CBDC ritel berbasis Hyperledger Fabric. Makalah

mereka memperlihatkan pembuatan akun dan transfer aset antarpengguna, tetapi tidak menyajikan pengawasan multipihak atau pengujian Caliper yang dapat diulang. Dari sisi evaluasi kinerja, CBDC ritel yang telah diluncurkan maupun yang masih berupa pilot (seperti eNaira di Nigeria [30] dan pilot ritel di Thailand [31]) belum menyertakan tolok ukur kinerja terstandar yang dapat dibandingkan langsung dengan purwarupa ini. Sistem berkinerja tinggi seperti Hamilton/OpenCBDC menggunakan arsitektur *account-based* terpusat, bukan model *blockchain* [32]. Akibatnya, data *throughput* dan latensi yang terukur untuk skenario CBDC ritel di atas *permissioned blockchain*, khususnya untuk Rupiah Digital, masih langka.

Atas dasar literatur dan artefak publik yang ditinjau, celah penelitian berada pada kebutuhan akan purwarupa CBDC ritel yang (i) mengintegrasikan penerbitan berjenjang dan transfer ritel, (ii) menguji kontrol KYC dan AML/CFT terpilih melalui batas otorisasi serta *endorsement* yang dapat diamati, dan (iii) melaporkan kinerja dengan konfigurasi serta artefak yang dapat diulang. Penelitian ini menempatkan verifikasi kontrol sebagai evaluasi utama melalui lingkungan studi terisolasi yang menggunakan empat organisasi Fabric dan tiga *chaincode* domain. Pengukuran Hyperledger Caliper pada purwarupa `digital-rupiah` v3.0 tetap dilaporkan sebagai bukti kinerja sekunder; hasil tersebut tidak dianggap sebagai pengukuran kinerja lingkungan kontrol yang baru.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah pada penelitian ini adalah sebagai berikut.

1. Bagaimana merancang dan membangun lingkungan studi Rupiah Digital ritel dengan empat organisasi dan tiga *chaincode* yang memisahkan operasi moneter, kontrol pelanggan, dan pembayaran?
2. Apakah kontrol KYC dan AML/CFT terpilih, pemisahan persetujuan staf, bukti pembatasan/pembukaan pembatasan, dan kebijakan *endorsement* bekerja sesuai skenario fungsional sintetis?
3. Bagaimana karakteristik kinerja purwarupa `digital-rupiah` v3.0 pada beban yang diuji menggunakan Hyperledger Caliper sebagai bukti sekunder yang terpisah dari lingkungan studi kontrol?

## 1.3 Tujuan Penelitian

Tujuan yang ingin dicapai pada penelitian ini adalah sebagai berikut.

1. Membangun lingkungan studi terisolasi dengan MSP `CentralBankMSP`, `DistributorMSP`, `ValidatorMSP`, dan `EndUserMSP`, serta tiga *chaincode* untuk domain moneter, kontrol, dan pembayaran.

2. Memverifikasi skenario sintetis untuk keputusan KYC, persetujuan staf yang berbeda, pembatasan/pembukaan pembatasan, alur pendanaan dan pembayaran, serta penolakan transaksi dengan dukungan organisasi yang tidak mencukupi.
3. Melaporkan hasil Caliper pada purwarupa `digital-rupiah` v3.0 sebagai evaluasi kinerja sekunder, tanpa mengatribusikannya kepada tiga *chaincode* lingkungan studi kontrol.

## 1.4 Batasan Penelitian

1. Sistem yang dibangun merupakan purwarupa (*prototype*) yang dijalankan pada lingkungan jaringan tersimulasi secara lokal menggunakan *container*, bukan sistem produksi.
2. Pengujian kontrol utama menggunakan jaringan Fabric 2.5 terisolasi pada kanal `rupiah-study`, empat MSP, satu *peer* per MSP, dan satu *orderer* di bawah `CentralBankMSP`. Pengukuran kinerja sekunder menggunakan lingkungan lima organisasi dan *chaincode* `digital-rupiah` v3.0 yang terpisah.
3. Penelitian berfokus pada kontrol KYC dan AML/CFT terpilih serta verifikasi fungsionalnya; penelitian ini tidak menilai kepatuhan institusional secara menyeluruh, efektivitas model risiko di dunia nyata, kebijakan moneter, atau dampak makroekonomi. Analisis strategi kebijakan moneter dan dampak makroekonomi CBDC, termasuk peran *CBDC rate*, didelegasikan pada studi terpisah seperti [3].
4. Evaluasi kinerja dilakukan menggunakan Hyperledger Caliper dengan beban yang dihasilkan oleh sejumlah *worker* pada lingkungan pengujian lokal.
5. Pengujian dijalankan pada satu inang (*single host*) dengan satu, dua, dan empat *worker* Caliper dan satu *peer* per organisasi, tanpa latensi jaringan dan persebaran geografis yang nyata. Karena itu, angka *throughput* dan latensi yang diperoleh tidak dapat langsung digeneralisasi ke perangkat keras maupun topologi produksi [32,33].
6. Komposisi beban transaksi dan distribusi nilai yang digunakan merupakan asumsi desain yang menyerupai pola pembayaran ritel di Indonesia, bukan jejak transaksi empiris. Profil transfer menggunakan konfigurasi terpisah untuk satu, dua, dan empat *worker*; setiap profil menetapkan populasi, slot transaksi, saldo awal, dan durasi pada berkas konfigurasi yang ikut dihash dalam manifes. Ronde pemanasan berlangsung 30 detik, sedangkan ronde terukur berlangsung 120 detik.
7. Lingkup pelanggan terdiri atas individu ritel, pedagang perorangan, dan bisnis/perusahaan. Wakil perusahaan dan pemilik manfaat dimodelkan dalam uji tuntas bisnis, bukan sebagai kategori pelanggan tambahan. Data uji bersifat sintetis; verifikasi Dukcapil, umpan daftar sanksi langsung, pengiriman laporan

PPATK, dan pemrosesan dokumen identitas nyata tidak dilakukan.

8. Evaluasi transfer dasar menggunakan skenario perebutan kunci rendah (*low-contention*) dengan rencana transaksi berputar dan populasi tetap. Skenario terpisah menguji perebutan satu dompet melalui permintaan transfer agregat yang melebihi saldo awal. Penelitian tidak menggunakan jejak transaksi empiris untuk menetapkan tingkat konsentrasi maupun kebijakan *retry/backoff* yang representatif. Hasilnya karena itu tidak mewakili beban pedagang populer atau kapasitas produksi.
9. Purwarupa ini tidak mengimplementasikan pembayaran CBDC tanpa koneksi jaringan. Kemampuan tersebut memerlukan desain perangkat aman, verifikasi kriptografis, pencegahan *double-spending*, dan rekonsiliasi setelah konektivitas kembali [34, 35]. Hasil penelitian ini hanya merepresentasikan transaksi yang diselesaikan melalui koneksi ke *ledger*.
10. Sistem belum menyediakan privasi transaksi pada kanal bersama. Topologi memakai satu *application channel* tanpa *private data collection* atau teknik kriptografi penjaga privasi. Akibatnya, metadata dompet dan graf transaksi terlihat oleh peserta kanal yang berwenang. Purwarupa ini belum menangani kompromi antara privasi dan keterlacakan [22, 36, 37].
11. Topologi hanya memiliki satu *consenter* Raft sehingga tidak menoleransi kegagalan *orderer*. Evaluasi juga tidak mencakup kompromi simpul, ketersediaan tinggi, RTO/RPO, rotasi atau revokasi identitas, peningkatan versi, respons insiden, dan pemulihan bencana. Kesiapan produksi serta dampak operasional dan inklusi keuangan tidak dinilai.
12. Proyek Garuda menetapkan tiga prinsip 3i sebagai prasyarat desain Rupiah Digital: integrasi dengan infrastruktur pasar keuangan (IPK) domestik, interoperabilitas teknis dan semantik dengan sistem yang ada maupun yang akan datang, serta interkoneksi lintas batas untuk jaringan CBDC antarnegara [5]. Purwarupa ini memenuhi prinsip tersebut hanya pada tataran internal: alur penerbitan–distribusi–pembayaran–pengawasan berjalan secara terpadu dalam satu jaringan Hyperledger Fabric. Namun, integrasi eksternal dengan BI-RTGS dan BI-FAST hanya dimodelkan sebagai titik masuk (*stub*) tanpa koneksi nyata, Penyimpan Digital Rupiah (KDR) belum diimplementasikan, konformitas terhadap standar SNAP API nasional tidak diterapkan, dan tidak ada gerbang (*gateway*) menuju IPK domestik maupun jaringan CBDC lintas batas. Pemenuhan prinsip 3i secara penuh merupakan pekerjaan lanjutan yang berada di luar cakupan purwarupa ini.

## 1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut.

1. Memberikan rujukan teoretis tentang perancangan CBDC ritel berbasis *permissioned blockchain* yang mencakup penerbitan berjenjang, distribusi kepada kustodian bank atau PJP, transaksi antarpengguna, dan pengawasan oleh bank sentral.
2. Menjelaskan penerapan Hyperledger Fabric untuk memodelkan pemisahan peran antara BI, peserta distribusi, dan pengguna ritel tanpa menempatkan data portal pada buku besar.
3. Menyediakan purwarupa yang dapat digunakan sebagai bahan eksperimen teknis untuk menguji alur penerbitan, transfer, otorisasi, dan pencatatan peristiwa pengawasan.
4. Memberikan dasar evaluasi kinerja awal melalui pengujian beban terbatas sehingga karakteristik *throughput*, latensi, dan keberhasilan transaksi dapat dibahas secara terukur dalam ruang lingkup purwarupa.
5. Mendukung diskusi teknis pengembangan Rupiah Digital ritel dengan menunjukkan batas kemampuan purwarupa, termasuk kebutuhan perluasan topologi, privasi transaksi, dan pengujian keamanan lanjutan, tanpa menyimpulkan dampak inklusi atau kesiapan produksi.

## 1.6 Sistematika Penulisan

Penulisan skripsi ini disusun ke dalam lima bab dengan sistematika sebagai berikut.

BAB I PENDAHULUAN berisi latar belakang, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA DAN DASAR TEORI memaparkan tinjauan terhadap penelitian terdahulu yang relevan serta dasar teori yang mendasari penelitian, meliputi CBDC, *blockchain* dan DLT, Hyperledger Fabric, dan Hyperledger Caliper.

BAB III METODE PENELITIAN menguraikan alat dan bahan, metode perancangan, metode implementasi, metode pengujian, serta alur penelitian yang ditempuh.

BAB IV HASIL DAN PEMBAHASAN menyajikan hasil implementasi sistem, hasil evaluasi kinerja menggunakan Hyperledger Caliper, serta perbandingan dengan penelitian terdahulu.

BAB V KESIMPULAN DAN SARAN memuat kesimpulan yang menjawab rumusan masalah serta saran untuk pengembangan penelitian selanjutnya.

# BAB II
# TINJAUAN PUSTAKA DAN DASAR TEORI

## 2.1 Tinjauan Pustaka

Kajian penerapan *blockchain* untuk mata uang digital bank sentral mencakup beragam fokus. Bagian ini meninjau penelitian terdahulu yang relevan dengan rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric, meliputi arsitektur, transfer ritel, pengawasan, dan evaluasi kinerja.

### 2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC

Sethaput dan Innet [14] menyusun *systematization of knowledge* (SoK) mengenai aplikasi *blockchain* pada CBDC dan mengidentifikasi bahwa pilihan antara *token-based* dan *account-based*, model satu jenjang versus dua jenjang, serta tingkat privasi merupakan dimensi desain yang paling menentukan. Bhawana dan Kumar [15] membangun jaringan CBDC berbasis *permissioned blockchain* dan menegaskan bahwa keanggotaan terbatas memberikan kendali tata kelola yang dibutuhkan bank sentral tanpa mengorbankan transparansi. Sun dkk. [25] mengusulkan model *multi-blockchain* untuk memisahkan beban transaksi antarwilayah, sedangkan Islam dan In [21] menggunakan *consortium blockchain* dengan model *unspent transaction output* (UTXO) untuk menjaga privasi sekaligus auditabilitas.

Kajian lain merumuskan model audit CBDC berbasis *blockchain* dari pengalaman *Digital Currency Electronic Payment* (DCEP) Tiongkok [38]. Hans dkk. [24] menerapkan pemodelan ancaman STRIDE untuk mengidentifikasi risiko keamanan pada sistem CBDC berbasis *blockchain*, menegaskan perlunya analisis keamanan sejak tahap perancangan. Block-PAD [39] menawarkan kerangka transaksi CBDC yang tangguh dan fleksibel dengan memanfaatkan identitas digital.

Lukito dkk. [40] membandingkan sembilan program CBDC berdasarkan dokumen publik dan menguraikan perbedaan peran perantara dalam distribusi dua jenjang. Rancangan Rupiah Digital dalam penelitian ini merujuk pada dokumen Bank Indonesia [5, 11].

Pada sisi ketahanan layanan, Project Polaris dari *BIS Innovation Hub* menunjukkan bahwa dukungan pembayaran tanpa konektivitas memerlukan desain perangkat aman dan kontrol kriptografis khusus [34, 35]; kemampuan ini diposisikan sebagai keterbatasan dan arah pengembangan lanjutan pada penelitian ini.

### 2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric

Rohmalia dan Nugraha [29] menyusun model CBDC ritel Indonesia berbasis Hyperledger Fabric yang mencakup pembuatan akun, konversi aset, dan transfer antarpengguna. Hasil implementasi yang mereka laporkan memperlihatkan pembuatan akun dan transfer, disertai waktu pengiriman untuk satu dan 50 data. Makalah tersebut tidak menyajikan pengawasan multipihak ataupun tolok ukur Caliper yang dapat diulang.

Pada aspek platform dan kinerja, Androulaki dkk. [16] memperkenalkan Hyperledger Fabric sebagai *distributed operating system* untuk *permissioned blockchain* dengan arsitektur *execute-order-validate*. Evaluasi kinerja Fabric mencakup pemodelan latensi [41], evaluasi kerangka berbasis Fabric [42], serta perbandingan *throughput* antara *blockchain permissioned* dan *permissionless* [43]. Literatur tersebut menjadi dasar pemilihan Fabric dan perancangan pengujian Caliper, tetapi angka kinerjanya tidak dibandingkan langsung karena versi, topologi, dan beban kerja berbeda.

### 2.1.3 Posisi dan Celah Penelitian

Penelitian terdahulu dapat dikelompokkan menurut fokusnya. Arsitektur dan tata kelola jaringan dibahas dalam [15, 25], privasi dan auditabilitas dalam [21–23], serta keamanan dan audit dalam [24, 38]. Kelompok lain membahas kinerja Hyperledger Fabric [16, 33, 41–43] dan desain CBDC untuk konteks Indonesia [26–29, 40]. Setiap kelompok menjawab sebagian persoalan. Dalam korpus yang ditinjau pada penelitian ini, belum ditemukan evaluasi satu artefak Fabric ritel yang memadukan distribusi berjenjang, transfer ritel, pengawasan, dan tolok ukur Caliper yang dapat diulang.

**Celah penelitian.** Sintesis terbatas pada sumber yang dirujuk di bab ini menunjukkan empat kebutuhan yang belum dipenuhi secara bersamaan. (1) Satu artefak perlu mengintegrasikan penerbitan dan distribusi likuiditas berjenjang, transfer antarpelanggan dan pelanggan-ke-pedagang, serta pengawasan. (2) Artefak tersebut perlu selaras dengan rancangan dua jenjang Rupiah Digital Bank Indonesia karena dokumentasi publik Proyek Garuda baru menjangkau lapisan *wholesale* [11]. (3) Tolok ukur kinerja yang dapat diulang untuk CBDC ritel di atas *permissioned blockchain* masih langka pada sistem yang telah diterapkan maupun pada sistem percontohan [30–32]. (4) Rancangan konseptual perlu diwujudkan sebagai demonstrasi yang dapat diuji secara empiris. Penelitian ini menempati irisan keempat hal tersebut. Tabel 2.1 merangkum posisi penelitian ini terhadap penelitian terdahulu.

**Kebaruan.** Dalam batas korpus yang ditinjau, kontribusi penelitian ini terletak pada integrasi alur distribusi dua jenjang, transaksi ritel, pengawasan, serta protokol Caliper dalam satu artefak Fabric yang dapat diperiksa dan diulang. Kontribusi ini

bukan klaim kebaruan arsitektur CBDC secara universal maupun klaim bahwa platform yang digunakan Proyek Garuda adalah Fabric [11, 30]. Repositori artefak berada di `https://github.com/mfachrizalg/bi-coin`; manifes setiap batch benchmark mencatat commit dan hash masukan agar konfigurasi hasil dapat dilacak.

Tabel 2.1. Perbandingan penelitian terdahulu dengan penelitian ini

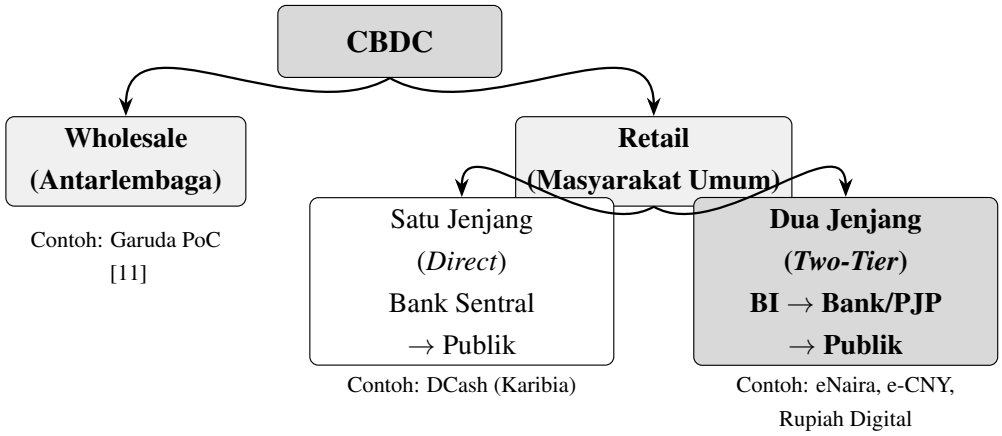
| **Penelitian** | **Platform** | **Fokus utama** | **Integrasi fitur ritel** | **Tolok ukur kinerja** |
|---|---|---|---|---|
| Bhawana & Kumar [15] | Permis sioned | Arsitektur CBDC | Tidak | Terbatas |
| Rohmalia & Nugraha [29] | Hyper ledger Fabric | Purwarupa transfer ritel | Sebagian (akun, transfer) | Waktu pengiriman |
| Islam & In [21] | Con sortium | Privasi (UTXO) | Tidak | Tidak |
| Hans dkk. [24] | Umum | Keamanan (STRIDE) | Tidak | Tidak |
| Auer & Böhme; Tsareva & Komarov [6, 20] | Agnostik | Taksonomi desain | Tidak (konsep.) | Tidak |
| PayOff [44] | Blockchain | Pembayaran tanpa konektivitas | Sebagian | Tidak |
| Block-PAD [39] | Blockchain | Identitas digital | Sebagian | Terbatas |
| Lovejoy dkk. (OpenCBDC) [32] | Account-based (non-DLT) | Kinerja tinggi | Sebagian | Ya (non-blockchain) |
| eNaira [30] | Hyper ledger Fabric | CBDC ritel terdeploy | Ya | Tidak publik |
| **Penelitian ini** | **Hyper ledger Fabric** | **CBDC ritel multipihak** | **Ya** | **Caliper** |

## 2.2 Dasar Teori

### 2.2.1 Central Bank Digital Currency

*Central Bank Digital Currency* (CBDC) adalah representasi digital dari mata uang suatu negara yang diterbitkan oleh dan menjadi kewajiban langsung bank sentral [10]. Sebagai kewajiban bank sentral, CBDC memiliki kualitas bebas risiko yang setara dengan uang tunai, berbeda dengan uang elektronik yang merupakan kewajiban penerbit swasta. Secara umum CBDC dibedakan menjadi dua jenis berdasarkan pengguna sasarannya, yaitu CBDC *wholesale* yang ditujukan untuk transaksi antarlembaga keuangan, dan CBDC *retail* yang ditujukan untuk masyarakat luas [3, 45].

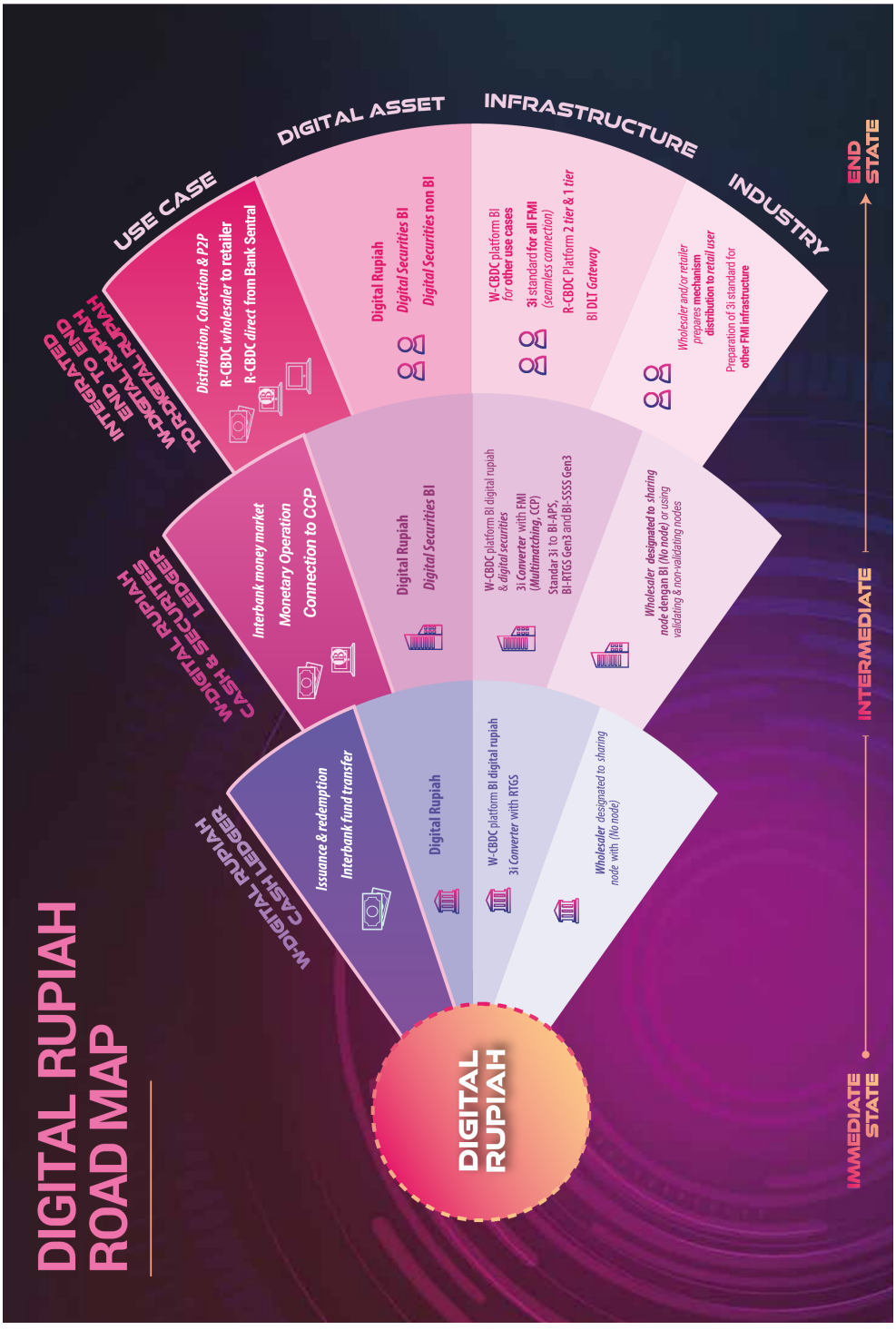
Dalam penerapannya, sebagian besar bank sentral memilih model dua jenjang (*two-tier*), yaitu bank sentral menerbitkan CBDC dan mengelola *ledger* pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui perantara seperti bank dan penyedia jasa pembayaran. Model ini menjaga peran intermediasi lembaga keuangan dan mengurangi risiko *disintermediation* pada sistem perbankan [14]. Di Indonesia, Bank Indonesia mengembangkan CBDC dengan nama Rupiah Digital melalui Proyek Garuda, yang menempuh pendekatan bertahap mulai dari Rupiah Digital *wholesale* kemudian diperluas menuju Rupiah Digital *retail* [5]. Desain Rupiah Digital menekankan interoperabilitas dengan infrastruktur sistem pembayaran dan setelmen, serta penerapan model dua jenjang dengan pihak perantara [11, 12]. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi ditunjukkan pada Gambar 2.1.




Gambar 2.1. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia

Peta jalan resmi Proyek Garuda menempatkan pengembangan Rupiah Digital dalam tiga fase, yaitu *immediate state* untuk w-Digital Rupiah berbasis *cash ledger*, *intermediate state* untuk w-Digital Rupiah dengan kas dan sekuritas digital, serta *end state* yang mengintegrasikan w-Digital Rupiah dengan r-Digital Rupiah. Pada fase *end*

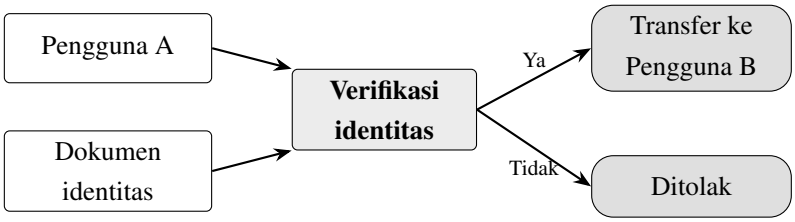
*state*, use case r-Digital Rupiah mencakup distribusi, penagihan, dan pembayaran P2P, baik melalui perantara maupun langsung dari bank sentral sebagaimana ditunjukkan pada Gambar 2.2 [5].



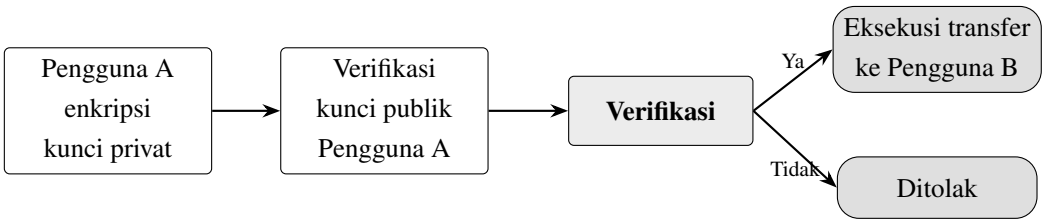

Gambar 2.2. Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [5].

#### 2.2.1.1 Model Pembayaran Berbasis Akun dan Token

Selain dibedakan berdasarkan segmen pengguna dan model distribusi, CBDC juga dapat diklasifikasikan berdasarkan mekanisme pembayaran berbasis akun (*account-based*) atau berbasis token (*token-based*). Pada model berbasis akun, akses pembayaran bergantung pada hubungan antara identitas pengguna, akun atau dompet, dan catatan saldo yang dikelola sistem. Pada model berbasis token, validitas pembayaran terutama bergantung pada verifikasi objek token atau instrumen nilai yang dipindahkan, sehingga penekanan desain berada pada keaslian token dan hak pemegangnya untuk membelanjakan token tersebut [6, 46]. Perbandingan alur verifikasi kedua model ditunjukkan pada Gambar 2.3.




**(a) Berbasis akun (*account-based*)**




**(b) Berbasis token (*token-based*)**

Gambar 2.3. Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6]

White Paper CBDC Bank Indonesia menempatkan Rupiah Digital dalam konfigurasi desain yang mencakup pemisahan w-Digital Rupiah dan r-Digital Rupiah, pilihan satu jenjang atau dua jenjang, akses berbasis token dan akun, serta infrastruktur DLT *permissioned* sebagaimana ditunjukkan pada Gambar 2.4. Konfigurasi tersebut menjadi dasar pembatasan ruang lingkup penelitian ini: purwarupa difokuskan pada model ritel dua jenjang berbasis dompet akun di atas jaringan *permissioned blockchain*, bukan implementasi penuh seluruh konfigurasi Rupiah Digital [5].

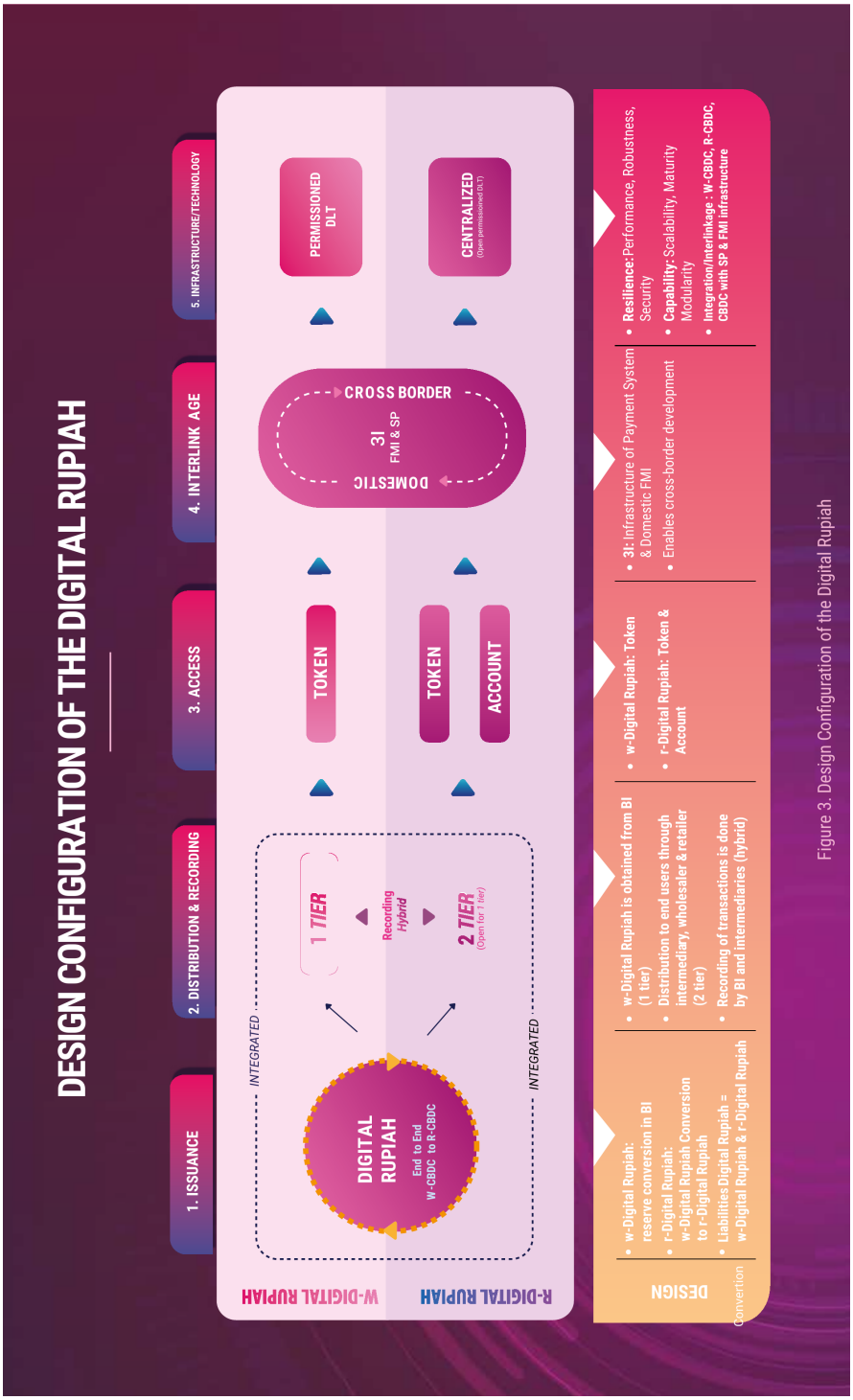



Figure 3. Design Configuration of the Digital Rupiah

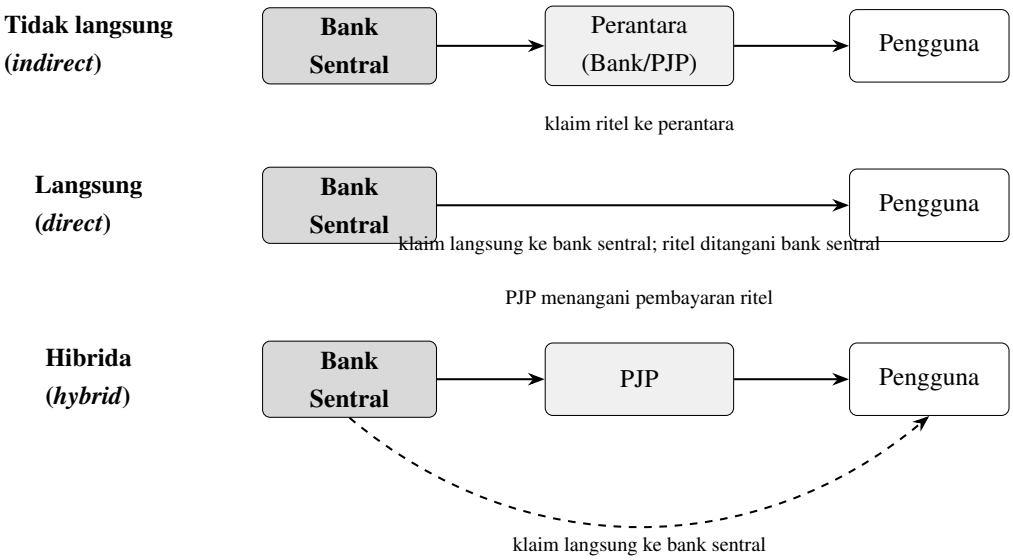
Gambar 2.4. Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].

Pembedaan ini relevan karena memengaruhi desain privasi dan auditabilitas transaksi. Syarifuddin dan Bakhtiar menempatkan *account-based* dan *token-based payment system* sebagai salah satu dimensi desain CBDC yang perlu dipertimbangkan dalam strategi kebijakan moneter [3]. Dalam konteks Indonesia, dokumen konsultatif Rupiah Digital menyatakan bahwa akses pada *wholesale Digital Rupiah* menggunakan

verifikasi berbasis token karena dianggap sesuai untuk transaksi pasar keuangan yang lebih kompleks [12]. Namun, dokumen tersebut menempatkan Rupiah Digital ritel sebagai bagian dari tahap integrasi *end-state*; karena itu, penelitian ini tidak mengklaim bahwa desain pembayaran Rupiah Digital ritel Bank Indonesia telah final sebagai berbasis akun ataupun berbasis token.

#### 2.2.1.2 Arsitektur Penerbitan CBDC

Selain model distribusi satu atau dua jenjang, penerbitan CBDC dikategorikan menurut struktur klaim menjadi tiga arsitektur, yaitu langsung (*direct*), tidak langsung (*indirect*), dan hibrida (*hybrid*) [6]. Pada arsitektur langsung, bank sentral menangani layanan pembayaran ritel secara penuh; pada arsitektur tidak langsung, klaim ritel berada pada perantara yang menjamin CBDC; sedangkan pada arsitektur hibrida, CBDC tetap menjadi klaim langsung bank sentral tetapi perantara mengelola interaksi dengan pengguna. Syarifuddin dan Bakhtiar menempatkan pilihan arsitektur ini sebagai salah satu dimensi desain CBDC yang memengaruhi strategi kebijakan moneter [3]. Ketiga arsitektur tersebut dibandingkan pada Gambar 2.5.



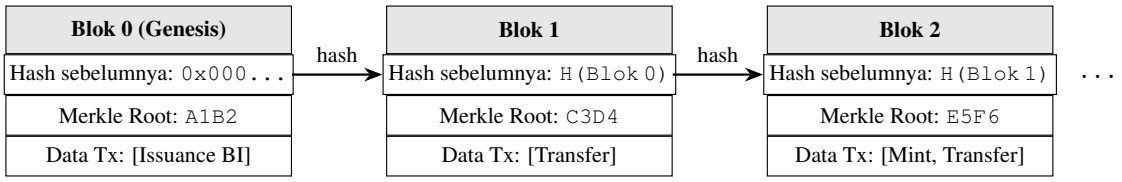

Gambar 2.5. Perbandingan arsitektur penerbitan CBDC: tidak langsung (*indirect*/dua jenjang), langsung (*direct*/satu jenjang), dan hibrida (*hybrid*). Diadaptasi dari Auer dan Böhme [6]

### 2.2.2 Blockchain dan Distributed Ledger Technology

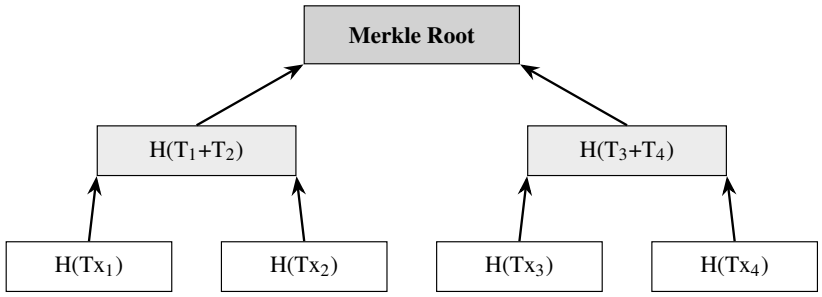
#### 2.2.2.1 Struktur Blockchain

*Distributed Ledger Technology* (DLT) adalah teknologi pencatatan terdistribusi yang memungkinkan banyak pihak memelihara salinan basis data yang konsisten tanpa otoritas terpusat tunggal. *Blockchain* merupakan salah satu bentuk DLT yang menyusun transaksi ke dalam blok-blok yang saling terhubung secara kriptografis. Setiap blok

memuat *hash* dari blok sebelumnya sehingga membentuk riwayat tambah-saja yang perubahan retrospektifnya dapat dideteksi; perubahan pada satu blok akan merusak keterkaitan *hash* pada seluruh blok berikutnya [13]. Sifat ini mendukung integritas dan ketertelusuran data selama mekanisme konsensus dan kendali akses tetap berfungsi. Struktur rantai blok diilustrasikan pada Gambar 2.6. Setiap blok juga memuat *Merkle root*, yaitu nilai hash yang dihasilkan dari pohon Merkle seluruh transaksi dalam blok tersebut, sehingga integritas setiap transaksi dapat diverifikasi secara efisien tanpa harus membaca semua data transaksi [13]. Struktur pohon Merkle ditunjukkan pada Gambar 2.7.



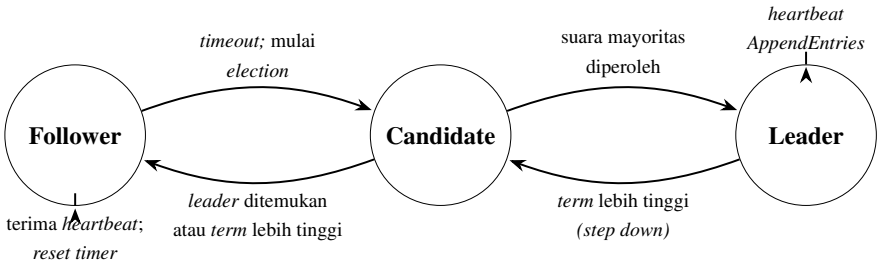

Gambar 2.6. Struktur rantai blok: setiap blok memuat *hash* kriptografis dari blok sebelumnya




Gambar 2.7. Pohon Merkle: *hash* transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok

#### 2.2.2.2 Algoritme Konsensus

Algoritme konsensus menjamin seluruh simpul pada jaringan menyepakati satu urutan transaksi yang sama. Pada *blockchain* publik digunakan mekanisme seperti *proof-of-work* (PoW) dan *proof-of-stake* (PoS) yang dirancang tahan terhadap perilaku jahat namun berbiaya komputasi tinggi. Sebaliknya, *blockchain permissioned* dapat memakai varian *Byzantine Fault Tolerance* (BFT) maupun *Crash Fault Tolerance* (CFT) seperti Raft. Identitas peserta mendukung autentikasi dan atribusi, tetapi pemilihan BFT atau CFT tetap harus mengikuti jenis kegagalan yang diasumsikan [16, 47]. Algoritme Raft membagi peran simpul menjadi tiga *state*: *Leader*, *Candidate*, dan *Follower*. Diagram transisi *state* Raft ditunjukkan pada Gambar 2.8.




Gambar 2.8. Diagram transisi *state* algoritme konsensus Raft: setiap simpul berstatus *Follower*, *Candidate*, atau *Leader*

#### 2.2.2.3 Tipe Blockchain

Berdasarkan model aksesnya, *blockchain* dapat dikelompokkan menjadi *public* (permissionless), *private*, dan *consortium* atau *permissioned* [15]. Pada *public blockchain*, siapa pun dapat bergabung dan berpartisipasi dalam konsensus, sehingga memerlukan mekanisme seperti *proof-of-work* yang mahal secara komputasi. Sebaliknya, pada *permissioned blockchain*, partisipasi dibatasi pada entitas yang teridentifikasi dan memperoleh izin, sehingga konsensus dapat menggunakan algoritme yang lebih efisien dan menghasilkan *throughput* lebih tinggi. Untuk kebutuhan CBDC, *permissioned blockchain* menjadi pilihan yang sesuai karena bank sentral memerlukan kendali tata kelola, identitas peserta yang jelas, dan kinerja yang memadai [14, 43].

### 2.2.3 Hyperledger Fabric

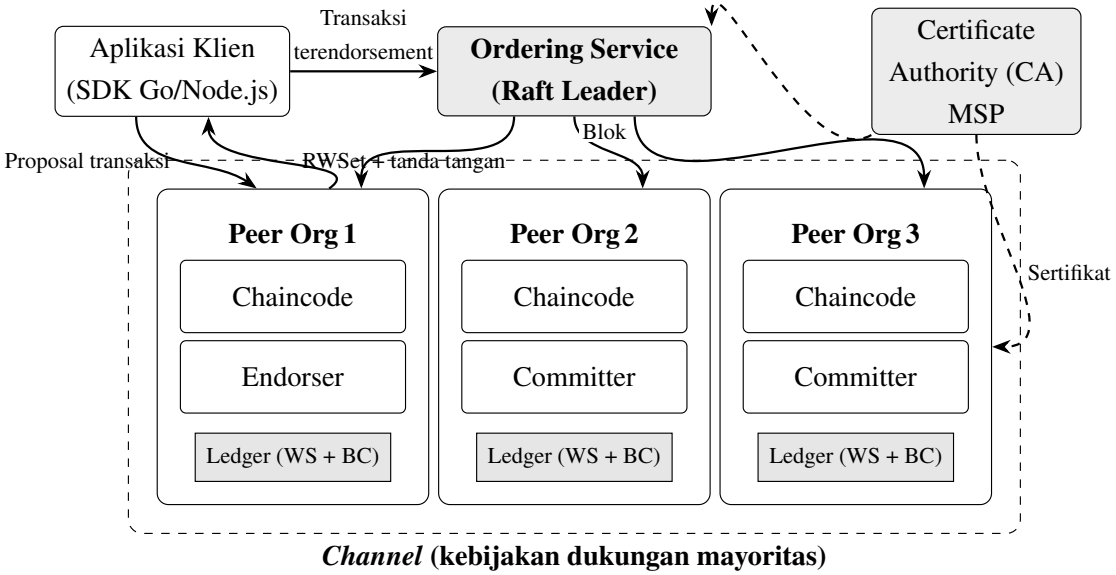
#### 2.2.3.1 Arsitektur Hyperledger Fabric

Hyperledger Fabric adalah kerangka kerja *permissioned blockchain* sumber terbuka di bawah naungan *Hyperledger Foundation* yang dirancang untuk aplikasi perusahaan [16]. Fabric memiliki arsitektur modular dengan komponen utama sebagai berikut.

1. **Peer**, yaitu simpul yang menyimpan salinan *ledger* dan menjalankan *chaincode*. *Peer* berperan sebagai *endorser* yang menyimulasikan transaksi dan sebagai *committer* yang menulis blok ke *ledger*.
2. **Orderer**, yaitu layanan pengurut (*ordering service*) yang menyusun transaksi ke dalam blok dan mendistribusikannya ke *peer*. Fabric mendukung konsensus berbasis Raft (etcdraft) yang bersifat *crash fault tolerant*.
3. **Membership Service Provider** (MSP), yaitu komponen yang mengelola identitas dan hak akses peserta berdasarkan sertifikat digital *X.509* yang diterbitkan oleh *Certificate Authority* (CA).
4. **Channel**, yaitu mekanisme pemisahan jaringan yang memungkinkan subkelompok peserta bertransaksi secara privat dengan *ledger* tersendiri.

5. **Chaincode**, yaitu program *smart contract* yang memuat logika bisnis dan aturan transaksi. Pada Fabric, *chaincode* dapat ditulis dalam bahasa Go, Java, atau Node.js.

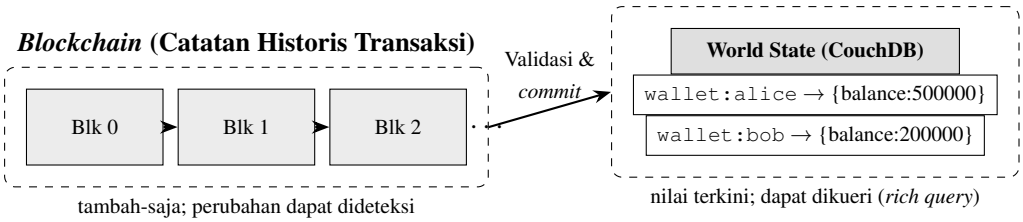
Gambar 2.9 menunjukkan hubungan antara aplikasi klien, *endorsing peer*, *ordering service*, dan *channel* dalam arsitektur Hyperledger Fabric.




Gambar 2.9. Arsitektur komponen Hyperledger Fabric: klien mengirim proposal ke *peer endorser*, mengirim transaksi terendorsement ke *orderer*, lalu menerima komitmen blok pada semua *peer* dalam *channel*.

#### 2.2.3.2 Hyperledger Fabric Ledger

*Ledger* pada Fabric terdiri atas dua bagian, yaitu *world state* yang menyimpan nilai terkini dari setiap kunci (umumnya menggunakan basis data LevelDB atau CouchDB), dan *blockchain* yang menyimpan riwayat transaksi secara berurutan dalam struktur tambah-saja. Penggunaan CouchDB memungkinkan penyimpanan dokumen JSON serta kueri kaya (*rich query*) [48]. Dualitas kedua komponen *ledger* tersebut diilustrasikan pada Gambar 2.10: *blockchain* berfungsi sebagai catatan historis yang perubahan retrospektifnya dapat dideteksi, sedangkan *world state* menyimpan nilai terkini setiap kunci yang dapat dibaca secara langsung dan efisien oleh *chaincode*.




Gambar 2.10. Dualitas komponen *ledger* Hyperledger Fabric: *blockchain* sebagai catatan transaksi historis tambah-saja yang perubahan retrospektifnya dapat dideteksi, dan *world state* sebagai representasi status terkini pada CouchDB

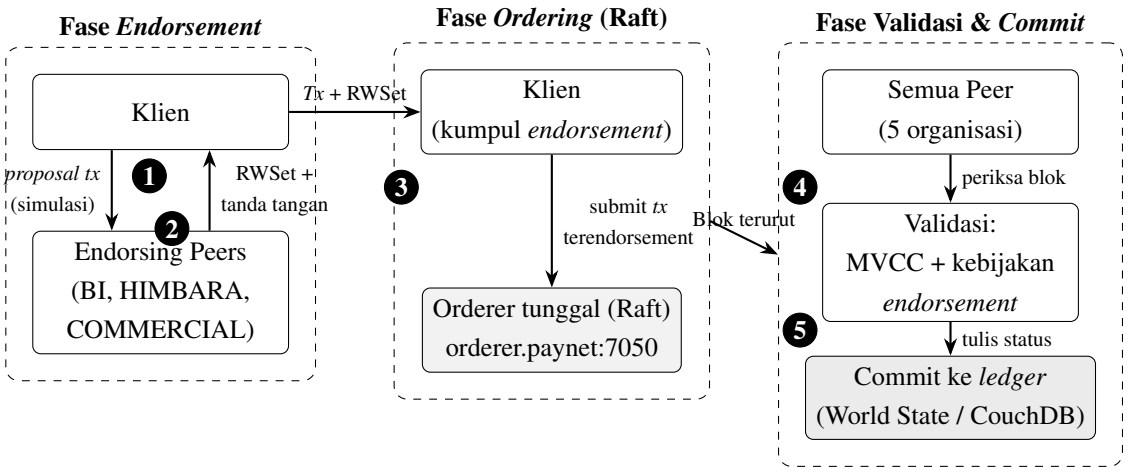
#### 2.2.3.3 Alur Transaksi Hyperledger Fabric

Karakteristik utama Fabric adalah model eksekusi *execute-order-validate* yang membedakannya dari model *order-execute* pada banyak *blockchain* lain. Alur transaksi Fabric berlangsung dalam lima tahap. (1) Aplikasi klien mengirim usulan transaksi (*transaction proposal*) ke sejumlah *peer endorser*. (2) *Endorser* menyimulasikan transaksi terhadap *chaincode* dan mengembalikan tanda tangan dukungan (*endorsement*) beserta himpunan baca-tulis (*read-write set*). (3) Klien mengumpulkan dukungan sesuai kebijakan dukungan (*endorsement policy*), lalu mengirimkannya ke *ordering service*. (4) *Orderer* mengurutkan transaksi ke dalam blok. (5) Setiap *peer* memvalidasi transaksi dengan memeriksa kebijakan dukungan dan konflik versi, kemudian menuliskannya ke *ledger* [16]. Model ini meningkatkan skalabilitas karena eksekusi *chaincode* dilakukan secara paralel sebelum pengurutan.

Pemeriksaan konflik versi pada tahap validasi menerapkan kontrol konkurensi multiversi (*Multiversion Concurrency Control*, MVCC). Setiap transaksi membawa himpunan baca (*read set*) berisi versi kunci status yang dibaca ketika endorsemen. Sebelum *commit*, *peer* membandingkan versi tersebut dengan versi terkini pada *ledger*. Apabila sebuah kunci telah berubah oleh transaksi lain dalam blok yang sama atau blok sebelumnya, versi tidak lagi cocok sehingga transaksi ditandai tidak valid sebagai konflik MVCC meskipun endorsemennya sah [16]. Mekanisme ini menjaga konsistensi status pada penulisan bersamaan dan menjadi dasar analisis perebutan dompet (*hot-wallet contention*) pada BAB IV.

*Endorsement policy* menentukan organisasi mana yang dukungannya diperlukan agar suatu transaksi dianggap sah. Pada jaringan multipihak, kebijakan seperti *majority* mensyaratkan dukungan dari mayoritas organisasi, sehingga tidak ada satu pihak pun yang dapat memutakhirkan *ledger* secara sepihak.

Gambar 2.11 mengilustrasikan siklus transaksi Hyperledger Fabric secara lengkap, mulai dari *proposal* ke *endorsing peer* hingga *commit* ke *ledger* melalui mekanisme konsensus Raft.




Gambar 2.11. Siklus transaksi Hyperledger Fabric pada topologi uji dengan satu *consenter* Raft tanpa toleransi kegagalan *orderer*.

### 2.2.4 Hyperledger Caliper

Hyperledger Caliper adalah kerangka kerja *benchmark* kinerja *blockchain* yang memungkinkan pengukuran kinerja terhadap sistem *blockchain* dengan beban kerja yang dapat dikustomisasi [1]. Caliper menghasilkan beban transaksi melalui sejumlah proses pekerja (*worker*) dan mengukur metrik kinerja utama berdasarkan metrik standar *Hyperledger Performance and Scale Working Group* [49], antara lain:

1. **Throughput**, yaitu jumlah transaksi yang berhasil diproses per satuan waktu, dinyatakan dalam *transactions per second* (TPS).
2. **Latency**, yaitu selang waktu antara pengiriman transaksi hingga transaksi tersebut tercatat dan terkonfirmasi pada *ledger*.
3. **Send rate**, yaitu laju pengiriman transaksi yang dikendalikan oleh *rate controller*.

Caliper juga dapat memantau penggunaan sumber daya *container* seperti penggunaan CPU dan memori selama pengujian, sehingga sesuai untuk mengevaluasi kelayakan kinerja sistem CBDC [33].

### 2.2.5 Transfer Ritel Langsung

Unit pembayaran pada purwarupa adalah fungsi `Transfer` yang sama untuk transfer antarpelanggan dan pelanggan-ke-pedagang. Penyatuan jalur ini menjaga pemeriksaan otorisasi, saldo, dan pencatatan audit tetap deterministik. Penelitian tidak memodelkan standar instrumen pembayaran atau proses akseptasi eksternal; fokusnya adalah kebijakan buku besar CBDC ritel.

### 2.2.6 Application Programming Interface

*Application Programming Interface* (API) adalah antarmuka yang memungkinkan dua perangkat lunak saling berkomunikasi melalui kontrak yang terdefinisi. Sistem yang dibangun menggunakan *REST API* (*Representational State Transfer*) [50] sebagai antarmuka komunikasi menuju lapisan *backend* melalui protokol HTTP. API ini menjembatani permintaan pengguna atau alat uji ke *chaincode* melalui *Fabric Gateway SDK* tanpa menjadikan antarmuka pengguna sebagai bagian kontribusi penelitian.

### 2.2.7 Bahasa Pemrograman Go

Go adalah bahasa pemrograman terkompilasi yang dikembangkan oleh Google dengan penekanan pada kesederhanaan dan dukungan konkurensi melalui *goroutine* [51]. Pada penelitian ini Go digunakan untuk menulis *chaincode* dan *backend* karena dukungannya yang kuat terhadap Hyperledger Fabric melalui *Fabric Contract API* dan *Fabric Gateway SDK*, serta kinerja konkurensi yang baik untuk melayani banyak permintaan secara bersamaan.

## 2.3 Analisis Perbandingan Metode

Empat keputusan desain utama penelitian ini dibandingkan secara sistematis: pilihan antara DLT dan sistem terpusat, pemilihan platform DLT, pemilihan algoritme konsensus, dan pemilihan model arsitektur CBDC. Tabel perbandingan memperlihatkan perbedaan antar-alternatif secara eksplisit sebagai dasar pertanggungjawaban ilmiah.

### 2.3.1 Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain

**DLT versus Sistem Terpusat.** Tabel 2.2 membandingkan tiga pendekatan infrastruktur untuk sistem pembayaran CBDC: sistem terpusat tradisional (non-DLT), *permissionless blockchain* (publik), dan *permissioned blockchain* (DLT berizin). Sistem terpusat menawarkan *throughput* sangat tinggi tetapi bergantung pada otoritas tunggal dan tidak memiliki auditabilitas multipartit secara inheren. *Permissionless blockchain* menyediakan transparansi tetapi *throughput*-nya rendah akibat mekanisme konsensus berbasis *proof-of-work* [14, 43]. DLT *permissioned* menggabungkan auditabilitas multipartit, kendali tata kelola yang dapat dikonfigurasi, dan *throughput* tinggi tanpa konsumsi energi besar [16].

Tabel 2.2. Perbandingan sistem terpusat, *permissionless blockchain*, dan *permissioned blockchain* untuk CBDC

| **Kriteria** | **Sistem Terpusat (non-DLT)** | **Permissionless Blockchain** | **Permissioned Blockchain** |
|---|---|---|---|
| Kekebalan data | Tidak (admin dapat ubah) | Ya | Ya |
| Auditabilitas multipartit | Terbatas (otoritas tunggal) | Ya (publik) | Ya (terkontrol) |
| Kendali tata kelola | Penuh terpusat | Tidak ada | Dapat dikonfigurasi |
| *Throughput* | >50.000 TPS (indikatif) | 7–50 TPS (indikatif) | >3.500 TPS [16] |
| Konsumsi energi | Rendah | Sangat tinggi (PoW) | Rendah |
| *Smart contract* | Opsional | Ya | Ya |
| Kapabilitas pemisahan data | Ya | Tidak (publik) | Tersedia melalui *channel*/data privat |
| Contoh artefak CBDC yang dilaporkan | OpenCBDC [32] | Tidak direkomendasikan | eNaira [30] |

**Simpulan:** *Permissioned blockchain* dipilih karena menyeimbangkan auditabilitas multipartit yang tidak dapat diperoleh dari sistem terpusat, kendali tata kelola bank sentral yang tidak dimiliki *permissionless blockchain*, dan *throughput* tinggi yang dibutuhkan sistem pembayaran [14, 43].

**Perbandingan Platform DLT.** Tabel 2.3 membandingkan empat platform DLT *permissioned* yang lazim digunakan dalam sistem keuangan dan CBDC: Hyperledger Fabric, R3 Corda, Quorum/Hyperledger Besu, dan Hyperledger Sawtooth [14, 16, 52].

Tabel 2.3. Perbandingan platform DLT *permissioned* untuk sistem CBDC

| **Kriteria** | **Hyperledger Fabric** | **R3 Corda** | **Quorum/ Besu** | **H. Sawtooth** |
|---|---|---|---|---|
| Konsensus | Raft (CFT), *pluggable* | Notary (CFT) | IBFT/Clique | PoET, Raft |
| Model eksekusi | *Execute-order-validate* | *Flow-based* | *Order-execute* | *Order-execute* |
| Bahasa *smart contract* | Go, Java, Node.js | Kotlin, Java | Solidity (EVM) | Python, Go, Java |
| Manajemen identitas | MSP + X.509 CA | X.500 PKI | Berbasis akun | Berbasis peran |
| Privasi | *Channel*, *private data* | *Need-to-know* | TX privat | Terbatas |
| *Throughput* | >3.500 TPS [16] | ∼170 TPS [52] | ∼300 TPS (indikatif) | ∼1.000 TPS (indikatif) |
| Penerapan CBDC yang dilaporkan | eNaira [30] | PoC Proyek Garuda [11] | PoC Proyek Garuda [11] | Tidak |

**Simpulan:** Hyperledger Fabric dipilih karena identitas organisasi berbasis MSP, model eksekusi modular, dukungan kontrak Go, dan integrasi langsung dengan alat uji Caliper. MSP mengautentikasi organisasi jaringan. *Channel* dan data privat merupakan kapabilitas platform, tetapi purwarupa penelitian memakai satu *application channel* dan tidak memisahkan buku besar *wholesale* dan ritel. PoC Proyek Garuda menggunakan Corda dan Hyperledger Besu; laporan itu menjadi pembanding kebutuhan Indonesia, bukan bukti penerapan Fabric [11].

Sebagai pembanding eksternal, Urbančok membandingkan kinerja transfer aset pada Ethereum, Hyperledger Fabric, dan Besu menggunakan skenario *send rate* 10, 20, 50, dan 100 TPS [7]. Hasil pada Gambar 2.12 menunjukkan bahwa Fabric kompetitif untuk beban transfer aset dan unggul pada beberapa titik uji, tetapi angka tersebut tidak digunakan sebagai hasil evaluasi purwarupa penelitian ini karena konfigurasi jaringan, versi perangkat lunak, dan beban kerja berbeda.

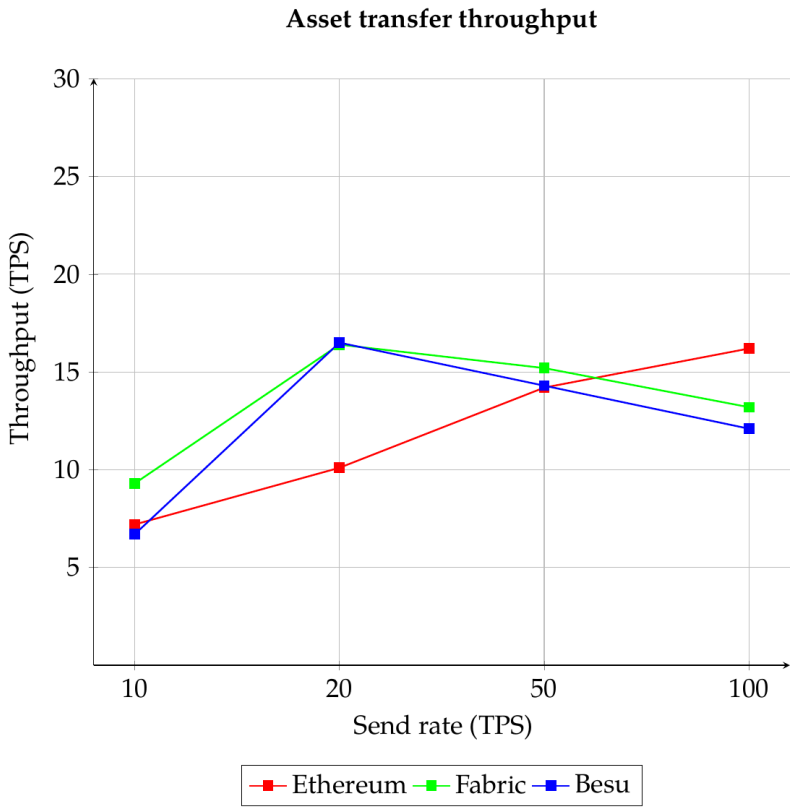



Figure 8.6: Throughput comparison of asset transfer

Gambar 2.12. Perbandingan *throughput* transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [7].

### 2.3.2 Pemilihan Algoritme Konsensus Hyperledger Fabric

Tabel 2.4 membandingkan algoritme konsensus yang relevan untuk jaringan *blockchain permissioned*. Identitas peserta mendukung autentikasi dan atribusi tindakan, tetapi tidak menghilangkan perilaku *Byzantine* dari simpul yang terkompromi. Finalisasi deterministik tetap diperlukan karena sistem keuangan tidak dapat mentoleransi ketidakpastian *fork* [16, 47].

Tabel 2.4. Perbandingan algoritme konsensus untuk *blockchain permissioned*

| **Algoritme** | **Toleransi kesalahan** | **Kompleksitas pesan** | **Throughput** | **Latensi** | **Finalisasi** |
|---|---|---|---|---|---|
| Raft | CFT | $O(n)$ | Tinggi | Rendah | Deterministik |
| PBFT | BFT | $O(n^2)$ | Menengah | Menengah | Deterministik |
| HotStuff | BFT | $O(n)$ | Menengah | Rendah | Deterministik |
| PoW | — | $O(n)$ | Sangat rendah | Sangat tinggi | Probabilistik |
| PoS/PoA | BFT/CFT | $O(n)$ | Menengah | Menengah | Semi-deterministik |

**Simpulan:** Raft (*etcdraft*) dipilih karena didukung secara *native* oleh Hyperledger Fabric dan memberi finalisasi deterministik dalam model *crash fault* [16]. Identitas MSP menyediakan autentikasi dan atribusi, tetapi tidak mencegah perilaku *Byzantine* dari simpul yang terkompromi. Selain itu, purwarupa hanya memakai satu *consenter*; konfigurasi ini tidak menoleransi satu pun kegagalan *orderer*. Penelitian karena itu mengevaluasi pilihan *ordering* purwarupa, bukan mendemonstrasikan toleransi kegagalan.

### 2.3.3 Pemilihan Alat Pengujian Blockchain

Hyperledger Caliper dan BlockBench sama-sama dapat mengevaluasi jaringan *blockchain permissioned*, tetapi memiliki cakupan berbeda. Caliper menjalankan beban kerja yang dikonfigurasi terhadap kontrak aktual dan melaporkan metrik tingkat sistem, sedangkan BlockBench dirancang untuk membandingkan kinerja keseluruhan dan komponen internal beberapa platform [1, 2]. Perbandingan ringkas disajikan pada Tabel 2.5.

Tabel 2.5. Perbandingan Hyperledger Caliper dan BlockBench berdasarkan [1, 2].

| **Kriteria** | **Hyperledger Caliper** | **BlockBench** |
|---|---|---|
| Fokus | Pengujian jaringan dengan beban kerja dan kontrak yang dapat dikonfigurasi | Perbandingan lintas platform pada tingkat makro dan mikro |
| Metrik utama | Tingkat keberhasilan, *throughput*, latensi, dan penggunaan sumber daya | *Throughput*, latensi, skalabilitas, dan toleransi kegagalan |
| Beban kerja | Modul beban kerja yang memanggil fungsi kontrak aktual | Kontrak pintar nyata dan sintetis melalui API benchmark |
| Kesesuaian | Integrasi langsung dengan jaringan Fabric dan *chaincode* penelitian | Tepat untuk analisis komponen dan perbandingan beberapa platform |

**Simpulan:** Hyperledger Caliper dipilih karena tujuan penelitian adalah mengukur perilaku *chaincode* CBDC ritel aktual pada satu jaringan Fabric, bukan membandingkan implementasi internal beberapa platform.

### 2.3.4 Pemilihan Arsitektur CBDC Dua Jenjang

Istilah “dua jenjang” pada bagian ini menyatakan pembagian peran distribusi antara bank sentral dan perantara.

Tabel 2.6 membandingkan model arsitektur satu jenjang (*one-tier/direct*), di mana bank sentral mendistribusikan CBDC langsung kepada publik, dengan model dua jenjang (*two-tier*), di mana distribusi dilakukan melalui bank dan penyedia jasa pembayaran (PJP) [6, 14, 20].

Tabel 2.6. Perbandingan arsitektur CBDC satu jenjang dan dua jenjang

| **Kriteria** | **Satu Jenjang (*Direct*)** | **Dua Jenjang (*Two-Tier*)** |
|---|---|---|
| Distribusi CBDC | Bank sentral langsung ke publik | Bank sentral $\rightarrow$ bank/PJP $\rightarrow$ publik |
| Risiko *disintermediation* | Tinggi | Rendah [14] |
| Beban infrastruktur bank sentral | Sangat tinggi | Dibagi ke perantara |
| Privasi pengguna dari bank sentral | Bergantung pada desain implementasi | Bergantung pada pemisahan data dan hak akses; bukan sifat otomatis dua jenjang |
| Interoperabilitas ekosistem | Sulit | Mudah (bank/PJP sudah terintegrasi) |
| Contoh CBDC nyata | DCash (Karibia) | eNaira, e-CNY, Sand Dollar [30] |
| Keselarasan Proyek Garuda | Tidak | Ya [5, 11] |

**Simpulan:** Model dua jenjang dipilih berdasarkan tiga pertimbangan. (1) Model ini selaras dengan desain Rupiah Digital Bank Indonesia dalam Proyek Garuda [11]. (2) Pembagian peran antarlapis meminimalkan risiko *disintermediation* perbankan yang menjadi kekhawatiran utama bank sentral [14]. (3) Volume transaksi *retail* skala masif tidak membebani *ledger wholesale* [6, 20].

### 2.3.5 Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal

Syarifuddin dan Bakhtiar merumuskan desain CBDC optimal pendukung strategi kebijakan moneter ke dalam beberapa dimensi: cakupan ritel dan *wholesale*; remunerasi berbunga (*wholesale*) dan tanpa bunga (ritel); model pembayaran berbasis akun dan token; derajat anonimitas yang terlacak; arsitektur hibrida; sistem buku besar berbasis DLT; serta cakupan domestik dan lintas batas [3, 27]. Karena penelitian ini berfokus pada perancangan, implementasi, dan evaluasi kinerja fungsional, dimensi yang bersifat

makroekonomi dan kebijakan moneter berada di luar cakupan; Tabel 2.7 memetakan pemenuhan tiap dimensi pada purwarupa beserta justifikasinya.

Tabel 2.7. Pemetaan purwarupa terhadap dimensi desain CBDC optimal [3]

| **Dimensi desain optimal** | **Status purwarupa** | **Keterangan** |
|---|---|---|
| Sistem buku besar DLT | Terpenuhi | Hyperledger Fabric 2.5 sebagai buku besar terdistribusi *permissioned* |
| Arsitektur hibrida | Tidak didemonstrasikan | Distribusi dua jenjang diimplementasikan pada satu buku besar bersama; pembagian tanggung jawab pencatatan hibrida tidak diuji |
| Cakupan ritel dan *wholesale* | Ritel saja | Dompet institusional memodelkan distribusi likuiditas, bukan instrumen atau buku besar *wholesale* yang terpisah |
| Model akun dan token | Sebagian | Implementasi berbasis akun (saldo); dimensi token dibahas namun tidak diklaim final (lihat Subbab Model Pembayaran Berbasis Akun dan Token) |
| Remunerasi ritel tanpa bunga | Konsisten | Rupiah Digital ritel purwarupa bersifat non-remunerasi, selaras dengan rekomendasi CBDC ritel optimal [3] |
| Remunerasi *wholesale* berbunga / *CBDC rate* | Di luar cakupan | Instrumen kebijakan moneter; tidak dimodelkan pada purwarupa fungsional |
| Anonimitas terlacak | Di luar cakupan | Privasi transaksi tidak diimplementasikan; lihat batasan privasi pada Bab I |
| Cakupan domestik dan lintas batas | Sebagian | Domestik; interkoneksi CBDC lintas batas merupakan kesenjangan prinsip 3i |

**Simpulan:** purwarupa mengimplementasikan buku besar DLT, distribusi dua jenjang, fungsi ritel berbasis saldo, dan sifat non-remunerasi. Arsitektur hibrida, instrumen *wholesale*, *CBDC rate*, anonimitas terlacak, serta interkoneksi lintas batas tidak didemonstrasikan. Pemisahan ini mencegah model distribusi, cakupan instrumen, dan bentuk arsitektur diperlakukan sebagai dimensi yang sama [3, 27].

Aryani [28] mengusulkan arsitektur dua jenjang Digital Rupiah dengan lapisan tambahan berupa *Digital Rupiah Depository* sebagai entitas perantara antara Bank

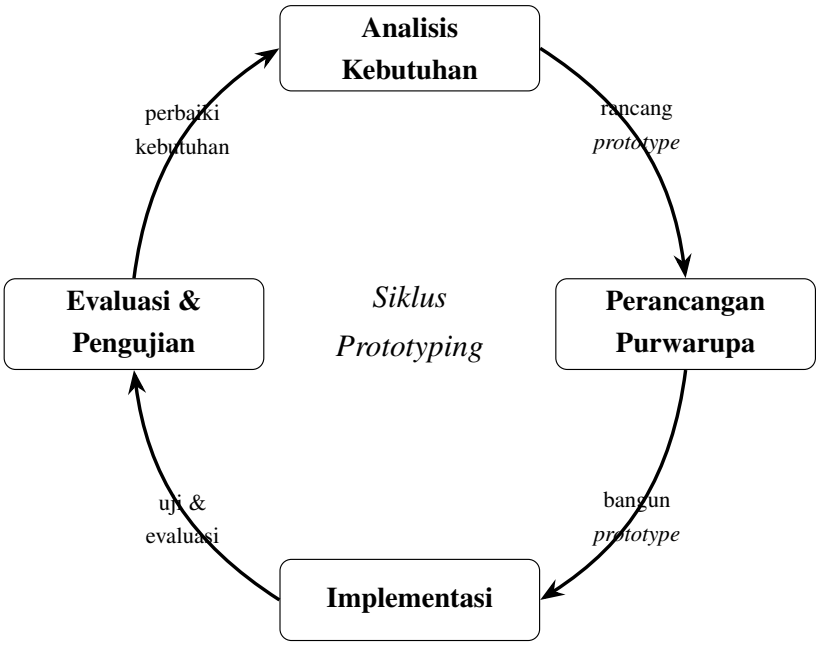
Indonesia dan peserta jaringan, pemisahan *wholesaler* dan *non-wholesaler* di jenjang atas, serta koneksi ke BI-RTGS dan jaringan *blockchain* lintas batas untuk interoperabilitas. Perbandingan ini menjadi rujukan konseptual untuk menempatkan purwarupa pada spektrum desain CBDC.

Penelitian ini mengadopsi prinsip inti arsitektur tersebut, yaitu distribusi berjenjang melalui bank validator dan PJP, namun dengan dua penyederhanaan yang disengaja sesuai cakupan purwarupa. Pertama, lapisan *Digital Rupiah Depository* dan pemisahan *wholesaler/non-wholesaler* diciutkan menjadi dua organisasi validator (HimbaraBankOrg dan CommercialBankOrg) yang langsung menerima penerbitan dari BankIndonesiaOrg. Kedua, integrasi BI-RTGS dan jaringan *blockchain* lintas batas tidak diimplementasikan dan dinyatakan secara eksplisit sebagai batasan cakupan penelitian (Bab I).

# BAB III
# METODE PENELITIAN

## 3.1 Metode yang Digunakan

Metode yang digunakan adalah *prototyping* dengan tahapan perancangan, implementasi, dan pengujian. Evaluasi utama memverifikasi kontrol terpilih melalui skenario fungsional dan kebijakan *endorsement*, serta mengukur kapasitas lokal tiga *chaincode* eksperimen pada jaringan empat MSP menggunakan Hyperledger Caliper. Siklus SDLC metode *prototyping* yang diterapkan ditunjukkan pada Gambar 3.13: keempat fase (analisis kebutuhan, perancangan, implementasi, dan evaluasi) diulang secara iteratif hingga purwarupa memenuhi spesifikasi.




Gambar 3.13. Siklus SDLC dengan metode *prototyping*: empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja

### 3.1.1 Metode Perancangan

Perancangan sistem dimulai dari analisis kebutuhan berdasarkan model Rupiah Digital dua jenjang, kemudian dituangkan ke dalam activity diagram, arsitektur sistem, arsitektur jaringan, perancangan use case, dan model data *ledger*.

#### 3.1.1.1 Analisis Kebutuhan Sistem

Analisis kebutuhan mengidentifikasi aktor sistem (Bank Indonesia, bank validator, PJP, pedagang, pelanggan ritel, supervisor, dan pengamat OJK) beserta kebutuhan fungsionalnya, yaitu penerbitan dan distribusi likuiditas berjenjang, transfer

antarpelanggan serta pelanggan-ke-pedagang, pengawasan, dan pelaporan. Kebutuhan ini menjadi dasar perancangan activity diagram dan arsitektur pada subbab berikutnya.

#### 3.1.1.2 Spesifikasi Kebutuhan Perangkat Lunak

Analisis kebutuhan tersebut diformalkan dalam bentuk Spesifikasi Kebutuhan Perangkat Lunak (*Software Requirements Specification*/SRS) yang diadaptasi ke struktur tesis ini. SRS digunakan untuk menetapkan ruang lingkup purwarupa, aktor dan batas wewenangnya, antarmuka utama sistem, kebutuhan fungsional, dan kebutuhan nonfungsional sebelum implementasi dilakukan. Pada penelitian ini, ruang lingkup perangkat lunak mencakup empat antarmuka utama, yaitu aplikasi klien, *backend API*, basis data luring PostgreSQL, dan jaringan *blockchain* Hyperledger Fabric beserta *chaincode*-nya. Aplikasi klien hanya berfungsi sebagai alat bantu demonstrasi dan tidak termasuk dalam evaluasi penelitian ini.

Pemangku kepentingan utama terdiri atas Bank Indonesia sebagai penerbit dan pengawas, bank validator dan PJP sebagai kustodian institusional, pedagang dan pelanggan ritel sebagai pengguna transaksi, serta supervisor dan pengamat OJK sebagai pihak yang membutuhkan fungsi baca dan pelaporan. Setiap pemangku kepentingan tersebut kemudian diterjemahkan menjadi peran akses pada *backend API* dan aturan bisnis pada *chaincode* agar kebutuhan operasional dapat ditelusuri secara konsisten.

Tabel 3.1. Kebutuhan fungsional purwarupa bi-coin

| ID | Deskripsi kebutuhan |
|---|---|
| FR-01 | Sistem harus mendukung pendaftaran, persetujuan, pembekuan, pencairan, dan penghentian peserta sesuai peran Bank Indonesia, bank validator, dan PJP. |
| FR-02 | Sistem harus mendukung penerbitan Digital Rupiah oleh Bank Indonesia ke dalam Treasury dan distribusi langsung kepada bank validator atau PJP sebagai kustodian institusional. |
| FR-03 | Sistem harus menolak transfer yang tidak diotorisasi, melibatkan dompet beku, bernilai tidak valid, atau melebihi saldo pengirim, lalu memperbarui saldo secara atomik. |
| FR-04 | Sistem harus menyediakan laporan rekonsiliasi, metrik, riwayat transaksi, dan *event* pengawasan bagi peran yang berwenang. |
| FR-05 | Sistem harus menegakkan pemisahan hak akses pada *backend API* antara peran publik, terautentikasi, PJP/bank, merchant, dan supervisor. |

Tabel 3.2. Kebutuhan nonfungsional purwarupa bi-coin

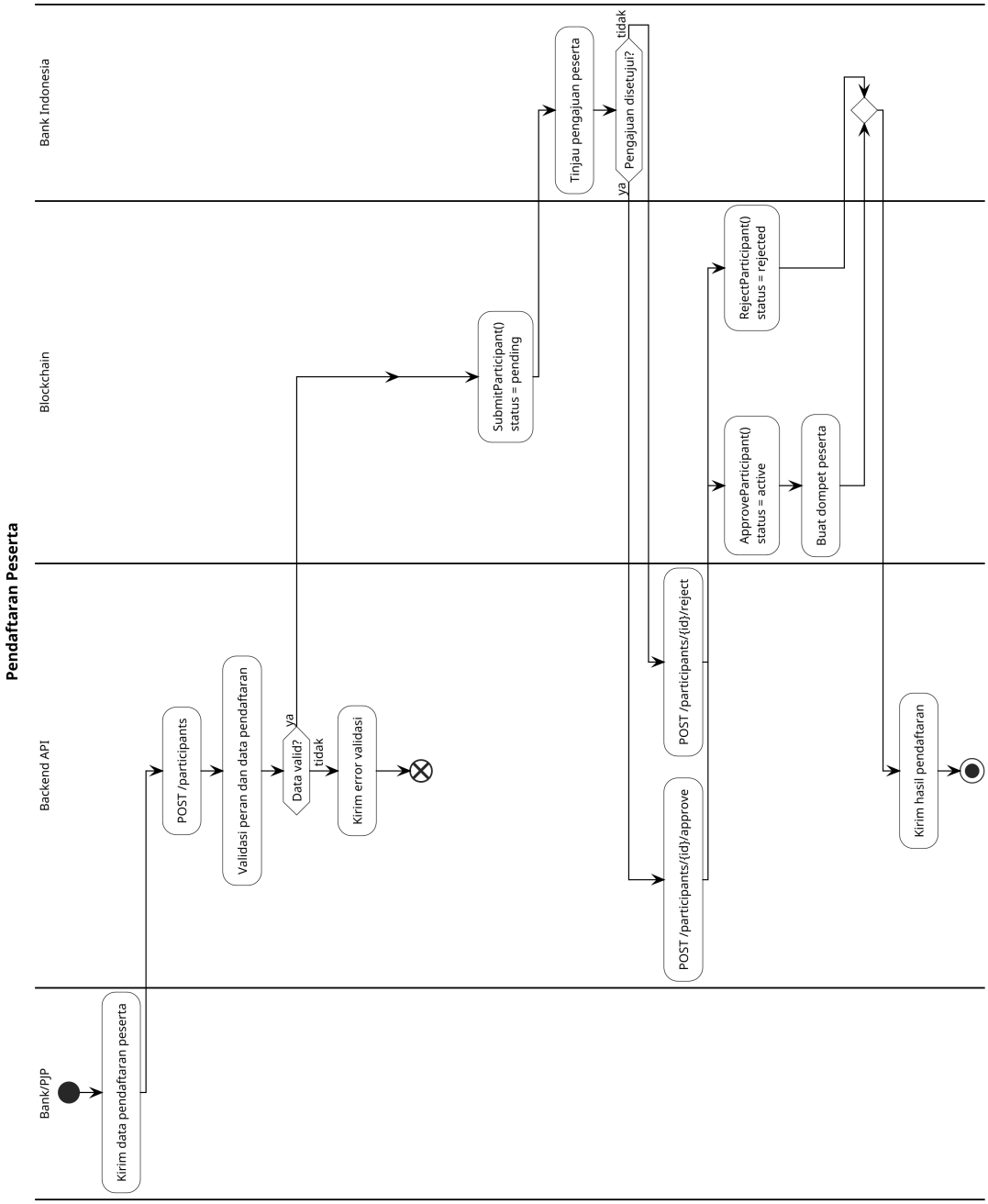
| ID | Deskripsi kebutuhan |
|---|---|
| NFR-01 | Sistem harus berjalan pada jaringan *permissioned* Hyperledger Fabric yang mendukung partisipasi multipihak dan kebijakan *endorsement* organisasi yang ditetapkan pada kanal. |
| NFR-02 | Sistem harus menyediakan keterlacakan melalui catatan audit, *event* pengawasan, dan riwayat transaksi yang dapat ditelusuri kembali ke peserta terkait. |
| NFR-03 | Sistem harus memisahkan antarmuka klien, *backend API*, basis data luring, dan *chaincode* agar kontrol akses dan integrasi dapat dikelola secara modular. |
| NFR-04 | Sistem harus dapat dievaluasi kinerjanya secara kuantitatif melalui metrik *throughput*, latensi, dan tingkat keberhasilan transaksi menggunakan Hyperledger Caliper. |
| NFR-05 | Sistem harus mendukung pengulangan eksperimen pada lingkungan yang sama melalui konfigurasi, *seed*, hash artefak, topologi jaringan, dan skenario uji yang terdokumentasi. |

Tabel 3.3. Keterlacakan kebutuhan terhadap metode verifikasi

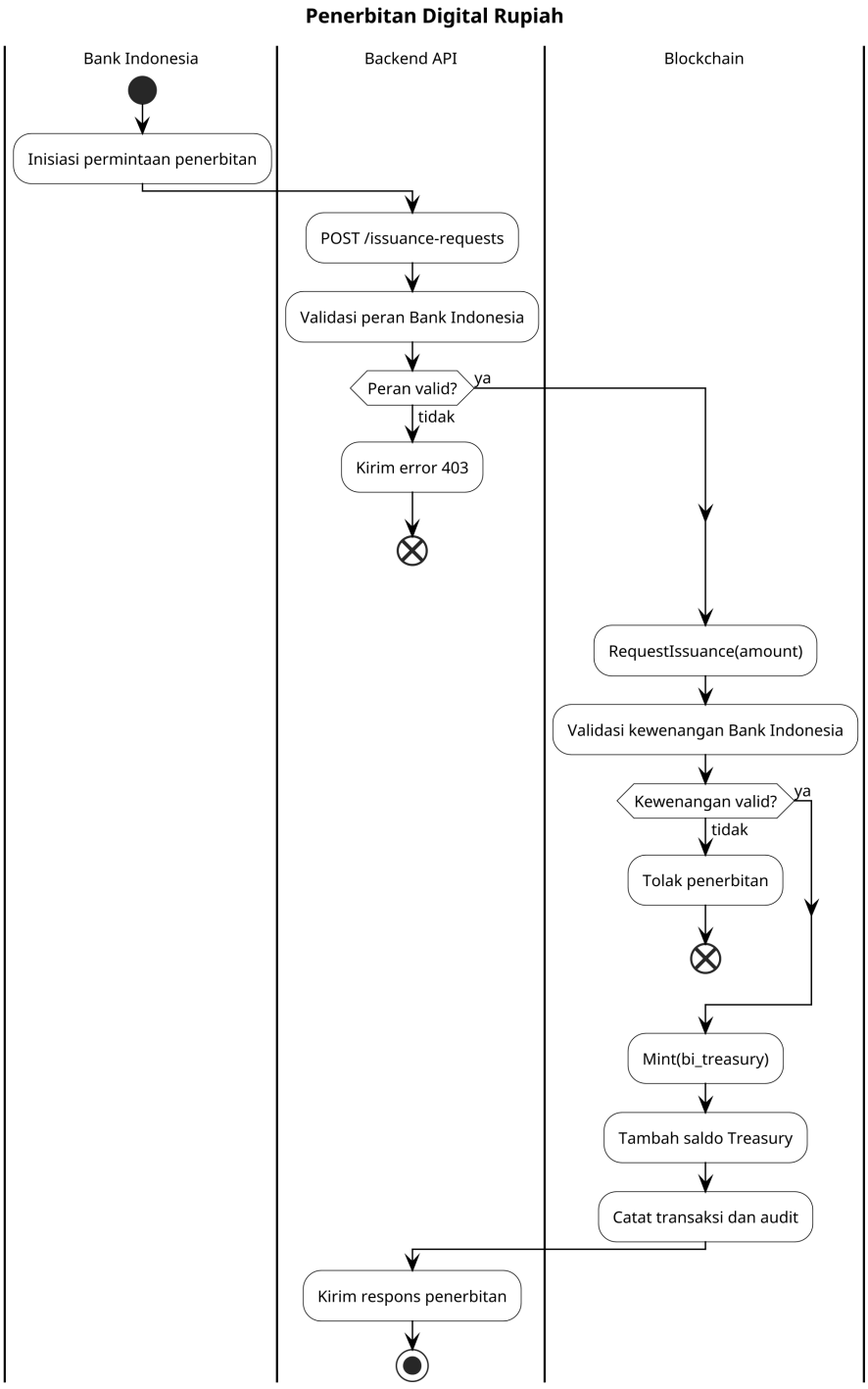
| **ID kebutuhan** | **Metode verifikasi utama** | **Bukti yang dituju** |
|---|---|---|
| FR-01, FR-02 | Skenario fungsional dan *unit test chaincode* | Siklus hidup peserta, penerbitan, dan distribusi dua jenjang berjalan sesuai aturan |
| FR-03 | *Unit test backend* dan *chaincode* serta skenario transfer | Otorisasi, status beku, validasi saldo, dan pembaruan atomik ditegakkan konsisten |
| FR-04, FR-05 | Skenario fungsional, *unit test backend*, dan inspeksi rute API | Hak akses, pelaporan, dan *event* pengawasan tersedia bagi peran berwenang |
| NFR-01, NFR-03, NFR-05 | Inspeksi arsitektur, konfigurasi, dan manifes benchmark | Struktur jaringan, pemisahan lapisan, dan *repeatability* purwarupa terdokumentasi |
| NFR-02 | Inspeksi model data dan *unit test* terpilih | Keterlacakan audit dan riwayat transaksi dapat dipertahankan |
| NFR-04 | *Benchmark* Hyperledger Caliper | Nilai *throughput*, latensi, dan tingkat keberhasilan transaksi diperoleh secara kuantitatif |

#### 3.1.1.3 Perancangan Activity Diagram

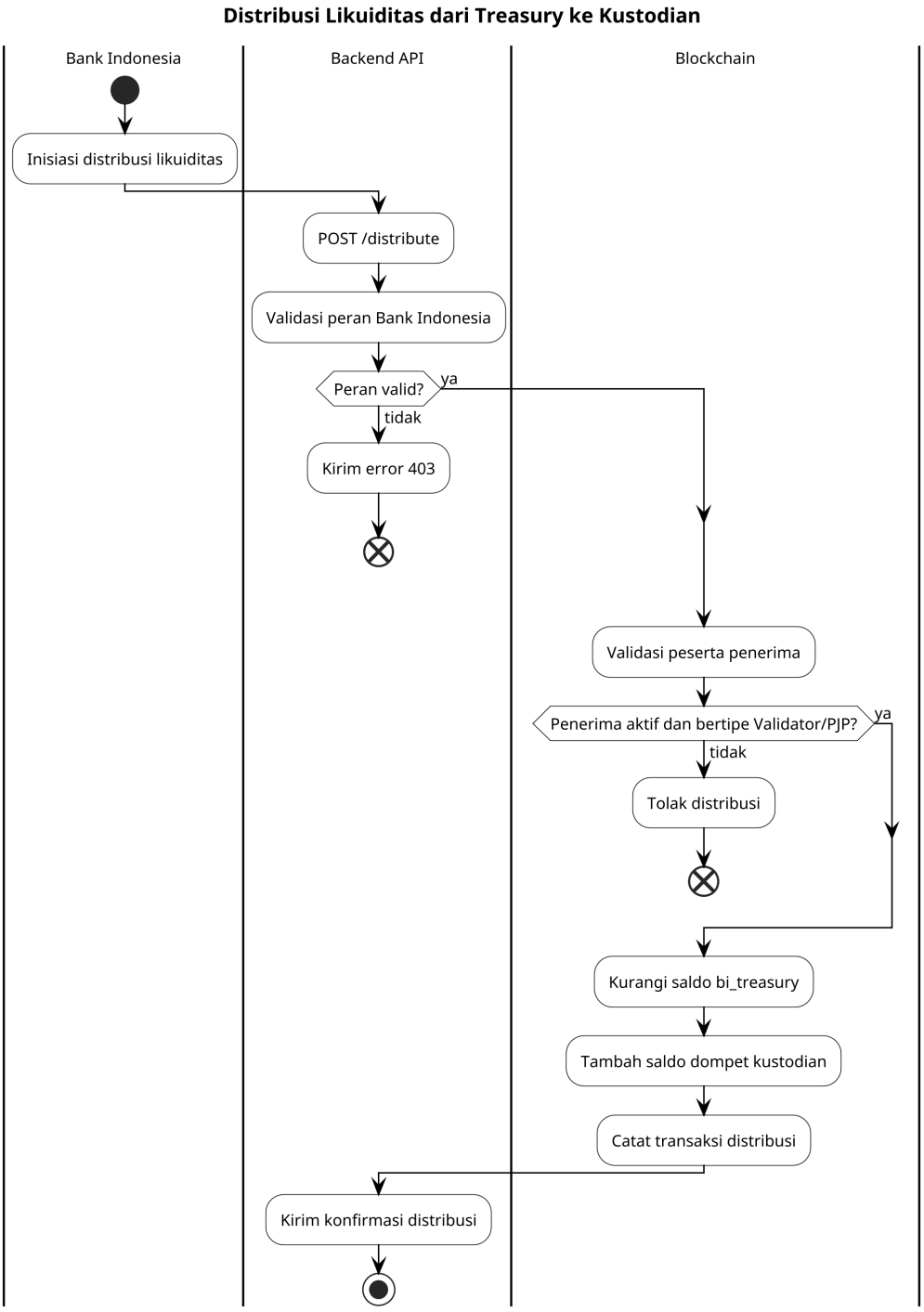
Setiap proses utama dimodelkan dengan activity diagram bergaya *swimlane* tiga jalur, yaitu jalur aktor, jalur *backend API*, dan jalur *blockchain* (*chaincode*). Diagram tersebut mencakup pendaftaran peserta (Gambar 3.14), penerbitan Digital Rupiah (Gambar 3.15), distribusi likuiditas dari Treasury Bank Indonesia kepada kustodian (Gambar 3.16). Proses lainnya meliputi transfer saldo ritel (Gambar 3.17), pembekuan dan pencairan peserta (Gambar 3.18), serta laporan dan pengawasan (Gambar 3.19).



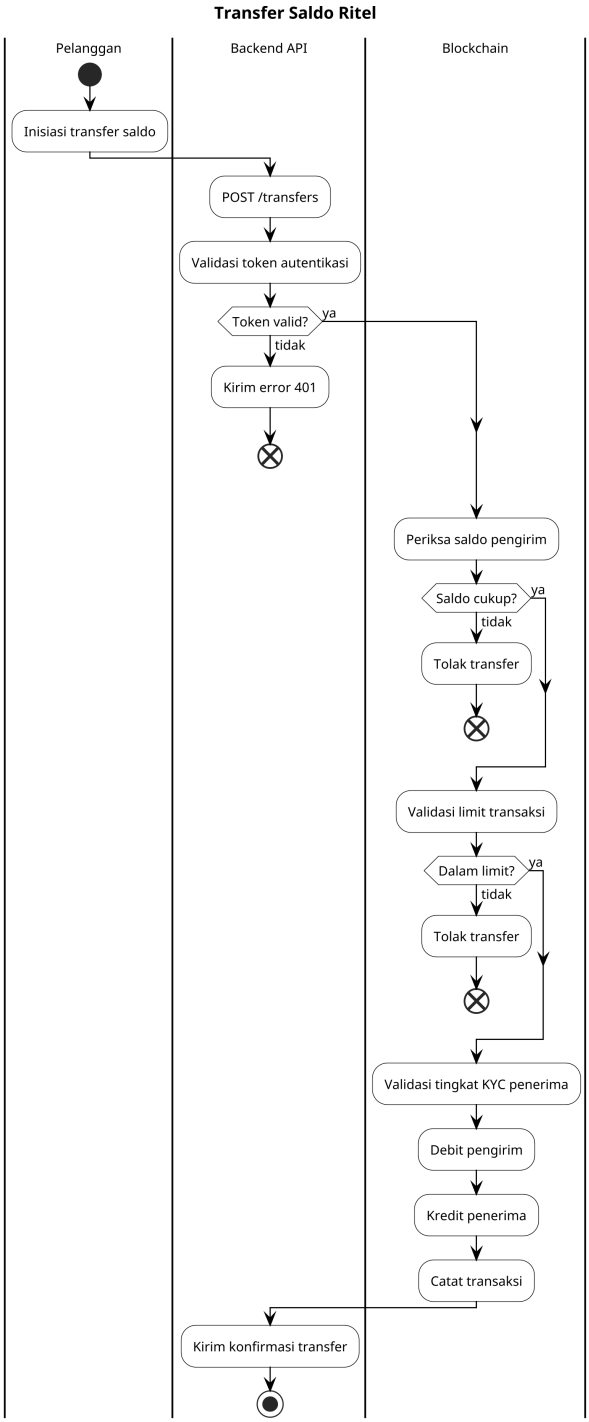

Gambar 3.14. Activity diagram pendaftaran peserta



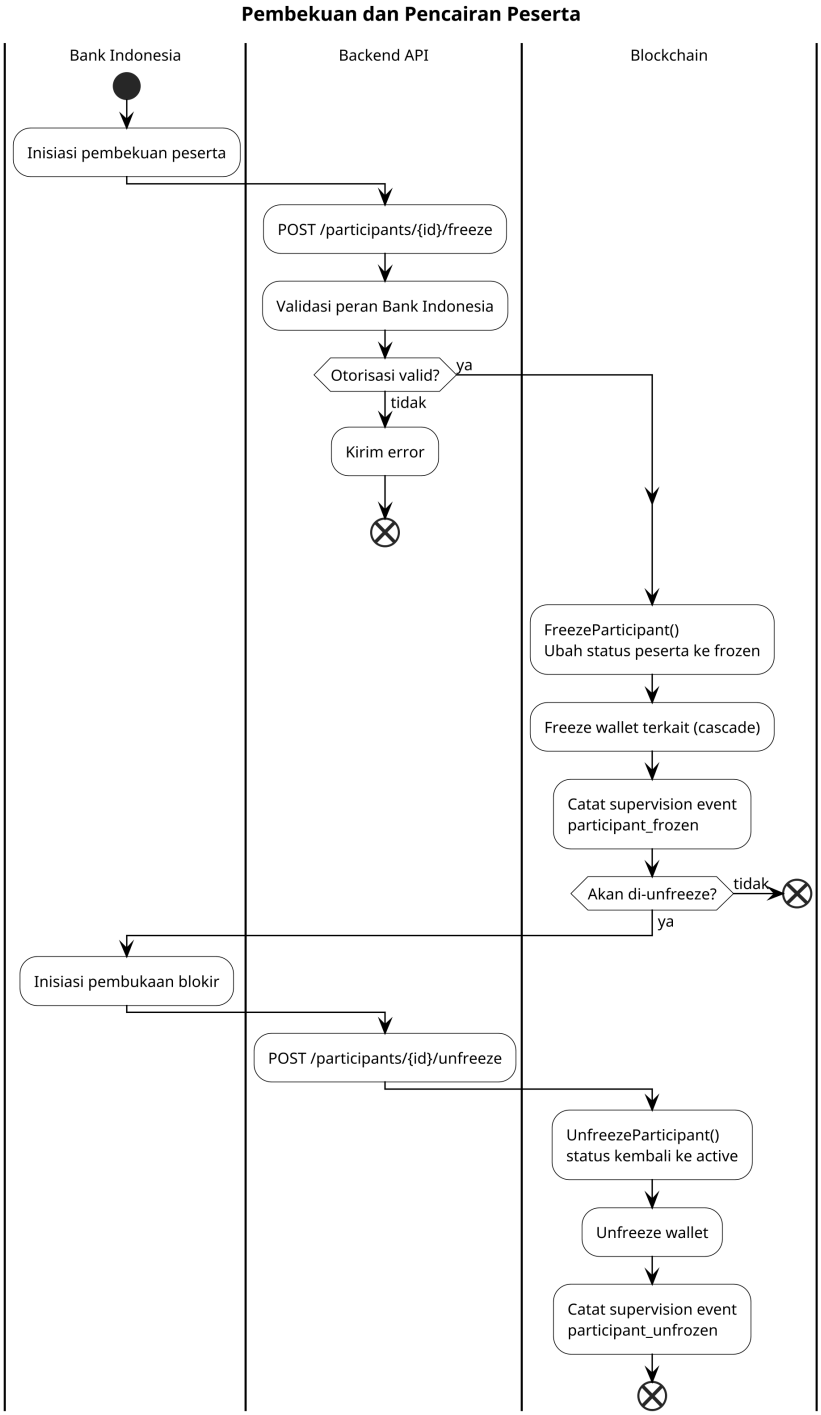

Gambar 3.15. Activity diagram penerbitan Digital Rupiah



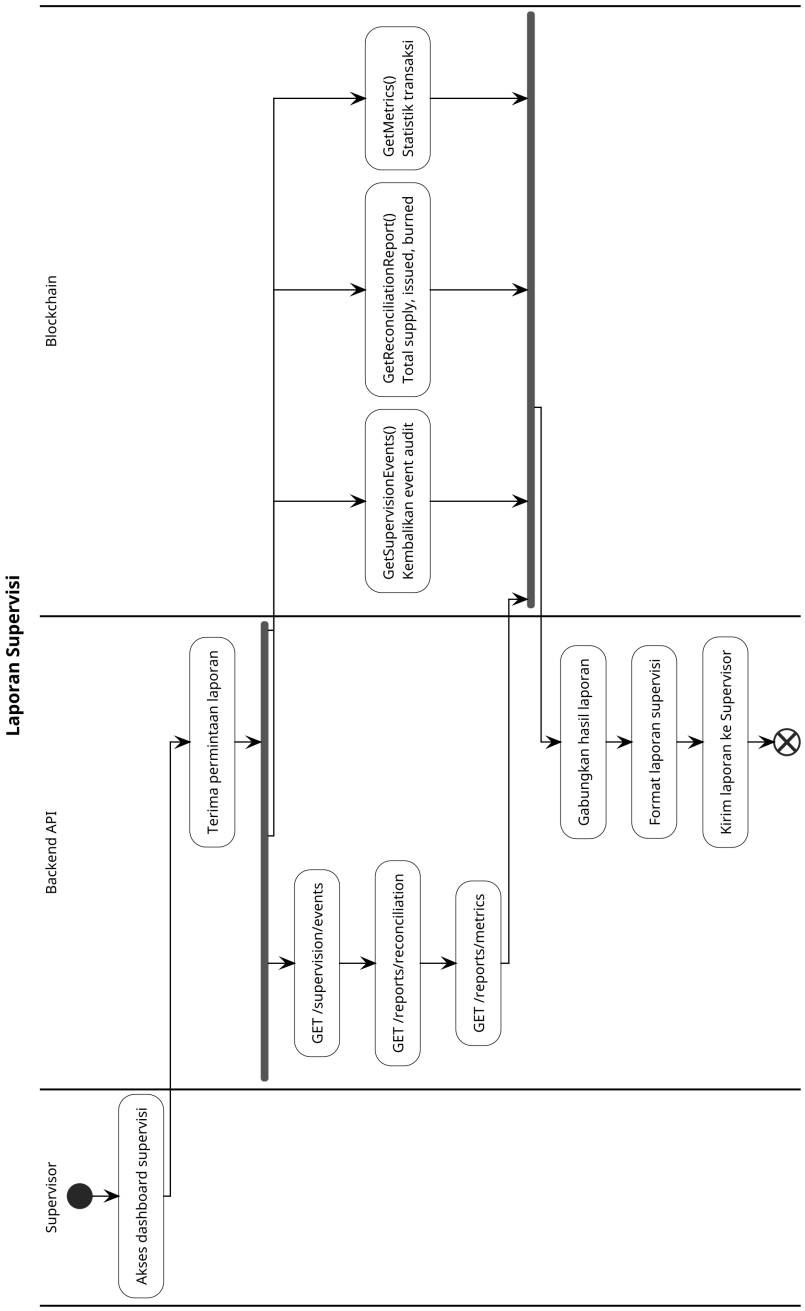

Gambar 3.16. Activity diagram distribusi likuiditas ke PJP




Gambar 3.17. Activity diagram transfer saldo ritel




Gambar 3.18. Activity diagram pembekuan dan pencairan peserta oleh Bank Indonesia



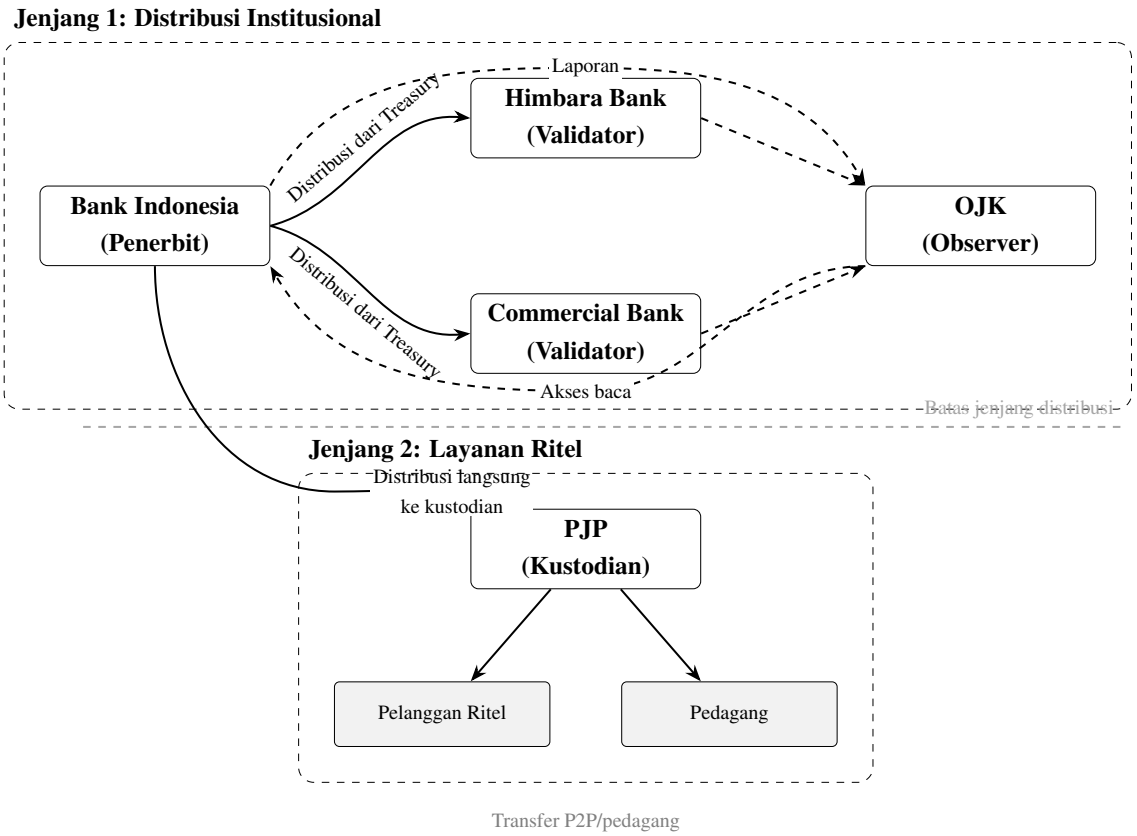

Gambar 3.19. Activity diagram laporan dan pengawasan supervisi

#### 3.1.1.4 Perancangan Arsitektur Sistem Blockchain

Arsitektur dan lima organisasi pada subbab ini menjelaskan baseline `digital-rupiah` v3.0 yang digunakan untuk pengukuran kinerja. Lingkungan empat MSP untuk pengujian kontrol dijelaskan terpisah pada bagian eksperimen kontrol di bawah.

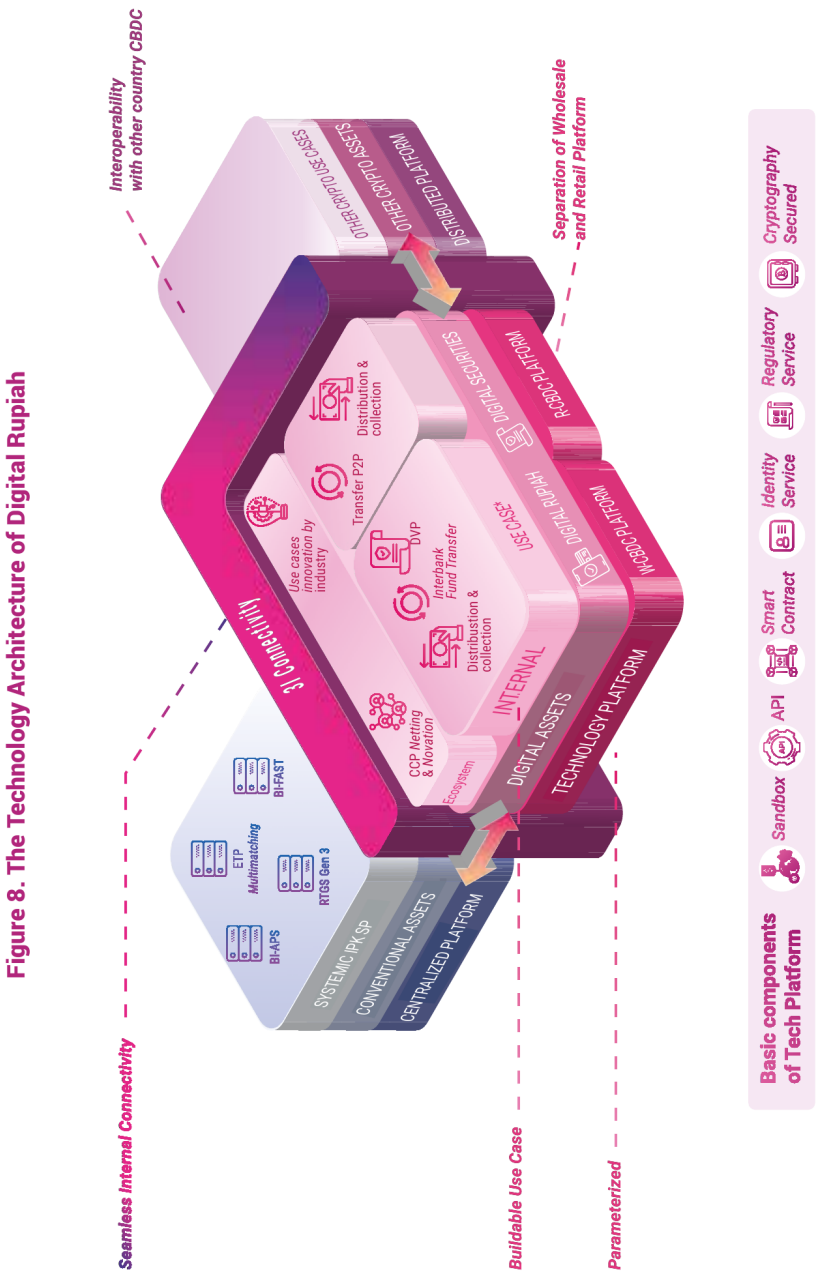
Secara konseptual, sistem menggunakan model distribusi Rupiah Digital dua jenjang sebagaimana ditunjukkan pada Gambar 3.20. Jenjang distribusi institusional menghubungkan Treasury Bank Indonesia selaku penerbit dengan bank validator

(HimbaraBank dan CommercialBank) atau PJP sebagai kustodian melalui jaringan Hyperledger Fabric. Jenjang layanan ritel menyediakan pembayaran melalui kustodian, termasuk transfer antarpelanggan, transfer pelanggan-ke-pedagang, dan kontrol akses berbasis peran. Jenjang institusional tersebut menjelaskan jalur distribusi menuju layanan ritel, bukan implementasi instrumen w-Digital Rupiah yang terpisah.




Gambar 3.20. Arsitektur distribusi dua jenjang pada purwarupa Rupiah Digital ritel.

Arsitektur teknologi Rupiah Digital dalam White Paper CBDC Bank Indonesia terdiri atas lapisan platform teknologi, lapisan aset digital, dan lapisan use case. Komponen dasarnya mencakup *sandbox*, API, *smart contract*, layanan identitas, layanan regulasi, dan pengamanan kriptografi sebagaimana ditunjukkan pada Gambar 3.21 [5]. Purwarupa *bi-coin-fabric* memetakan gagasan tersebut secara terbatas melalui API backend, *chaincode*, identitas organisasi, peran pengawasan, serta *ledger* Hyperledger Fabric. Pemetaan terbatas ini menjadikan purwarupa sebagai model eksperimental yang selaras dengan prinsip arsitektur BI, bukan representasi penuh arsitektur produksi Rupiah Digital.

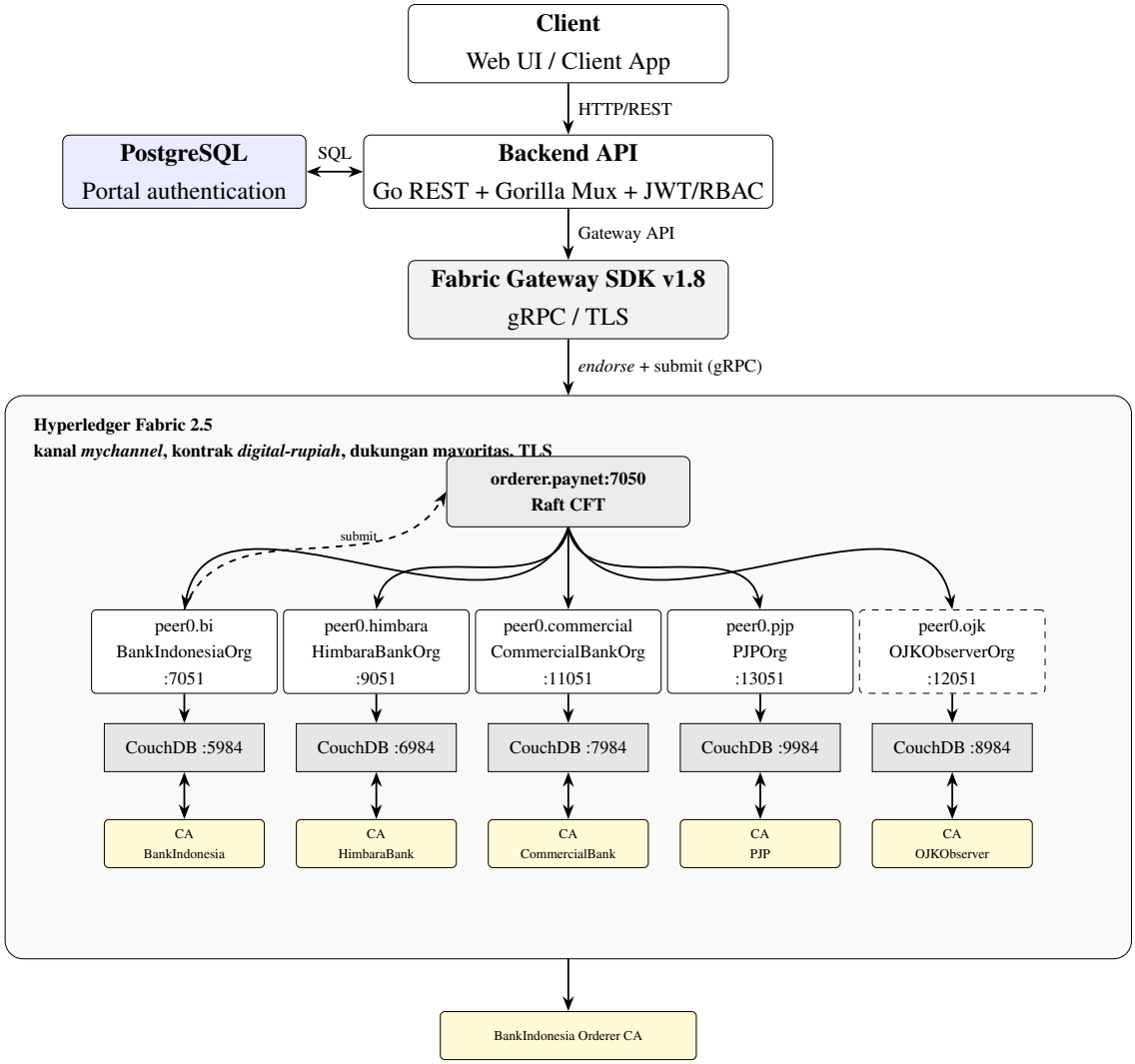



Gambar 3.21. Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].

Sistem dirancang sebagai arsitektur layanan *backend API* dan jaringan *blockchain* sebagaimana ditunjukkan pada Gambar 3.22. Lapisan *backend API* bertindak sebagai *gateway* yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* sekaligus menegakkan kontrol akses berbasis peran, sedangkan lapisan jaringan *blockchain* menjalankan *chaincode* serta menyimpan *ledger*.

Kontrol akses memakai dua batas kepercayaan. Backend memvalidasi JWT, membatasi rute menurut peran, mencatat subjek dan peserta pemanggil, lalu memilih gateway Fabric berdasarkan MSP kustodian pada konteks permintaan. Untuk pengguna ritel, backend juga mencocokkan subjek JWT dengan `owner_id` dompet pengirim. Chaincode memvalidasi MSP pemanggil dan MSP kustodian dompet: mutasi moneter, dan siklus hidup peserta hanya menerima MSP yang diizinkan, sedangkan MSP pengamat tidak dapat memutasi buku besar. Kepemilikan nasabah tetap menjadi batas kepercayaan backend karena identitas pemilik tidak digantikan oleh identitas klien Fabric pada chaincode.

Berdasarkan klasifikasi model pembayaran pada Bab 2, purwarupa *bi-coin-fabric* dikategorikan sebagai model dompet ritel berbasis akun (*account-based wallet model*) yang diimplementasikan di atas *ledger* Hyperledger Fabric. Klasifikasi ini diturunkan dari rancangan data *on-chain*: objek `Wallet` menyimpan `OwnerID`, status beku, dan `Balance`; fungsi `Transfer` memeriksa otorisasi, status beku, dan saldo, kemudian mendebit saldo pengirim serta mengkredit saldo penerima. Sistem tidak membentuk objek token pembawa (*bearer token*), UTXO, atau nomor seri token yang berpindah antarpemilik. Mekanisme pembayaran purwarupa berupa dompet berbasis akun yang disederhanakan, bukan model token murni. Model tersebut digunakan untuk menguji transfer ritel dan auditabilitas transaksi.



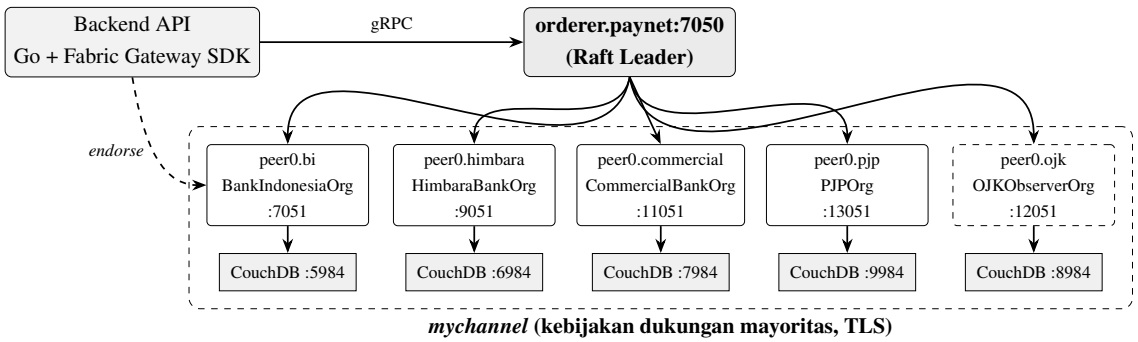

Gambar 3.22. Arsitektur baseline kinerja `digital-rupiah` v3.0: lapisan *client*, *backend API* (Go REST + JWT/RBAC), basis data PostgreSQL luring, *Fabric Gateway SDK*, serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi *peer*, CouchDB, CA per organisasi, dan *orderer* Raft

Jaringan baseline *blockchain* dirancang dengan lima organisasi peserta dan satu organisasi *orderer*, merepresentasikan ekosistem Rupiah Digital multipihak. Setiap organisasi memiliki MSP dan *peer* tersendiri dengan basis data status CouchDB. Rincian organisasi disajikan pada Tabel 3.4.

Tabel 3.4. Organisasi pada jaringan baseline Rupiah Digital

| **Organisasi** | **Peran** | **Keterangan** |
|---|---|---|
| BankIndonesiaOrg | Penerbit | Penerbitan dan penarikan Digital Rupiah, pengawasan |
| HimbaraBankOrg | Validator | Validator jaringan; penerima distribusi BI |
| CommercialBank Org | Validator | Validator jaringan; penerima distribusi BI |
| PJPOrg | PJP | Penyedia layanan ritel; penerima distribusi BI |
| OJKObserverOrg | Observer | Pengamatan/pengawasan (baca) |
| BankIndonesia Orderer | Orderer | Layanan pengurutan (Raft) |

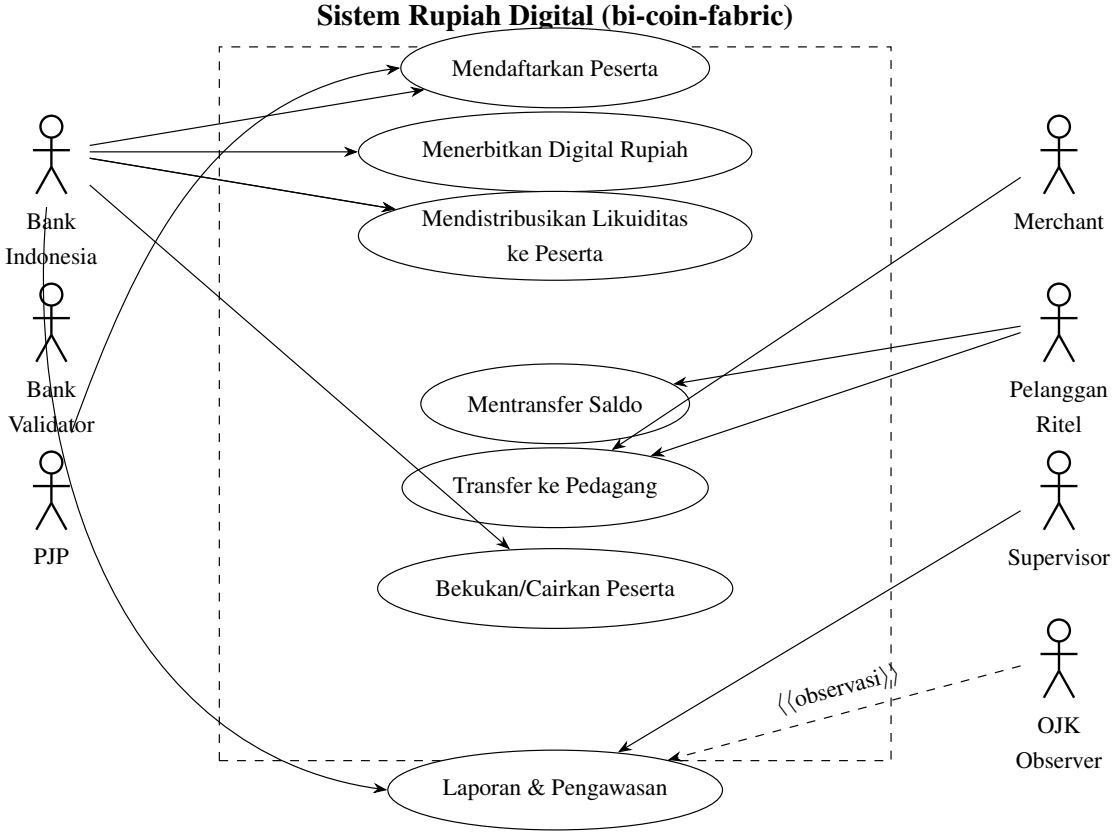
Seluruh peserta tergabung dalam satu kanal (*channel*) dengan kebijakan *endorsement* yang menerima salah satu MSP institusional yang ditetapkan pada konfigurasi kanal. Kebijakan siklus hidup *chaincode* tetap memerlukan persetujuan Bank Indonesia dan salah satu peserta institusional. Komunikasi antarsimpul diamankan dengan TLS. Topologi jaringan secara lengkap ditunjukkan pada Gambar 3.23, yang memperlihatkan hubungan antara *orderer*, kelima *peer*, basis data CouchDB masing-masing organisasi, serta *backend API* yang terhubung ke jaringan melalui *Fabric Gateway SDK*.



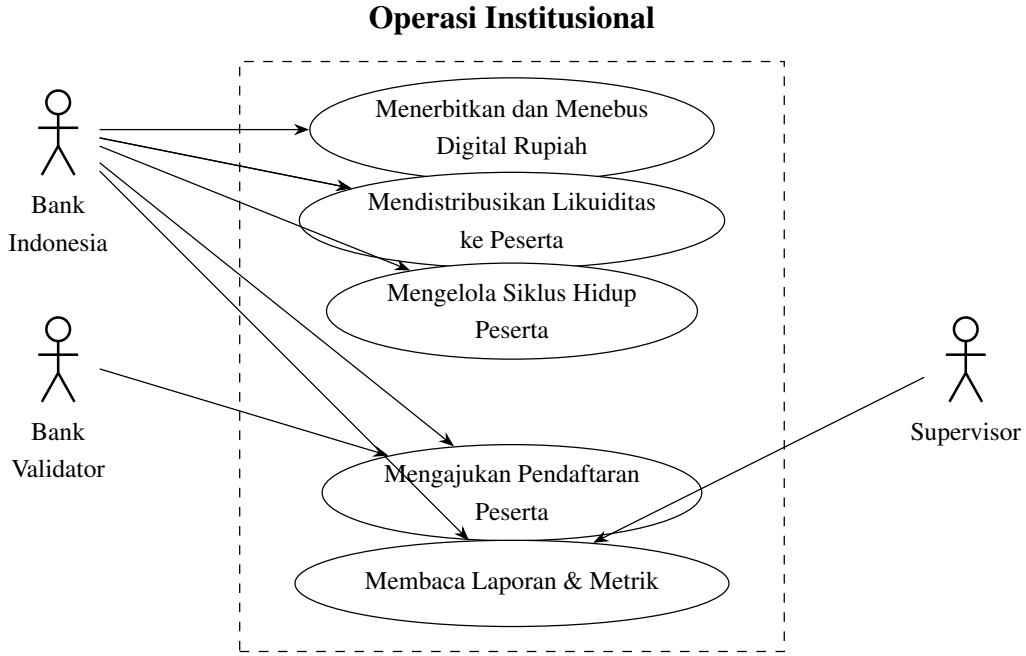

Gambar 3.23. Arsitektur jaringan baseline kinerja `digital-rupiah` v3.0 dengan lima organisasi

Kebutuhan fungsional sistem dimodelkan dengan diagram use case. Aktor sistem terdiri atas Bank Indonesia, bank validator, PJP, pedagang (*merchant*), pelanggan

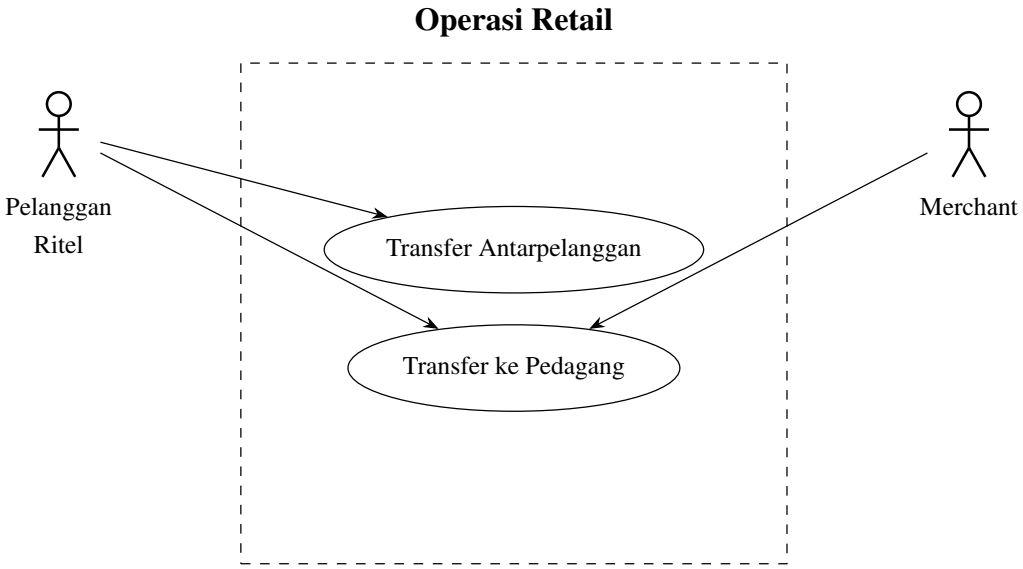
ritel, supervisor, dan pengamat OJK. Diagram use case keseluruhan ditunjukkan pada Gambar 3.24, sedangkan rincian operasi institusional dan ritel ditunjukkan pada Gambar 3.25 dan Gambar 3.26.




Gambar 3.24. Diagram use case keseluruhan sistem Rupiah Digital




Gambar 3.25. Diagram use case operasi institusional




Gambar 3.26. Diagram use case operasi retail

Pemetaan aktor pada diagram tersebut mengikuti kendali peran pada lapisan *backend*. Pendaftaran peserta dapat diajukan oleh institusi, sedangkan persetujuan, penolakan, pembekuan, dan *offboarding* hanya dapat dilakukan oleh Bank Indonesia. Supervisor dan pengamat OJK hanya memperoleh jalur baca pengawasan. Status persetujuan peserta menjadi bagian dari siklus hidup peserta, bukan bukti alur persetujuan terpisah.

Status sistem disimpan pada *world state* dalam bentuk objek-objek yang dirangkum pada Tabel 3.5. Objek-objek ini menjadi dasar seluruh operasi *chaincode*.

Tabel 3.5. Objek status utama pada ledger

| **Objek** | **Keterangan** |
|---|---|
| `Participant` | Entitas peserta (validator/observer/PJP), MSP kustodian, dan BIC opsional beserta status siklus hidup |
| `Wallet` | Dompet dengan pemilik, peserta dan MSP kustodian, saldo, dan status beku |
| `TransactionRecord` | Catatan transaksi untuk audit dan pelaporan |

#### 3.1.1.5 Perancangan Basis Data Luring

Selain status *on-chain* pada *world state* yang dirangkum pada Tabel 3.5, sistem memakai PostgreSQL untuk menyimpan kredensial autentikasi portal dan jejak audit yang bersifat *append-only*. Data identitas mentah tidak ditempatkan pada *ledger*.

### 3.1.2 Metode Implementasi

Pemilihan teknologi dalam implementasi didasarkan pada kesesuaian dengan karakteristik sistem Rupiah Digital yang bersifat *permissioned*, multijenjang, dan memerlukan kueri status yang kompleks.

#### 3.1.2.1 Software Development Life Cycle

Metode *prototyping* dipilih sebagai SDLC penelitian ini. Sistem dibangun sebagai purwarupa fungsional yang mencakup seluruh komponen inti (*chaincode*, jaringan Fabric, dan *backend API*), kemudian dievaluasi secara kuantitatif. Pendekatan ini sesuai dengan karakteristik penelitian eksplorasi pada sistem *blockchain* yang belum memiliki spesifikasi teknis baku dari regulator [12].

#### 3.1.2.2 Kerangka Kerja Blockchain

Hyperledger Fabric 2.5 dipilih sebagai kerangka kerja *blockchain* karena mendukung model keanggotaan berbasis MSP yang sesuai dengan struktur hierarkis Rupiah Digital dua jenjang [16]. Fabric memungkinkan pembatasan akses per peran melalui *endorsement policy*, pemisahan kanal, dan integrasi basis data status yang fleksibel.

#### 3.1.2.3 Mekanisme Konsensus

Konsensus Raft (*etcdraft*) dipilih karena mendukung model *crash fault tolerance* (CFT) yang memadai untuk skenario jaringan berizin antarpeserta terpercaya [16]. Dibanding Kafka, Raft memiliki latensi *ordering* lebih rendah dan tidak bergantung pada infrastruktur tambahan.

#### 3.1.2.4 State Database

CouchDB dipilih sebagai basis data status *world state* menggantikan LevelDB bawaan. CouchDB mendukung kueri kaya berbasis JSON (*rich query*) yang diperlukan untuk pelaporan portofolio dompet, pemfilteran status, dan pengawasan saldo. Kemampuan ini tidak dapat dicapai dengan efisien menggunakan LevelDB yang hanya mendukung kueri berbasis kunci. Perbandingan kedua pilihan tersebut dirangkum pada Tabel 3.6.

Tabel 3.6. Perbandingan basis data status Hyperledger Fabric berdasarkan dokumentasi resmi [4].

| **Kriteria** | **LevelDB** | **CouchDB** |
|---|---|---|
| Model data | Pasangan kunci–nilai yang tertanam pada proses *peer* | Dokumen JSON pada layanan basis data terpisah |
| Kueri | Pembacaan berdasarkan kunci dan rentang kunci | Kueri kaya berbasis bidang JSON dan selector |
| Indeks | Tidak menyediakan indeks JSON | Mendukung indeks JSON untuk kueri yang deterministik |
| Operasional | Konfigurasi dan kebutuhan sumber daya lebih sederhana | Memerlukan kontainer dan pengelolaan indeks tambahan |
| Kesesuaian | Status sederhana dengan pola akses berbasis kunci | Pelaporan dompet, pemfilteran status, dan pengawasan saldo |

#### 3.1.2.5 Basis Data Luring

Sementara CouchDB menampung *world state* Fabric, basis data luring (*off-chain*) yang menampung data sensitif memerlukan paradigma yang berbeda. Pemilihan basis data relasional PostgreSQL didasarkan pada lima kriteria yang dipaparkan berikut ini, kemudian dirangkum pada Tabel 3.7.

**Relasional versus non-relasional.** Kredensial autentikasi dan jejak audit merupakan data terstruktur dengan skema kaku, hubungan antarentitas yang jelas, serta persyaratan integritas transaksional yang ketat. Basis data relasional menjamin aspek tersebut melalui transaksi ACID, kendala *foreign key*, dan transaksi DDL yang atomik. Sebaliknya, basis data dokumen NoSQL (mis. MongoDB) dirancang untuk data semiterstruktur dengan skema fleksibel serta beban baca dominan, dan secara tradisional mengorbankan konsistensi transaksional demi skalabilitas horizontal [53]. Untuk jejak audit yang bersifat *append-only*, penjaminan bahwa suatu peristiwa benar-benar tersimpan sebelum komit (MVCC tanpa *read skew*) merupakan kebutuhan mutlak, bukan opsional.

**PostgreSQL versus MySQL dan SQL Server.** Antara Alternatif relasional, PostgreSQL dipilih berdasarkan pertimbangan teknis berikut:

1. *JSON/JSONB dengan indeks GIN.* Metadata autentikasi dan struktur peristiwa audit dapat disimpan sebagai `JSONB` yang diindeks secara native melalui GIN,

memungkinkan kueri *rich* dengan transaksi ACID [54]. MySQL hanya mendukung tipe `JSON` dengan kapabilitas pengindeksan yang lebih terbatas.

2. *Row-Level Security (RLS).* PostgreSQL menyediakan RLS secara native, sehingga isolasi data antarorganisasi peserta dapat ditegakkan pada tingkat basis data, bukan sekadar lapisan aplikasi. Hal ini sejalan dengan prinsip regulasi UU Pelindungan Data Pribadi (UU PDP) yang mensyaratkan kontrol akses berbasis peran. MySQL tidak memiliki fitur ini dan SQL Server hanya menyediakannya pada edisi Enterprise [54].
3. *MVCC dan transaksi DDL.* PostgreSQL mengelola konkurensi via MVCC tanpa *lock* memblokir pembaca, dan membungkus perubahan skema (DDL) dalam transaksi yang dapat dieksekusi secara atomik. Fitur ini tidak dimiliki MySQL karena DDL-nya bersifat *implicit commit* [55].
4. *Lisensi terbuka.* Lisensi PostgreSQL mirip BSD memungkinkan penggunaan, modifikasi, dan distribusi tanpa biaya royalti, menghindari *vendor lock-in*. SQL Server bersifat proprietari, sedangkan MySQL berlisensi ganda (GPL/komersial) yang membatasi integrasi pada produk berlisensi tidak kompatibel [55].
5. *Kesesuaian ekosistem Fabric.* Pola arsitektur Fabric merekomendasikan penyimpanan data sensitif di luar *ledger* melalui *private data collections* atau basis data eksternal, dengan hanya hash kriptografis yang dibukukan [56]. PostgreSQL merupakan *backend* relasional yang banyak diadopsi dalam implementasi Fabric pendidikan dan industri, sehingga ketersediaan *driver* Go (`pgx`) dan pola integrasi yang teruji lebih matang dibandingkan alternatif lain.

Pola pemisahan on-chain/off-chain yang diadopsi penelitian ini menempatkan kredensial autentikasi dan jejak audit di luar *ledger*, sementara status dompet dan transaksi tetap dibukukan pada jaringan Fabric. Pemisahan tersebut membatasi data operasional yang disimpan pada buku besar bersama tanpa menghilangkan keterlacakan transaksi.

Tabel 3.7. Perbandingan basis data kandidat untuk penyimpanan luring.

| **Kriteria** | **PostgreSQL** | **MySQL** | **SQL Server** |
|---|---|---|---|
| Transaksi ACID | Ya (DDL transaksional) | Sebagian (DDL non-atomik) | Ya |
| JSONB + indeks GIN | Native | Terbatas | Terbatas |
| Row-Level Security | Native | Tidak ada | Edisi Enterprise saja |
| MVCC tanpa *lock* baca | Ya | Ya (InnoDB) | Ya (snapshot) |
| Lisensi | BSD-like (bebas) | GPL / komersial | Proprietari |
| Adopsi ekosistem Fabric | Umum | Jarang | Jarang |

#### 3.1.2.6 Bahasa Pemrograman

*Chaincode* dan *backend API* ditulis dalam bahasa Go. Go dipilih karena (1) SDK *Fabric Contract API* dan *Fabric Gateway SDK* tersedia secara resmi dan aktif dipelihara untuk ekosistem Fabric; (2) model konkurensi goroutine cocok untuk beban I/O tinggi pada *backend API*; dan (3) kompilasi statis mempermudah distribusi kontainer tanpa dependensi eksternal [16].

### 3.1.3 Metode Pengujian

Pengujian dibagi menjadi pengujian fungsional dan pengujian kinerja. Pengujian fungsional menggunakan skenario *ledger smoke test*, *unit test*, serta matriks otorisasi dan validasi API. Skenario tersebut memeriksa status transaksi dan dampak yang diharapkan pada ledger. Penolakan hanya dinilai benar jika keadaan ledger sebelum dan sesudah transaksi tetap sama.

Pengujian kinerja mengukur 30 metode kontrak publik dengan Hyperledger Caliper dan konektor Fabric. Metode `Initialize` dan `DebitReserveForFunding` hanya diperiksa secara fungsional karena `DebitReserveForFunding` dipanggil secara internal oleh konversi. Metode `Transfer` diuji dengan variasi penerima ritel dan pedagang sehingga terdapat 31 beban kerja kinerja. Operasi baca dilaporkan sebagai operasi terverifikasi per detik, sedangkan transaksi tulis dilaporkan sebagai TPS terverifikasi. Semua panggilan kinerja diarahkan langsung ke *chaincode*; latensi pemrosesan API, PostgreSQL, dan KYC tidak

digabungkan ke hasil Fabric.

Seluruh pengujian kinerja memakai satu *worker* lokal. Inisialisasi ronde Caliper menyiapkan koneksi; tidak ada ronde pemanasan bertimer. Matriks API merupakan pemeriksaan kebenaran yang terpisah: 207 sel peran-rute untuk otorisasi, 27 kasus validasi permintaan JSON, dan 15 pemeriksaan hasil aksi tanpa badan permintaan.

Prosedur pengujian kinerja kapasitas berjalan dalam lima langkah berurutan. Pertama, jaringan studi dibuat dari awal pada lingkungan Docker-in-Docker yang terisolasi tanpa port host, lalu sumber kode diambil sebagai *snapshot* dan diperiksa hash-nya; pada pemulihan, jaringan dinaikkan kembali dengan materi identitas yang sudah ada. Kedua, setiap beban kerja diuji pada laju tertentu dengan ronde Caliper 10 detik; laju lulus hanya jika semua hasil terverifikasi dan throughput terverifikasi terhadap durasi ditambah drain mencapai 90 persen laju terkirim, setara dengan drain paling lama sepersembilan durasi probe. Ketiga, pencarian biner mempersempit braket hingga selisih relatif 10%. Keempat, laju lulus tertinggi dikonfirmasi tiga kali, masing-masing 30 detik. Kelima, sampel sumber daya dicatat setiap detik untuk setiap komponen selama jendela ukur, yaitu sekitar sepuluh sampel per probe 10 detik dan 30 sampel per jendela konfirmasi 30 detik. Setiap percobaan yang terhenti, gagal, atau tidak lengkap dipertahankan sebagai catatan terpisah. Galat pengambilan pada harness, bagaimanapun, dicatat sebagai kegagalan transaksi dan dapat memengaruhi batas pencarian, sebagaimana dilaporkan pada Bab 4. Pelanjutan ulang hanya diperbolehkan dengan *stamp* dan sumber yang sama, sehingga percobaan awal tidak tertimpa. Tahap dan kriteria pengukuran dirangkum pada Tabel 3.8; batas antara pengukuran kinerja dan pemeriksaan fungsional dirangkum pada Tabel 3.9.

Tabel 3.8. Tahap pencarian dan konfirmasi kapasitas

| **Tahap** | **Prosedur dan kriteria** |
|---|---|
| Kesiapan | Petakan penanda tangan ke MSP yang berwenang, konfigurasi tiga *chaincode* dan empat MSP, lalu pastikan setiap beban kerja berhasil. Inisialisasi ronde Caliper menyiapkan koneksi tanpa ronde pemanasan bertimer. |
| Braket | Gunakan hasil terdahulu hanya sebagai petunjuk laju awal; bila tidak tersedia, mulai pada 32 operasi/detik. Tetapkan rentang 0,1–4.096 operasi/detik. Jika laju awal lulus tetapi belum ada batas gagal, kalikan laju dengan empat; jika laju awal gagal tetapi belum ada batas lulus, bagi laju dengan empat sampai braket terbentuk. |
| Pencarian biner | Uji titik tengah braket dalam ronde 10 detik sampai selisih relatif laju gagal terhadap laju lulus paling besar 10%. Kesalahan konfigurasi atau data uji menghentikan pencarian beban kerja tersebut untuk diagnosis. |
| Konfirmasi | Jalankan tiga ronde terpisah yang dikelompokkan untuk satu beban kerja, masing-masing 30 detik, pada laju lulus tertinggi. Jika konfirmasi gagal, turunkan kandidat dan ulangi konfirmasi; hasil campuran dilaporkan sebagai batas tidak stabil. Galat penyiapan fixture tidak dihitung sebagai kegagalan konfirmasi; pada fund-reserve, percobaan tersebut diarsipkan dan pencarian beban kerja diulang. |
| Interpretasi | Kelulusan pada batas atas rentang pencarian, 4.096 operasi/detik, dilaporkan sebagai kapasitas sekurang-kurangnya sebesar laju terukur. Durasi konfirmasi minimum seluruh 31 beban kerja adalah 46,5 menit, di luar penyiapan, pencarian, dan verifikasi. |

Tabel 3.9. Cakupan 20 skenario fungsional dan assertion yang dijalankan

| **Kelompok skenario** | **Skenario katalog dan hasil yang diperiksa** |
|---|---|
| KYC dan pemisahan kewenangan | `kyc-risk-pep`, `staff-separation`, `business-authority`, `restriction-reasons`, dan `fresh-release-evidence`; hasil izin/penolakan dan bukti pelepasan diperiksa terhadap keadaan subjek pada ledger. |
| Otorisasi dan transfer | `authorization-endorsement`, `transaction-limits`, `transfer-category-matrix`, dan `transfer-replay-conflict`; identitas pemanggil, kebijakan endorsement, batas, kategori transfer, referensi ulang, serta dampak ledger diperiksa. |
| Konversi dan penebusan | `conversion-replay-conflict`, `conversion-insufficient-reserve`, `conversion-holdings-limit`, `conversion-failure-atomicity`, `redemption-replay-conflict`, dan `redemption-conservation`; konflik/replay, saldo reserve, batas kepemilikan, atomisitas kegagalan, dan konservasi nilai diverifikasi. |
| Akses baca, inisialisasi, dan API | `msp-read-authorization`, `initialization-and-nested-debit`, `api-route-authorization-matrix`, `api-request-validation-matrix`, dan `api-route-action-outcomes`; hasil ledger, validasi, otorisasi, dan dampak aksi diperiksa secara terpisah. |

Setiap skenario katalog dipetakan ke assertion yang dapat dijalankan; jumlah baris katalog tidak diperlakukan sebagai jumlah pengujian yang berhasil. Metrik setiap laju mencakup jumlah dan laju operasi yang dikirim, hasil terverifikasi, kegagalan, waktu habis, hasil belum terselesaikan, antrean pada batas akhir ronde, serta penyelesaian terlambat. Latensi dilaporkan sebagai median, p95, maksimum, dan

jumlah sampel. Penggunaan sumber daya mencakup rata-rata dan puncak CPU serta memori per komponen, juga delta I/O jaringan dan disk; sampel diambil setiap satu detik [49]. Kegagalan dicatat bersama ID transaksi, diagnostik Gateway/peer, kode validasi, dan kelas sebab; sebab terkonfirmasi dibedakan dari hipotesis.

Untuk sebuah kandidat kapasitas, seluruh hasil pada jendela ukur harus terverifikasi, tanpa kegagalan tak terduga, waktu habis, ataupun hasil belum terselesaikan, serta dengan jejak transaksi dan jendela sumber daya yang lengkap. Penolakan yang diharapkan hanya dihitung benar apabila assertion memastikan keadaan ledger tidak berubah. Jumlah operasi yang terkirim harus berada dalam toleransi 5% dari target dengan pembulatan satu permintaan, dan laju terverifikasi setelah drain harus mencapai sedikitnya 90% dari laju terkirim. Dengan demikian, 100% merupakan ketepatan yang diamati hanya untuk beban kerja dan durasi yang diuji; skenario fungsional yang belum terselesaikan menghalangi klaim konformansi keseluruhan.

Laju yang ditawarkan dihitung dengan Persamaan 3-1.

$$T_{\text{offered}} = \frac{N_{\text{delivered}}}{t_{\text{end}} - t_{\text{start}}} \tag{3-1}$$

Laju terverifikasi setelah drain dihitung dengan Persamaan 3-2.

$$T_{\text{verified,drain}} = \frac{N_{\text{verified}}}{(t_{\text{end}} - t_{\text{start}}) + t_{\text{drain}}} \tag{3-2}$$

Latensi transaksi berhasil ke-$i$ dihitung dengan Persamaan 3-3.

$$L_i = t_{\text{final},i} - t_{\text{create},i} \tag{3-3}$$

Pada persamaan tersebut, $N_{\text{delivered}}$ adalah jumlah operasi yang terkirim selama jendela ukur, $N_{\text{verified}}$ adalah jumlah hasil yang terverifikasi termasuk penolakan yang diharapkan dengan keadaan ledger tetap, dan $t_{\text{drain}}$ adalah waktu sejak akhir jendela sampai penyelesaian terakhir. Untuk operasi baca, $T_{\text{verified,drain}}$ dilaporkan sebagai operasi kueri terverifikasi per detik; untuk operasi tulis, nilai tersebut dilaporkan sebagai laju lulus terkonfirmasi dalam TPS, bersama throughput terverifikasi. Latensi dihitung dari waktu mulai hingga penyelesaian hasil pada jejak transaksi Caliper. Angka kinerja hanya mencakup jalur langsung ke *chaincode*; waktu validasi, otorisasi, dan pemrosesan API, PostgreSQL, maupun KYC diperiksa terpisah dan tidak digabungkan ke hasil Fabric.

**Justifikasi pemilihan parameter.** Parameter pengujian dipilih berdasarkan dua pertimbangan: literatur dan rekayasa sistem.

Parameter pencarian mengikuti konfigurasi Caliper Fabric v0.6 dan protokol batas yang ditetapkan: satu *worker*, rentang 0,1–4.096 operasi/detik, probe 10 detik, dan toleransi braket 10% [1]. Petunjuk laju dari pengukuran terdahulu hanya menentukan titik awal, bukan menetapkan hasil; braket dan konfirmasi diperoleh ulang untuk setiap beban kerja. Tiga ronde konfirmasi masing-masing 30 detik mengurangi ketergantungan pada satu pengukuran singkat, tetapi hasilnya tetap hanya mendukung klaim kapasitas lokal yang diamati.

Fixture disiapkan sebelum jendela ukur dan ledger diverifikasi setelahnya. Pool berputar digunakan untuk transisi yang mengizinkan pengulangan, sedangkan operasi satu kali memperoleh rekaman baru. Kunci moneter yang secara inheren dibagi tetap dipertahankan dan setiap transaksi memakai referensi unik agar replay dapat dibedakan dari operasi baru. Progres mencatat tahap, laju, jumlah fixture yang disiapkan, dan waktu berjalan; hasil probe yang gagal atau tidak konklusif tetap disimpan dan tidak dihitung sebagai kapasitas lulus.

Pengujian berlangsung pada satu host dengan satu *worker* Caliper. Karena itu, hasil membatasi bukti pada kapasitas lokal jangka pendek; perbandingan jumlah *worker* dan pengujian lintas-host ditempatkan sebagai penelitian lanjutan. Saldo awal dan populasi fixture merupakan pilihan rekayasa, bukan estimasi distribusi pengguna nasional.

**Kesesuaian dengan skenario CBDC ritel.** Beban kerja pengujian dirancang untuk merepresentasikan alur transaksi Rupiah Digital ritel secara struktural. Setiap fungsi diuji secara terisolasi agar throughput, latensi, dan alasan kegagalan dapat diatribusikan pada fungsi yang diukur. Transfer antarpelanggan, pembayaran pelanggan–pedagang, penerbitan, distribusi, penebusan, kueri saldo, serta perubahan status dompet dipisahkan; transfer ritel juga memiliki profil kontensi rendah dan kontensi tinggi. Pola ini dipilih karena CBDC ritel mencakup transaksi pengguna, distribusi, dan pengawasan, bukan hanya setelmen antarlembaga. Literatur teknologi CBDC ritel menempatkan desain dompet, privasi, dan arsitektur distribusi sebagai elemen inti sistem pembayaran publik [6]. Oleh karena itu, beban kerja yang hanya menguji transfer institusional tidak memadai untuk menjawab rumusan masalah purwarupa Rupiah Digital ritel.

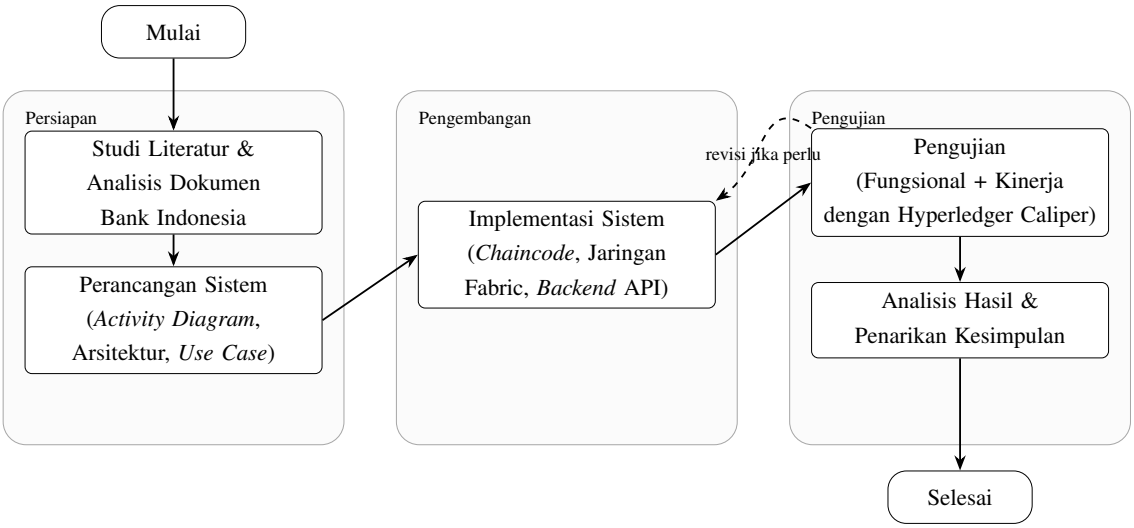
Model dua jenjang penerbitan ($\text{BI} \rightarrow \text{bank} \rightarrow \text{PJP} \rightarrow$ pelanggan ritel) sesuai dengan rancangan Rupiah Digital yang ditetapkan dalam dokumen konsultatif Bank Indonesia [11, 12]. Logika bisnis yang diuji bukan beban sintetik yang terlepas dari sistem, melainkan *chaincode* aktual yang mengimplementasikan penerbitan, distribusi, transfer ritel, dan pembacaan status dompet. Penggunaan Caliper dan metrik *throughput*/latensi mengikuti praktik pengukuran kinerja blockchain yang menekankan laju transaksi, latensi, dan karakterisasi kegagalan transaksi [1, 33, 49]. Konservasi nilai,

replay, dan atomisitas kegagalan diperiksa melalui skenario fungsional dengan verifikasi keadaan ledger sebelum dan sesudah transaksi.

Perbedaan utama dengan sistem CBDC ritel produksi adalah skala (populasi pengujian sintetis berbanding jutaan pengguna nyata), topologi jaringan (satu *peer* per organisasi dalam satu mesin lokal berbanding jaringan terdistribusi geografis), dan integrasi identitas produksi (disimulasikan secara terprogram). Keterbatasan ini berlaku umum pada studi prototipe CBDC berbasis *permissioned blockchain* [14] dan bukan kelemahan metodologis melainkan batasan cakupan yang dinyatakan secara eksplisit pada Batasan Penelitian (Bab I).

## 3.2 Alur Tugas Akhir

Tugas akhir ini dilaksanakan melalui tahapan berikut. Gambar 3.27 menyajikan alur dari studi literatur hingga analisis hasil. Implementasi dan pengujian saling memberi umpan balik sesuai siklus *prototyping*.




Gambar 3.27. Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian

### 3.2.1 Mempelajari Proses Bisnis & Sistem Blockchain

Penelitian diawali dengan mempelajari konsep Rupiah Digital berdasarkan dokumen White Paper dan Consultative Paper Bank Indonesia [5, 11, 12], serta memahami mekanisme *blockchain* permissioned berbasis Hyperledger Fabric sebagai landasan implementasi.

### 3.2.2 Melakukan Studi Literatur

Mengkaji konsep CBDC, desain Rupiah Digital, Hyperledger Fabric, dan distribusi dompet sebagai dasar perancangan sistem.

### 3.2.3 Menentukan Metode Penelitian

Menentukan aktor, kebutuhan fungsional, dan aturan bisnis sistem Rupiah Digital ritel berdasarkan model dua jenjang, serta menetapkan metode prototyping, pemilihan teknologi, dan parameter pengujian.

### 3.2.4 Pengembangan Sistem Blockchain bi-coin

Pengembangan sistem dilakukan secara iteratif meliputi instalasi infrastruktur Fabric, konfigurasi jaringan, *deploy chaincode*, pengembangan *gateway*, dan pengembangan *backend API*.

#### 3.2.4.1 Instalasi Kerangka Kerja Hyperledger Fabric

Lingkungan pengembangan disiapkan dengan memasang Docker, Docker Compose, dan biner Fabric. Material kriptografi (sertifikat MSP dan TLS) dibangkitkan menggunakan `cryptogen`. Blok genesis dan profil konfigurasi kanal dibuat dengan `configtxgen`.

#### 3.2.4.2 Konfigurasi Jaringan Blockchain

Jaringan Fabric dibangun melalui skrip otomatisasi yang membangkitkan material kriptografi, blok genesis, dan konfigurasi kanal, kemudian menjalankan kontainer untuk lima *peer*, lima CouchDB, dan satu *orderer*. Chaincode dipaketkan, dipasang pada seluruh organisasi, disetujui sesuai kebijakan dukungan, lalu di-*commit* ke kanal.

#### 3.2.4.3 Deploy Chaincode

Logika bisnis Rupiah Digital ditulis dalam bahasa Go menggunakan *Fabric Contract API* v2. Chaincode mengimplementasikan fungsi penerbitan (`Mint`, `RequestIssuance`), penarikan (`Burn`, `RequestRedemption`), distribusi dua jenjang (`DistributeToParticipant`), transfer (`Transfer`), serta pelaporan dan pengawasan.

Gambar 3.28 menunjukkan implementasi fungsi `Mint` yang menambah saldo dompet setelah memverifikasi otorisasi dan status beku.

```go
func (s *SmartContract) Mint(ctx contractapi.
    TransactionContextInterface,
    walletID string, amount int64) error {
    if walletID != "bi_treasury" {
        return fmt.Errorf("issuance must credit bi_treasury")
    }
    if amount <= 0 {
        return fmt.Errorf("amount must be positive")
    }
    wallet, err := s.getWallet(ctx, walletID)
    if err != nil { return err }
    if wallet.Frozen {
        return fmt.Errorf("wallet %s is frozen", walletID)
    }
    // ... validasi batas transaksi dan pasokan global ...
    newBalance, err := checkedAddInt64(wallet.Balance, amount)
    if err != nil { return err }
    wallet.Balance = newBalance
    wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
    if err := s.emitAudit(ctx, "MINT", walletID, "", amount);
    err != nil {
        return err
    }
    return s.recordTransaction(ctx, TxMint, walletID, "", amount
    , TxSettled, "")
}
```

Gambar 3.28. Implementasi fungsi `Mint` pada *chaincode*: memverifikasi otorisasi dan status beku sebelum menambah saldo dompet

Gambar 3.29 menunjukkan pemanggilan fungsi transfer bersama. Fungsi tersebut memeriksa otorisasi, status dompet, nilai positif, batas masuk, dan saldo pengirim sebelum memperbarui kedua dompet secara atomik.

```
func (s *SmartContract) Transfer(ctx contractapi.
    TransactionContextInterface,
     senderID string, receiverID string, amount int64,
    referenceID string) (map[string]interface{}, error) {
     // ... periksa MSP kustodian, KYC penerima, status dompet,
    batas, dan saldo ...
     if sender.Tier == "" {
          // Pendanaan kustodian ke dompet ritel hanya mengkredit
    penerima.
          // ... applyWholesaleFundingPolicy, putWallet, dan
    receipt idempotensi ...
     } else {
          // Transfer ritel mengurangi pengirim dan menambah
    penerima.
          // ... applyRetailTransferPolicy, putWallet, dan receipt
     idempotensi ...
     }
     // ... putWallet, recordTransaction, audit, dan receipt
    idempotensi ...
}
```

Gambar 3.29. Pemeriksaan otorisasi, status dompet, dan kecukupan saldo sebelum transfer

#### 3.2.4.4 Pengembangan Fabric Gateway

*Fabric Gateway SDK* v1.8 digunakan untuk menghubungkan *backend API* ke jaringan Fabric melalui gRPC. *Gateway* mengelola siklus transaksi mulai dari pembuatan proposal, pengumpulan *endorsement*, pengiriman ke *orderer*, hingga konfirmasi blok, sehingga lapisan *backend* tidak perlu menangani protokol Fabric secara langsung.

#### 3.2.4.5 Pengembangan Backend

Lapisan *backend* ditulis dalam bahasa Go menggunakan Gorilla mux dan *Fabric Gateway SDK*. Backend menyediakan antarmuka REST, menegakkan kontrol akses berbasis peran melalui *middleware*, serta menerjemahkan permintaan menjadi pemanggilan `SubmitTransaction` atau `EvaluateTransaction` pada *chaincode*. Gambar 3.30 menunjukkan pengelompokan rute REST berdasarkan peran akses menggunakan *middleware* `RequireRole`.

```go
func (h *Handler) RegisterRoutes(r *mux.Router) {
    // Publik: tidak memerlukan autentikasi
    r.HandleFunc("/health", h.Health).Methods("GET")

    // Pembacaan dompet oleh aktor terautentikasi
    auth := r.NewRoute().Subrouter()
    auth.Use(middleware.RequireRole(
        middleware.RoleAuthenticated, middleware.RoleBankPjp,
        middleware.RoleValidatorBank, middleware.RolePJP,
        middleware.RoleBankIndonesia))
    auth.HandleFunc("/wallets", h.ListWallets).Methods("GET")

    // Bank/PJP mengelola dompet dan distribusi ritel
    walletMutation := r.NewRoute().Subrouter()
    walletMutation.Use(middleware.RequireRole(
        middleware.RoleBankPjp, middleware.RoleValidatorBank,
        middleware.RolePJP))
    walletMutation.HandleFunc("/wallets", h.CreateWallet).
    Methods("POST")

    // Pelanggan/pedagang yang disetujui melakukan transfer
    payments := r.NewRoute().Subrouter()
    payments.Use(middleware.RequireRole(
        middleware.RoleAuthenticated, middleware.RoleMerchant))
    payments.HandleFunc("/transfers", h.Transfer).Methods("POST"
    )
}
```

Gambar 3.30. Pengelompokan rute REST *backend* berdasarkan peran akses

### 3.2.5 Pengujian Sistem Blockchain bi-coin

Pengujian mencakup dua tahap: pengujian fungsional untuk memverifikasi kebenaran seluruh alur proses, dan pengujian kinerja menggunakan Hyperledger Caliper untuk mengukur *throughput* dan latensi.

Pada tahap pengujian fungsional, verifikasi tidak hanya dilakukan melalui eksekusi skenario pada aplikasi, tetapi juga melalui *unit test* otomatis yang menargetkan dua lapisan utama. Lapisan pertama adalah layanan *backend* yang menjembatani permintaan API dengan pemanggilan fungsi pada *ledger*. Lapisan kedua adalah *chaincode* yang merealisasikan aturan bisnis Rupiah Digital, termasuk siklus hidup peserta, dompet, transfer, dan pengawasan. Pendekatan ini dipilih agar kebenaran aturan

pada purwarupa dapat diperiksa secara lebih terisolasi sebelum hasilnya dikaitkan dengan pengujian kinerja pada BAB IV.

#### 3.2.5.1 Unit Test Chaincode dan Backend

*Unit test chaincode* memverifikasi fungsi `CreateWallet` dan `Transfer` secara terisolasi menggunakan konteks transaksi tiruan (*mock*). Pengujian mencakup pembuatan dompet, pemeriksaan status beku, otorisasi, dan pembaruan saldo atomik.

Pada lapisan *backend*, *unit test* memverifikasi bahwa batas kepemilikan dompet ritel ditegakkan sebelum permintaan transfer diteruskan ke *chaincode*. Permintaan transfer ditolak ketika identitas JWT pemanggil tidak cocok dengan pemilik dompet pengirim.

Verifikasi lengkap kedua lapisan dijalankan melalui `go test ./...` pada masing-masing modul. Jumlah pengujian yang lulus pada setiap lapisan dipaparkan pada Bab IV.

#### 3.2.5.2 Konfigurasi Hyperledger Caliper

Hyperledger Caliper dikonfigurasi dengan file jaringan (*network config*) yang mendeskripsikan topologi *peer*, *orderer*, kanal, dan identitas MSP. Modul beban kerja ditulis dalam JavaScript dan mengambil identitas penanda tangan serta daftar operasi dari katalog benchmark. Katalog mencakup seluruh metode publik tiga *chaincode*; variasi `Transfer` untuk penerima ritel dan pedagang dihitung sebagai dua beban kerja, sedangkan `Initialize` dan debit reserve internal hanya diuji secara fungsional.

Nilai `version: "2.0.0"` pada tingkat teratas konfigurasi jaringan menyatakan versi skema konfigurasi Caliper, bukan versi Hyperledger Fabric atau *chaincode*. Lingkungan studi kontrol dirangkum pada Tabel 3.10; seluruh organisasi dan komponen jaringan dijalankan pada satu inang lokal berbasis *container* Docker.

Tabel 3.10. Lingkungan pengujian studi kontrol

| **Komponen** | **Spesifikasi** |
|---|---|
| Inang | Satu mesin lokal dengan *container* Docker 29.6.2; satu *worker* Caliper |
| CPU | Intel Core i7-7700HQ @ 2,80 GHz (4 inti, 8 utas) |
| Memori (RAM) | 16 GB |
| Sistem operasi | Fedora Linux 44 (*Workstation Edition*), x86_64 |
| Hyperledger Fabric | 2.5 (rilis 2.5.16) |
| Topologi | Empat MSP, satu *peer* per MSP, kanal `rupiah-study` |
| *Chaincode* | `rupiah-controls`, `rupiah-monetary`, dan `rupiah-payments` |
| Hyperledger Caliper | 0.6.0 |

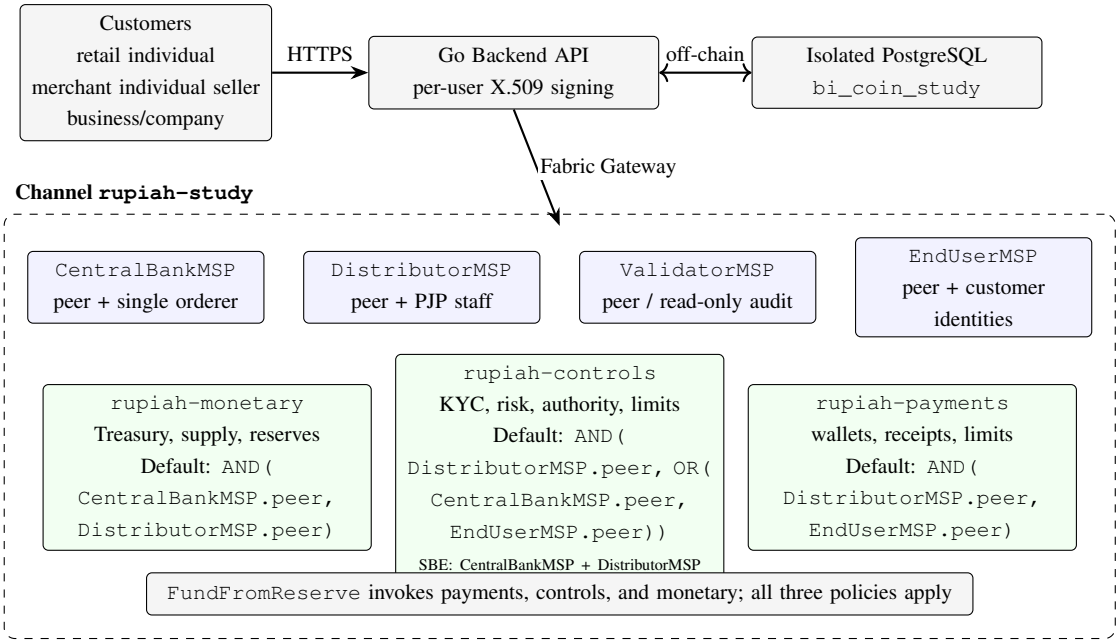
#### 3.2.5.3 Menjalankan Pengujian Hyperledger Caliper

Pengujian memakai konektor Fabric dan kontrol laju *fixed-rate* dengan satu *worker*; setiap beban kerja diukur langsung pada *chaincode*. Inisialisasi ronde Caliper menyiapkan koneksi tanpa pemanasan bertimer. Kueri dilaporkan sebagai operasi terverifikasi per detik, sedangkan operasi tulis dilaporkan sebagai laju lulus terkonfirmasi dalam TPS. Prosedur pencarian dan kriteria kandidat lulus mengikuti Tabel 3.8; pemeriksaan validasi, otorisasi, dan hasil aksi API tetap terpisah dari jendela kinerja.

#### 3.2.5.4 Eksperimen Kontrol pada Lingkungan Terisolasi

Pengujian kontrol menggunakan jaringan Fabric 2.5 terpisah dengan satu kanal `rupiah-study` dan empat organisasi: `CentralBankMSP`, `DistributorMSP`, `ValidatorMSP`, dan `EndUserMSP`. `CentralBankMSP` mengoperasikan *peer* penerbit dan satu *orderer*; `ValidatorMSP` hanya menyediakan akses baca untuk audit. Basis data PostgreSQL studi menggunakan volume tersendiri dan data sintetis.

Arsitektur integrasi dan pemisahan domain ditunjukkan pada Gambar 3.31.

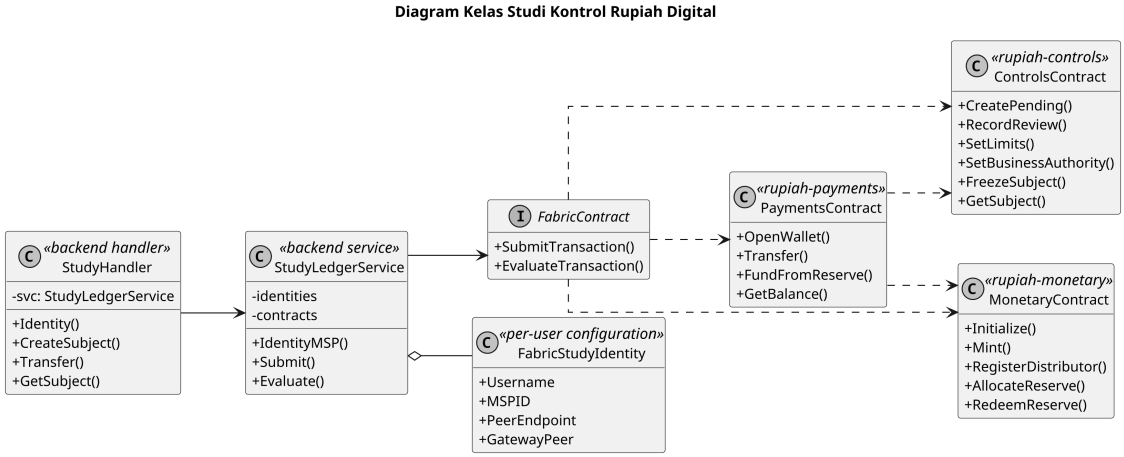



Gambar 3.31. Arsitektur lingkungan studi kontrol dengan empat MSP, tiga *chaincode*, identitas per pengguna, dan basis data PostgreSQL terisolasi.

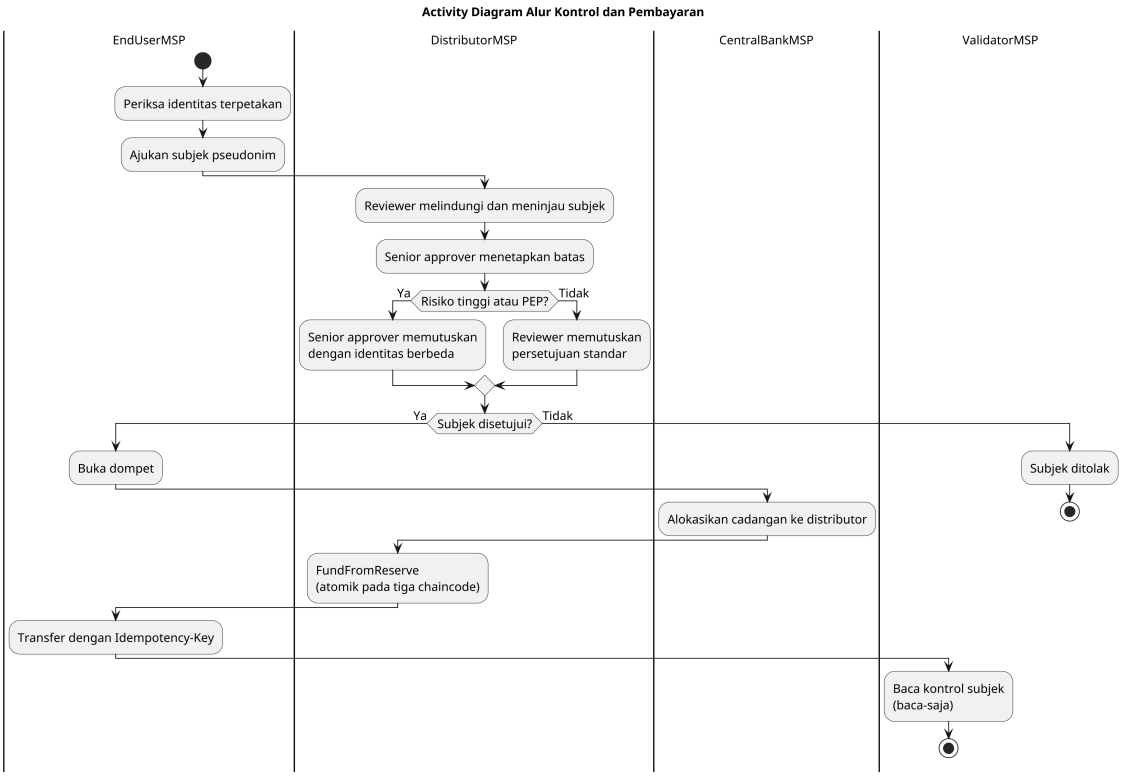
Tiga *chaincode* memisahkan kepemilikan state dan kebijakan default: `rupiah-monetary` mengelola Treasury, suplai, dan cadangan dengan `AND('CentralBankMSP.peer','DistributorMSP.peer')`; `rupiah-controls` mengelola subjek pseudonim, keputusan KYC, risiko, peran staf, dan batas dengan kebijakan default `AND(DistributorMSP, OR(CentralBankMSP, EndUserMSP))`; `rupiah-payments` mengelola dompet dan transfer dengan `AND('DistributorMSP.peer','EndUserMSP.peer')`. Kunci kontrol efektif menggunakan kebijakan berbasis state yang mensyaratkan `CentralBankMSP` dan `DistributorMSP`. Pendanaan cadangan memanggil ketiga *chaincode* dalam satu transaksi sehingga seluruh kebijakan yang terlibat harus terpenuhi. Matriks ini merupakan rancangan eksperimental, bukan pernyataan bahwa aturan hukum menetapkan kebijakan *endorsement* tersebut.

Identitas klien X.509 dipetakan per nama pengguna dan dikelola oleh layanan *backend*; identitas penanda tangan, subjek pelanggan, dan kustodian dompet tetap dipisahkan. Uji integrasi menggunakan subjek sintetis untuk keputusan KYC berisiko tinggi dengan dua staf berbeda, pembatasan dan pembukaan pembatasan dengan bukti, pendanaan, pembayaran, pembacaan oleh `ValidatorMSP`, serta transaksi negatif dengan dukungan yang tidak mencukupi. Pengujian ini tidak mengukur kinerja tiga *chaincode* dan tidak mencakup verifikasi identitas langsung, umpan penyaringan eksternal, pengiriman laporan PPATK, atau efektivitas deteksi pada data nyata.

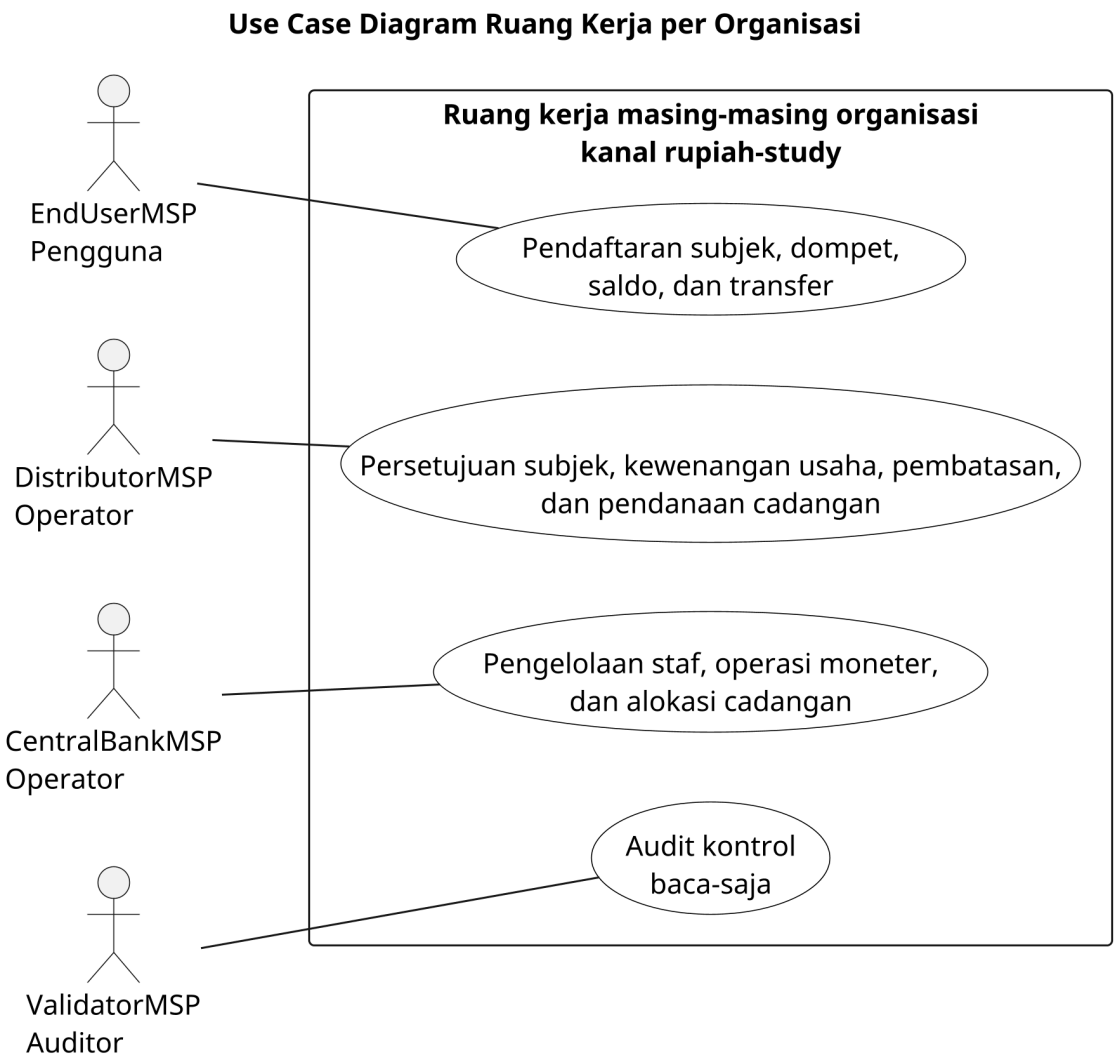
Struktur kelas, fungsi setiap organisasi, dan aktivitas utama pada lingkungan studi dirangkum melalui diagram kelas (Gambar 3.32), activity diagram (Gambar 3.33), dan diagram use case (Gambar 3.34). Urutan pesan untuk pelepasan pembatasan, konversi *wholesale-to-retail*, dan transfer ritel ditunjukkan pada Gambar 3.35, Gambar 3.36, dan Gambar 3.37. Seluruh diagram membatasi uraian pada empat MSP dan tiga *chaincode* eksperimen kontrol.



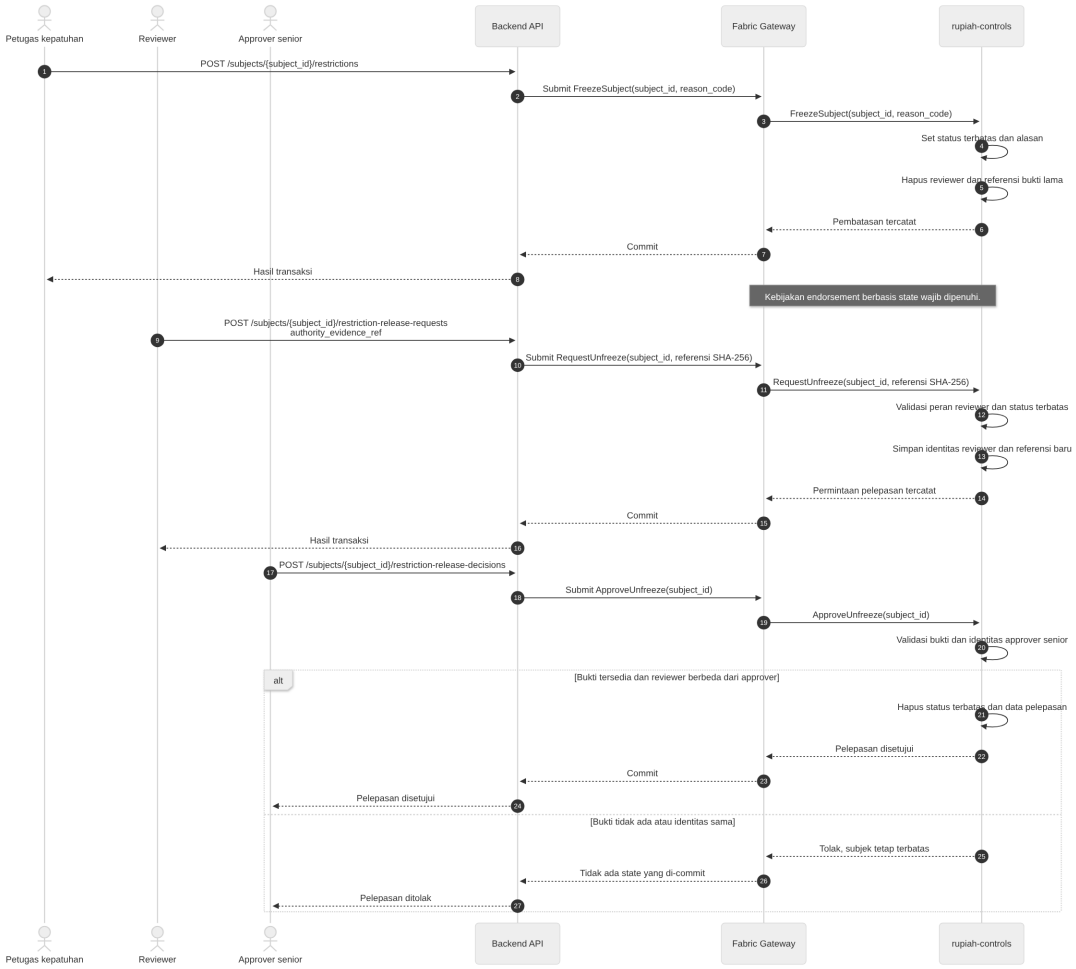

Gambar 3.32. Diagram kelas layanan backend dan tiga *chaincode* pada studi kontrol.



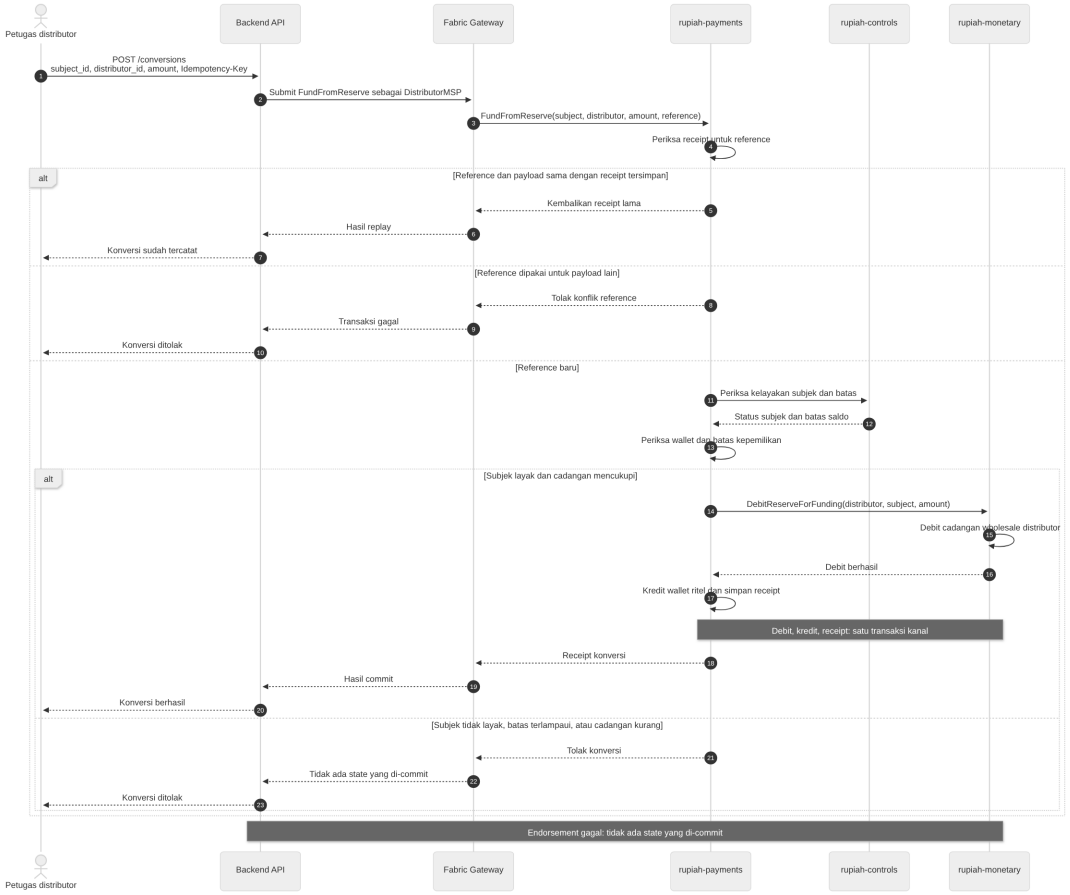

Gambar 3.33. Activity diagram alur onboarding subjek, persetujuan, pendanaan, dan transfer pada studi kontrol.



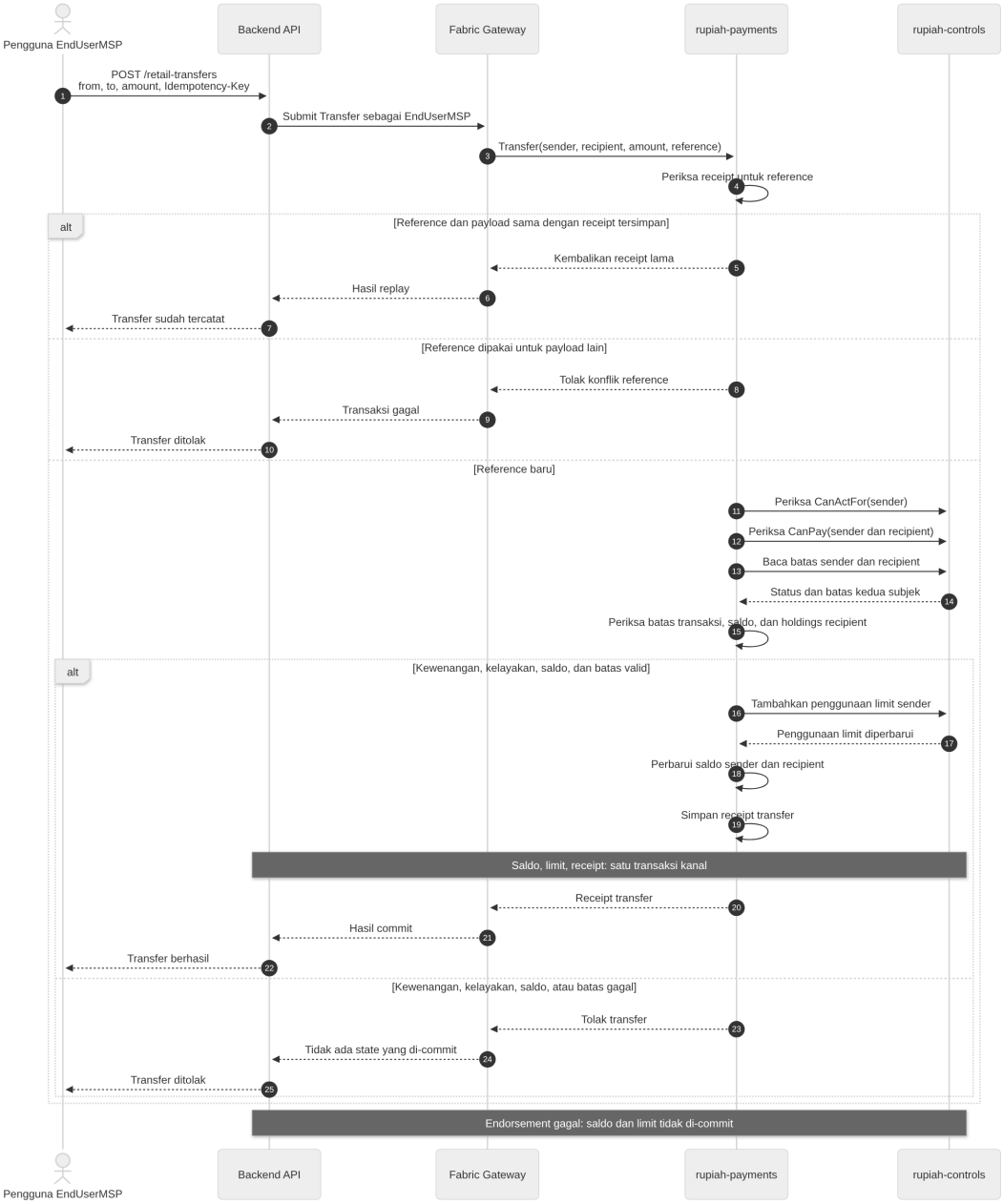

Gambar 3.34. Diagram use case fungsi empat organisasi pada studi kontrol.




Gambar 3.35. Urutan pembatasan subjek dan pelepasan pembatasan dengan bukti otoritas dan dua identitas staf berbeda.




Gambar 3.36. Urutan konversi *wholesale-to-retail* dari cadangan distributor ke *wallet* ritel dalam satu transaksi kanal.




Gambar 3.37. Urutan transfer ritel dengan pemeriksaan kewenangan, kelayakan, batas, saldo, dan receipt.

# BAB IV
# HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric beserta pembahasannya. Bagian pertama memaparkan hasil implementasi sistem yang menjawab tujuan pertama, bagian kedua memaparkan hasil evaluasi kinerja yang menjawab tujuan kedua, dan bagian ketiga membandingkan hasil penelitian dengan penelitian terdahulu.

## 4.1 Hasil Implementasi Sistem

Bagian ini melaporkan implementasi baseline `digital-rupiah` v3.0 dan hasil kinerjanya. Lingkungan kontrol empat MSP dan tiga *chaincode* dilaporkan terpisah pada bagian akhir bab ini.

### 4.1.1 Jaringan Hyperledger Fabric

Jaringan *blockchain* berhasil dibangun dengan lima organisasi peserta (BankIndonesiaOrg, HimbaraBankOrg, CommercialBankOrg, PJPOrg, dan OJKObserverOrg) serta satu organisasi *orderer*. Setiap organisasi peserta menjalankan satu *peer* dengan basis data status CouchDB, dan layanan pengurutan menggunakan konsensus Raft (etcdraft). Seluruh komponen dijalankan sebagai *container* melalui Docker Compose. Material kriptografi, blok genesis, dan konfigurasi kanal dibangkitkan secara otomatis melalui skrip, kemudian *chaincode* `digital-rupiah` dipaketkan, dipasang pada seluruh organisasi, lalu definisinya disetujui sesuai kebijakan siklus hidup yang memerlukan Bank Indonesia dan salah satu peserta institusional. Kebijakan *endorsement* kanal menerima MSP institusional yang ditetapkan sehingga pengamat OJK tidak menjadi pihak penulis atau pemberi persetujuan *chaincode*.

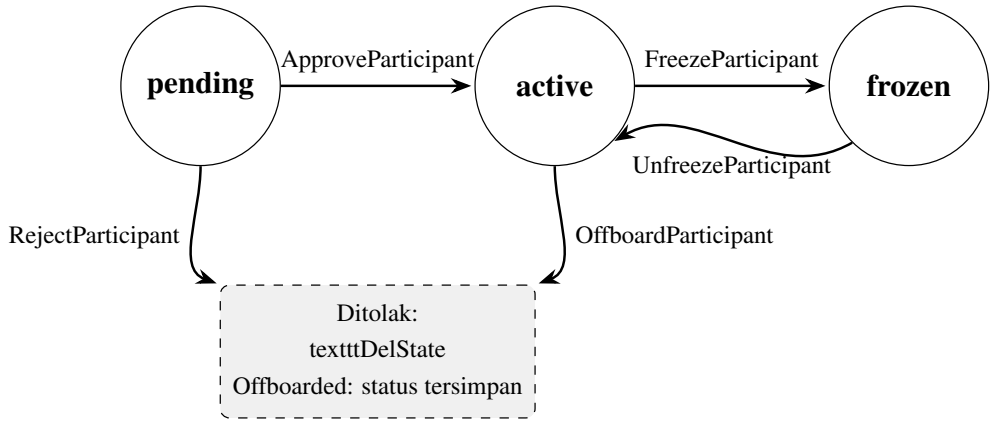
### 4.1.2 Chaincode Digital Rupiah

*Chaincode* `digital-rupiah` v3.0 mengimplementasikan logika bisnis Rupiah Digital pada bahasa Go. Fungsi-fungsi *chaincode* dikelompokkan menurut domainnya sebagaimana dirangkum pada Tabel 4.1. Seluruh operasi yang mengubah status *ledger* mencatatkan jejak audit dan/atau *event* pengawasan sehingga dapat ditelusuri oleh pengawas.

Tabel 4.1. Kelompok fungsi chaincode digital-rupiah

| **Kelompok** | **Fungsi utama** |
|---|---|
| Peserta | `SubmitParticipant`, `ApproveParticipant`, `FreezeParticipant`, `UnfreezeParticipant`, `RejectParticipant`, `OffboardParticipant` |
| Dompet | `CreateWallet`, `CreateWholesaleWallet`, `Mint`, `Burn`, `Transfer`, `FreezeWallet`, `UnfreezeWallet` |
| Likuiditas | `RequestIssuance`, `RequestIssuanceRtgs`, `DistributeToParticipant`, `RequestRedemption` |
| Pengawasan | `GetReconciliationReport`, `GetMetrics`, `GetSupervisionEvents`, `GetTransactionHistory` |

Pemeriksaan otorisasi dan keselamatan transfer dilakukan langsung pada *chaincode*. Dompet beku tidak dapat bertransaksi, sedangkan mutasi saldo memerlukan MSP kustodian yang sesuai. Gambar 4.38 menunjukkan siklus hidup peserta dari pendaftaran hingga pelepasan keanggotaan.




Gambar 4.38. Siklus hidup peserta pada *chaincode*: dari *pending* (menunggu persetujuan) menjadi *active*, kemudian dapat dibekukan (*frozen*) dan dicairkan kembali. Penolakan menghapus peserta yang masih *pending*, sedangkan `OffboardParticipant` menyimpan status *offboarded*, membekukan dompet terkait, dan mensyaratkan saldo agregat nol.

Pada fungsi `Transfer`, sistem memeriksa status beku dompet, otorisasi pemanggil, nilai positif, dan kecukupan saldo pengirim sebelum transaksi dieksekusi

secara atomik. Transfer, distribusi, dan notifikasi RTGS juga menggunakan referensi bisnis serta receipt yang dapat diverifikasi. Transfer memeriksa MSP kustodian dompet pengirim sebelum perubahan saldo.

### 4.1.3 Backend API

Lapisan *backend* menyediakan antarmuka REST yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* melalui *Fabric Gateway SDK*. Kontrol akses berbasis peran ditegakkan melalui *middleware* dengan tujuh peran sebagaimana dirangkum pada Tabel 4.2. Pemisahan peran ini memastikan bahwa, misalnya, hanya peran `bank_indonesia` yang dapat melakukan penerbitan, sedangkan peran `supervisor` hanya memiliki akses baca terhadap laporan pengawasan. Untuk pelanggan dan pedagang, backend juga membatasi daftar dompet berdasarkan nama pengguna pada JWT dan menolak transfer apabila `owner_id` dompet pengirim tidak cocok. Pada batas jaringan, chaincode memeriksa MSP: mutasi moneter dan siklus hidup peserta hanya menerima MSP Bank Indonesia, sedangkan MSP pengamat tidak dapat melakukan mutasi.

Tabel 4.2. Peran akses pada backend API

| **Peran** | **Kewenangan utama** |
|---|---|
| `public` | Pemeriksaan kesehatan dan topologi jaringan |
| `authenticated` | Pembacaan daftar dompet |
| `bank_pjp` / `validator_bank` / `pjp` | Pembuatan dompet, pendanaan dompet ritel, dan pengajuan peserta |
| `bank_indonesia` | Penerbitan, distribusi/penebusan likuiditas, siklus hidup peserta, dan pengawasan |
| `merchant` | Transfer dan pembacaan saldo pedagang yang telah disetujui |
| `supervisor` | Akses baca laporan dan pengawasan |

### 4.1.4 Hasil Pengujian Fungsional

Pengujian fungsional dilakukan untuk memverifikasi setiap alur proses yang dirancang pada BAB III. Hasil pengujian menunjukkan bahwa seluruh alur berjalan sesuai rancangan, yaitu:

1. **Penerbitan dan distribusi berjenjang**. Bank Indonesia menerbitkan Rupiah Digital hanya ke dompet `bi_treasury`. Bank Indonesia kemudian mendistribusikan likuiditas secara langsung dari Treasury kepada peserta aktif bertipe bank validator atau PJP melalui `DistributeToParticipant`. Kustodian tersebut menyalurkan saldo kepada pelanggan ritel melalui `Transfer`; penerbitan langsung ke dompet ritel ditolak oleh aturan `Mint`.
2. **Transfer dan status dompet**. Transfer ditolak ketika pemanggil tidak berwenang, dompet beku, nilai tidak valid, atau saldo tidak mencukupi. Jalur `Transfer` yang sama berhasil digunakan untuk transaksi antarpelanggan dan pelanggan-ke-pedagang.
3. **Pengawasan**. Laporan rekonsiliasi, metrik, dan *event* pengawasan dapat diambil oleh peran berwenang.

Selain pengujian berbasis skenario tersebut, verifikasi fungsional juga diperkuat dengan *unit test* otomatis pada lapisan *backend* dan *chaincode*. Hasil verifikasi menunjukkan bahwa paket uji *backend* lulus seluruh pengujian dengan 76 pengujian pada 6 paket, sedangkan paket uji *chaincode* lulus seluruh pengujian dengan 58 pengujian pada 1 paket. Pada lapisan *backend*, pengujian tersebut memverifikasi pemilihan metode layanan ke *chaincode* beserta urutan argumennya, pembentukan *anchor hash*, propagasi kegagalan penyimpanan *off-chain*, serta penanganan layanan untuk pembuatan pelanggan ritel, transfer, penerbitan, penebusan, dan distribusi kepada peserta. Pada lapisan *chaincode*, pengujian memverifikasi aksi siklus hidup peserta, batasan pembuatan dompet, jalur berhasil dan jalur penolakan transfer terkait otorisasi, status dompet, dan saldo, disertai persistensi serta pembaruan status atau *event* transaksi yang relevan. Temuan tersebut mendukung kebenaran logika layanan *backend* dan aturan bisnis *chaincode* pada purwarupa, namun tidak ditafsirkan sebagai bukti validasi menyeluruh, pembuktian *end-to-end*, atau kesiapan sistem untuk lingkungan produksi berskala penuh.

Jika dirujukkan ke SRS pada Tabel 3.3, hasil pada subbab ini memberikan bukti utama untuk FR-01 sampai FR-05 dan NFR-02 sampai NFR-04. FR-01 dan FR-02 didukung oleh keberhasilan skenario pendaftaran peserta, penerbitan, dan distribusi dua jenjang. FR-03 didukung oleh *unit test* dan skenario fungsional yang menunjukkan bahwa otorisasi, status dompet, saldo, dan pembaruan atomik berjalan sesuai rancangan. FR-04 dan FR-05 didukung oleh skenario pengawasan dan verifikasi hak akses pada lapisan *backend*. Dengan demikian, hasil pengujian fungsional pada bab ini berperan sebagai bukti bahwa kebutuhan perangkat lunak utama purwarupa telah diimplementasikan dan diverifikasi pada tingkat yang sesuai untuk penelitian prototipe.

## 4.2 Hasil Evaluasi Kinerja Final

Profil kinerja final dijalankan pada jaringan yang dibangun ulang dari keadaan bersih untuk setiap profil. Manifes final dan manifes studi pengganti mencatat *seed* suite dan *seed* workload, *chaincode* `digital-rupiah` versi 3.0 sequence 1, Hyperledger Caliper 0.6.0, dan hash masukan serta artefak. Pengujian memakai satu *host*, lima organisasi, satu *peer* per organisasi, satu *orderer* Raft, dan CouchDB pada setiap *peer*. Topologi tersebut menjadi konteks hasil dan tidak menunjukkan toleransi kegagalan produksi.

Tabel 4.3. Lingkungan pengujian kinerja final

| **Komponen** | **Nilai** |
|---|---|
| Host | Intel Core i7-7700HQ, 4 inti fisik, 8 thread, 2,80 GHz |
| Memori | 15,52 GiB |
| Runtime | Node.js 22.23.2, npm 10.9.8, Docker client 29.5.3 / Engine 29.8.1 |
| Fabric / CouchDB | 2.5 / 3.2 |
| Caliper | 0.6.0 |
| Jaringan / kontrak | Garuda / `digital-rupiah` v3.0, sequence 1 |
| *Seed suite / workload* | 20260909 / 20260725 |

Pengujian primer menggunakan satu *worker* yang dipilih setelah kalibrasi 1, 2, 4, dan 8 *worker*. Setiap jenis transaksi diuji secara terpisah dengan lima pengulangan pada laju tetap. Transfer ritel memiliki profil kontensi rendah dan kontensi tinggi; tidak ada komposisi 70/25/5. Laju yang ditawarkan adalah jumlah transaksi yang dikirim dibagi durasi pengukuran, sedangkan laju yang berhasil dicapai adalah jumlah penyelesaian berhasil dibagi durasi yang sama. Latensi yang dilaporkan menggunakan batas Fabric Caliper, `time_final - time_create`, untuk penyelesaian berhasil.

Studi pengganti menetapkan jumlah transfer ritel sebesar Rp100, lebih rendah daripada batas harian dompet STANDARD sebesar Rp10.000.000. Jumlah tersebut digunakan sebagai kontrol sintetis untuk memisahkan pengaruh batas kebijakan dompet dari kontensi, bukan sebagai representasi nominal pembayaran ritel. Profil berpasangan menguji kontensi rendah dan tinggi pada 20, 75, dan 150 TPS dengan lima pengulangan. Studi pengganti Rp1.000 mengulang distribusi pada 75 TPS, penebusan pada 20 TPS, serta kalibrasi transfer ritel pada 30, 40, dan 50 TPS.

Tabel 4.4. Beban berkelanjutan pada pengujian primer

| Fungsi | Kontensi | Beban | Offered | Achieved | Sukses (%) | p95 (s) | p99 (s) |
|---|---|---|---|---|---|---|---|
| Kueri saldo | rendah | 190 | 190,0 | 190,0 | 100,00 | 0,01 | 0,01 |
| Pembekuan dompet | rendah | 50 | 50,0 | 50,0 | 100,00 | 0,28 | 0,37 |
| Pembukaan pembekuan | rendah | 65 | 65,0 | 65,0 | 100,00 | 0,27 | 0,35 |
| Penebusan | rendah | 25 | 25,0 | 25,0 | 100,00 | 0,53 | 0,60 |
| Transfer ritel | rendah | 20 | 20,0 | 20,0 | 100,00 | 0,72 | 0,89 |
| Pembayaran pedagang | rendah | 20 | 20,0 | 20,0 | 100,00 | 0,66 | 0,75 |
| Penerbitan | rendah | – | – | – | <99,00 | – | – |
| Distribusi | rendah | – | – | – | <99,00 | – | – |
| Transfer ritel | tinggi | – | – | – | <99,00 | – | – |

Beban berkelanjutan ditetapkan sebelum interpretasi hasil sebagai laju tertinggi yang mencapai sedikitnya 99% penyelesaian berhasil dan p95 latensi transaksi berhasil paling tinggi 2,5 detik pada seluruh lima pengulangan. Tabel ini menunjukkan batas tersebut untuk enam profil. Penerbitan, distribusi, dan transfer ritel dengan kontensi tinggi tidak memiliki laju berkelanjutan pada rentang yang diuji; tanda "–" bukan nilai nol, melainkan ketiadaan titik yang memenuhi kriteria. Laju uji dan statistik lengkap per pengulangan tersedia pada artefak *summary* benchmark.

Tabel 4.5. Kategori kegagalan pada pengujian primer pengganti

| Beban kerja | Kategori | Jumlah |
|---|---|---|
| Distribusi / kalibrasi Rp1.000 | *MVCC read conflict* | 46.716 |
| Transfer rendah Rp100 | *MVCC read conflict* | 68.922 |
| Transfer rendah Rp100 | Batas konkurensi gateway | 45.774 |
| Transfer tinggi Rp100 | *MVCC read conflict* | 110.570 |
| Transfer tinggi Rp100 | Batas konkurensi gateway | 1.501 |

Pada studi pengganti, analyzer mengklasifikasikan 272.924 kegagalan sebagai *MVCC read conflict* dan 47.275 sebagai kegagalan batas konkurensi gateway. Angka tersebut hanya mencakup beban kerja pengganti pada tabel, bukan seluruh operasi pada suite bukti final sebelumnya. Konflik MVCC terjadi ketika transaksi membaca versi state yang berubah sebelum validasi, sedangkan batas konkurensi gateway menunjukkan kejenuhan pada batas Fabric Gateway. Kedua kategori tersebut tidak ditafsirkan sebagai bukti tunggal dari bottleneck produksi.

Kontrol berpasangan Rp100 memperjelas pengaruh kontensi. Pada 20 TPS, kontensi rendah menyelesaikan 2.401 dari 2.401 permintaan, dengan laju berhasil 20,01 TPS, tingkat keberhasilan 100%, dan p95 0,63 detik. Kontensi tinggi hanya mencapai sekitar 10% penyelesaian, dengan laju berhasil 2,01 TPS dan p95 0,59 detik. Pada 75

dan 150 TPS, kedua profil juga tidak memenuhi kriteria keberlanjutan karena menghasilkan konflik MVCC atau kegagalan batas konkurensi gateway.

Tabel 4.6. Statistik pengulangan pada beban berkelanjutan

| **Fungsi** | **Beban** | **Offered** | **Achieved** | **Sukses (%)** | **p95 (s)** |
|---|---|---|---|---|---|
| Kueri saldo | 190 | 190,0 | 190,0 | 100,00 | 0,01 |
| Pembekuan dompet | 50 | 50,0 | 50,0 | 100,00 | 0,28 |
| Pembukaan pembekuan | 65 | 65,0 | 65,0 | 100,00 | 0,27 |
| Penebusan | 25 | 25,0 | 25,0 | 100,00 | 0,53 |
| Transfer ritel | 20 | 20,0 | 20,0 | 100,00 | 0,72 |
| Pembayaran pedagang | 20 | 20,0 | 20,0 | 100,00 | 0,66 |

Nilai pada tabel merupakan rerata lima pengulangan untuk setiap fungsi pada beban berkelanjutan tertinggi yang memenuhi kriteria. Simpangan baku antar-pengulangan dilaporkan pada berkas *summary* dan digunakan untuk menilai variasi, sedangkan nilai p95 dipakai sebagai batas latensi untuk menentukan keberlanjutan beban. Penerbitan, distribusi, dan kontensi tinggi tidak dimasukkan karena tidak memiliki beban primer yang memenuhi kriteria.

Tabel 4.7. Demonstrasi terpisah PostgreSQL/KYC

| **Tahap** | **Pengulangan** | **Rerata (ms)** | **SD (ms)** |
|---|---|---|---|
| Login | 5 | 186,85 | 34,77 |
| Pembuatan pelanggan | 5 | 2.069,02 | 8,60 |
| Pengajuan KYC | 5 | 2.071,99 | 7,02 |
| Penyegaran KYC | 5 | 2.064,70 | 8,31 |
| Pembuatan dompet | 5 | 2.075,16 | 6,05 |
| Pembacaan profil KYC | 5 | 1,14 | 0,41 |

Demonstrasi ini mengukur alur Go/PostgreSQL/KYC secara *end-to-end* dan tidak digabungkan dengan pengujian Fabric. Lima pengulangan menyelesaikan alur login, pembuatan pelanggan, pengajuan dan penyegaran KYC, pembuatan dompet, serta pembacaan profil. Waktu KYC dan pembuatan dompet berada pada kisaran dua detik karena melibatkan layanan backend, PostgreSQL, dan koneksi ledger.

Tabel 4.8. Hasil konformitas semantik Caliper final

| **Profil** | **Sukses** | **Gagal** | **Hasil oracle** |
|---|---|---|---|
| Otorisasi | 2 | 1 | PASS; satu penolakan MSP kustodian yang memang diharapkan. |
| Batas transaksi | 4 | 4 | PASS; delapan kasus lulus, nol ketidakcocokan, dan nol kesalahan infrastruktur. |
| Jalur negatif | 0 | 7 | PASS; tujuh dari tujuh aturan penolakan ditegakkan. |
| Agregat *overspend* | 10 | 190 | PASS; saldo akhir Rp40.000 tetap nonnegatif. |

Kolom "gagal" pada tabel konformitas tidak selalu berarti cacat sistem. Pada jalur negatif, batas transaksi, otorisasi, dan *overspend*, penolakan merupakan keluaran yang diharapkan dan dinilai oleh oracle semantik. Profil agregat mencatat sepuluh transfer yang dikomit dan 190 penolakan dari total 200 permintaan terhadap saldo awal Rp50.000; saldo akhir Rp40.000 memenuhi invariant nonnegatif. Bukti ini mendukung aturan saldo pada topologi yang diuji, tetapi tidak merupakan pembuktian formal keamanan.

Tabel 4.9. Metrik sumber daya yang dikumpulkan pada pengujian primer

| **Lapisan** | **Metrik** |
|---|---|
| Host | CPU, memori, IOPS, dan latensi disk |
| Container Fabric | CPU, memori, jaringan, IOPS, dan latensi layanan disk |
| Klien Caliper | CPU proses klien |
| Prometheus | CPU dan memori pemantauan |

Pemantauan mencakup I/O jaringan, I/O disk, IOPS, latensi layanan disk, penggunaan sumber daya setiap container Fabric, dan CPU klien Caliper. Nilai mentah serta ringkasan rerata, p95, dan maksimum tersedia pada artefak sumber daya benchmark. Karena penerbitan, distribusi, dan kontensi tinggi tidak memenuhi kriteria keberlanjutan, interpretasi bottleneck dibatasi pada konfigurasi dan beban yang benar-benar diuji.

Dalam konteks SRS pada Bab III, hasil ini memberikan bukti untuk NFR-04 dan NFR-05 serta melengkapi bukti fungsional sebelumnya. Hasil fungsional dan konformitas menunjukkan bahwa alur dan batas kebijakan berjalan pada skenario yang ditetapkan. Hasil kinerja menunjukkan karakteristik yang terukur, termasuk penurunan keberhasilan dan konflik pada beban tinggi. Tidak ada angka pada subbab ini yang ditafsirkan sebagai kapasitas nasional, jaminan produksi, atau reproduktibilitas lintas mesin.

## 4.3 Perbandingan Hasil Penelitian dengan Hasil Terdahulu

Penelitian ini dibandingkan dengan tinjauan literatur Aryani [28] dan PoC Proyek Garuda [11] karena keduanya membahas kebutuhan CBDC Indonesia dari sisi kebijakan dan pembuktian teknologi. Tabel 4.10 membandingkan jenis artefak, lingkup bukti, dan posisi kontribusi. Jika angka kinerja disebutkan, angka tersebut hanya menjelaskan konfigurasi masing-masing artefak dan tidak dimaksudkan sebagai peringkat langsung antarsistem. Dalam kerangka itu, penelitian ini menyajikan purwarupa CBDC ritel berbasis Hyperledger Fabric dengan pengawasan dan tolok ukur Caliper, sedangkan Aryani berfokus pada kajian literatur dan PoC Proyek Garuda pada Rupiah Digital grosir.

Tabel 4.10. Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024).

| **Dimensi** | **Aryani (2023)** | **PoC Garuda (2024)** | **Penelitian ini** |
|---|---|---|---|
| Jenis CBDC | Ritel (analisis konseptual) | Grosir (PoC eksperimental) | Ritel (purwarupa) |
| Metodologi | Tinjauan literatur kritis komparatif | PoC dengan 55 skenario bisnis dan teknis | Purwarupa fungsional + Caliper |
| Platform DLT | Tidak dievaluasi (kutipan dari kasus negara) | R3 Corda + Hyperledger Besu | Hyperledger Fabric |
| Distribusi | Dua jenjang secara konseptual | Grosir pada PoC | BI Treasury–bank/PJP kustodian–ritel |
| Transfer ritel | Direkomendasikan secara konseptual | Tidak diuji | Antarpelanggan dan pelanggan–pedagang |
| Metrik kinerja | Kutipan sekunder; tidak diukur langsung | Corda 37 TPS, Besu 39 TPS (Locust) | Caliper primer lima pengulangan per laju; batas berkelanjutan 190 TPS untuk kueri saldo, 50 TPS untuk pembekuan, 65 TPS untuk pembukaan pembekuan, 25 TPS untuk penebusan, dan 20 TPS untuk transfer ritel/pembayaran pedagang pada kontensi rendah |
| Lingkup pengujian | Tidak ada pengujian | Penerbitan, penebusan, transfer antarlembaga | Transfer antarpelanggan, pelanggan–pedagang, baca dompet |
| Pembayaran luring | Direkomendasikan untuk r-Digital Rupiah | Tidak dibahas | Tidak diuji dalam purwarupa ini |
| Tokenisasi | Didukung (konseptual) | UTXO (Corda) / ERC-20 + ERC-1400 (Besu) | Transfer berbasis kunci–nilai pada Fabric |

Rohmalia dan Nugraha [29] juga melaporkan purwarupa CBDC ritel berbasis Fabric dengan pembuatan akun dan transfer antarpengguna. Implementasi penelitian ini mencakup distribusi melalui bank/PJP, pembayaran pedagang, dan pengawasan, lalu mencatat kinerjanya melalui Caliper. Waktu pengiriman satu dan 50 data yang dilaporkan Rohmalia dan Nugraha tidak dibandingkan langsung dengan *throughput* atau

latensi penelitian ini karena beban dan batas pengukurannya berbeda.

Perbandingan ini terbatas pada posisi kontribusi dan ruang lingkup teknis. Kalibrasi menggunakan satu, dua, empat, dan delapan *worker*, sedangkan pengujian primer menggunakan satu *worker* yang dipilih berdasarkan beban berkelanjutan tertinggi. Hasilnya tetap berlaku hanya untuk topologi, jumlah peserta, dan skenario beban purwarupa. Selisih angka kinerja terhadap PoC Proyek Garuda (Corda 37 TPS dan Besu 39 TPS) bersumber pada perbedaan platform, *harness*, tujuan pengujian, dan komposisi beban, bukan pada klaim keunggulan langsung Hyperledger Fabric. Variasi tingkat sukses pada lima pengulangan setiap beban menunjukkan bahwa hasil ini merupakan garis dasar lokal yang dapat diulang, bukan kapasitas maksimum Hyperledger Fabric atau bukti reproduksibilitas lintas mesin [33].

## 4.4 Hasil Uji Kontrol pada Lingkungan Terisolasi

Lingkungan studi kontrol dijalankan terpisah dari jaringan lima organisasi yang digunakan untuk pengukuran Caliper. Empat *peer* bergabung pada kanal `rupiah-study`; tiga *chaincode*, yaitu `rupiah-monetary`, `rupiah-controls`, dan `rupiah-payments`, berhasil dikomit dengan persetujuan definisi dari keempat MSP.

Urutan pesan pada pembatasan dan pelepasan, konversi *wholesale-to-retail*, serta transfer ritel ditampilkan pada Gambar 3.35, Gambar 3.36, dan Gambar 3.37. Hasil pengujian berikut mengevaluasi alur kontrol sintetis tersebut.

*Unit test* pada ketiga modul *chaincode* dan seluruh paket *backend* lulus. Uji integrasi sintetis pada jaringan juga lulus untuk pendaftaran subjek, persetujuan risiko tinggi oleh staf berbeda, alur pembukaan dompet dan pembayaran, pendanaan dari cadangan, pembatasan dan pembukaan pembatasan dengan bukti otoritas, serta pembacaan subjek oleh `ValidatorMSP`. Uji tersebut menolak transaksi moneter, pembayaran, pendanaan, dan perubahan kontrol ketika organisasi pendukung tidak memenuhi kebijakan yang berlaku.

Hasil ini menunjukkan perilaku fungsional pada skenario sintetis yang diuji, bukan kepatuhan hukum menyeluruh, efektivitas deteksi AML/CFT pada populasi nyata, atau keberhasilan pengiriman laporan kepada PPATK.

Pengujian kinerja untuk ketiga *chaincode* studi dijalankan dengan satu *worker* Caliper pada satu host lokal. Satuan laju dinyatakan dalam ops/s; untuk operasi tulis, nilai dalam ops/s sama dengan TPS. Dari 31 beban kerja, seluruhnya telah diukur dan dikonfirmasi pada *stamp* `run-20261014-01`, dan 30 di antaranya memiliki braket terukur. Braket `fund-reserve`, 0,183594 hingga 0,21875 ops/s dengan lebar sekitar 19 persen, belum tertutup: dua probe batas atasnya gagal oleh galat pengambilan (*fetch*

*failed*) sebelum pengukuran, sehingga batas gagal pertama yang terukur adalah 0,21875 ops/s. Pengklasifikasi yang dipakai pada run ini mencatat galat transport sebagai kegagalan transaksi; perbaikan kode dilakukan setelah run dan belum tercermin pada hasil ini. Run ini sempat terhenti karena gangguan daya perangkat, yang tidak tercatat dalam artefak run, dan dilanjutkan dengan *stamp* yang sama dari titik pemeriksaan terakhir. Setelah pemulihan, jaringan dinaikkan kembali dengan materi identitas yang sudah ada; definisi chaincode yang telah ter-commit dilewati dan tidak dipasang ulang. Dua beban baca pembayaran yang diukur setelah pemulihan (272 dan 296 ops/s) lebih rendah daripada delapan beban baca kontrol dan moneter yang diukur sebelumnya (296 dan 320 ops/s); karena chaincode dan pemulihan saling terkait, selisih ini tidak dapat dikaitkan dengan salah satunya. Konfirmasi `fund-reserve` pada percobaan sebelumnya gagal dengan HTTP 500 dan tidak dihitung, sehingga pencariannya diulang. Seluruh pemeriksaan kebenaran lulus, yaitu 20 dari 20 skenario, 207 dari 207 sel otorisasi, 27 dari 27 kasus validasi, dan 15 dari 15 hasil aksi, tanpa pemeriksaan yang tidak terselesaikan. Kapasitas terkonfirmasi disajikan pada Tabel 4.11.

Tabel 4.11. Batas bawah laju lulus terkonfirmasi tiga *chaincode* studi pada satu *worker* (ops/s)

| Beban kerja | *Chaincode* | Jenis | Terkonfirmasi (ops/s) | Gagal pertama (ops/s) |
|---|---|---|---|---|
| `create-pending` | `rupiah-controls` | tulis | 4,75 | 5,125 |
| `protect-subject` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `create-staff-pending` | `rupiah-controls` | tulis | 0,195313 | 0,207031 |
| `protect-staff` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `set-staff-role` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `record-review` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `approve-standard` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `decide-high-risk` | `rupiah-controls` | tulis | 0,195313 | 0,207031 |
| `set-limits` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `set-business-authority` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `freeze-subject` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `request-unfreeze` | `rupiah-controls` | tulis | 0,78125 | 0,828125 |
| `approve-unfreeze` | `rupiah-controls` | tulis | 0,195313 | 0,207031 |
| `read-caller-identity` | `rupiah-controls` | baca | 296 | 320 |
| `can-pay` | `rupiah-controls` | baca | 296 | 320 |
| `read-limits` | `rupiah-controls` | baca | 296 | 320 |
| `can-act-for` | `rupiah-controls` | baca | 296 | 320 |
| `validator-read-subject` | `rupiah-controls` | baca | 296 | 320 |
| `register-distributor` | `rupiah-monetary` | tulis | 0,78125 | 0,828125 |
| `mint` | `rupiah-monetary` | tulis | 0,195313 | 0,207031 |
| `allocate-reserve` | `rupiah-monetary` | tulis | 0,195313 | 0,207031 |
| `redeem-reserve` | `rupiah-monetary` | tulis | 0,195313 | 0,207031 |
| `read-treasury` | `rupiah-monetary` | baca | 296 | 320 |
| `read-total-supply` | `rupiah-monetary` | baca | 296 | 320 |
| `read-reserve` | `rupiah-monetary` | baca | 296 | 320 |
| `transfer-retail` | `rupiah-payments` | tulis | 0,78125 | 0,828125 |
| `transfer-merchant` | `rupiah-payments` | tulis | 0,195313 | 0,207031 |
| `fund-reserve` | `rupiah-payments` | tulis | 0,183594 | 0,21875 |
| `read-balance` | `rupiah-payments` | baca | 272 | 296 |
| `read-receipt` | `rupiah-payments` | baca | 272 | 296 |
| `open-wallet` | `rupiah-payments` | tulis | 4,625 | 5 |

Operasi baca yang terverifikasi mencapai 272 hingga 296 ops/s yang ditawarkan; angka ini merupakan batas bawah kapasitas baca, karena pada probe yang gagal laju terkirim hanya 0,50 hingga 0,93 dari target. Laju lulus tertinggi untuk operasi tulis berada pada rentang 0,184 hingga 4,75 TPS, dengan `create-pending` (4,75 TPS) dan `open-wallet` (4,625 TPS) sebagai nilai tertinggi. Throughput terverifikasi pada laju tersebut, yaitu rata-rata tiga ronde konfirmasi per beban kerja, berada pada rentang

0,200 hingga 4,613 TPS; 19 dari 21 nilai tulis berada pada 0,78125 ops/s atau lebih rendah. Pada 20 dari 21 beban tulis (fund-reserve dikecualikan karena probe gagalnya adalah galat pengambilan), probe gagal pertama melampaui batas drain: throughput terverifikasi terhadap durasi ditambah drain harus mencapai 90 persen laju terkirim, setara dengan drain paling lama sepersembilan durasi probe (1,1 detik untuk probe 10 detik). Median drain pada probe gagal 10 detik adalah 1,73 detik, dibandingkan 0,06 detik pada probe lulus. Konfigurasi orderer studi memakai `BatchTimeout` 2 detik; ini merupakan penjelasan yang mungkin, tetapi pengukuran tidak mencatat isi blok sehingga hubungan tersebut belum diisolasi. Pada beban yang jauh lebih tinggi, penelitian terdahulu menunjukkan bahwa validasi transaksi membatasi throughput Hyperledger Fabric [57], dan bahwa overhead pada tahap pemesanan dan validasi menghambat throughput [58]; kedua studi tersebut tidak langsung menjelaskan batas di bawah 1 TPS. Angka tersebut berlaku untuk satu *worker*, satu host, dan durasi konfirmasi 30 detik, sehingga tidak dimaksudkan sebagai kapasitas produksi.

Konsumsi sumber daya dicatat setiap detik selama jendela konfirmasi 30 detik untuk 31 beban kerja yang terkonfirmasi, yaitu 30 sampel per jendela dan tiga jendela per beban kerja. Tabel 4.12 merangkum nilai per kelompok komponen: median CPU rata-rata dan median delta RX per jendela, serta puncak maksimum CPU dan memori antar jendela. Kelompok dengan beberapa instans, yaitu CouchDB dan kontainer chaincode, digabung seluruh instansnya; baris Klien Caliper hanya mencakup proses klien Caliper.

Tabel 4.12. Konsumsi sumber daya komponen pada jendela konfirmasi 30 detik (31 beban kerja)

| Komponen | CPU rata-rata median (%) | CPU puncak maks (%) | Memori puncak maks (MiB) | Delta RX median (MB) |
|---|---|---|---|---|
| Peer Fabric `centralbank` | 1,59 | 68,83 | 154,2 | 0,10 |
| Peer Fabric `distributor` | 1,44 | 6,99 | 154,1 | 0,00 |
| Peer Fabric `enduser` | 1,44 | 91,23 | 160,8 | 0,00 |
| Peer Fabric `validator` | 1,36 | 70,99 | 153,4 | 0,09 |
| Orderer Fabric | 0,12 | 1,52 | 80,8 | 0,00 |
| CouchDB | 1,04 | 70,76 | 66,8 | 0,00 |
| Fabric CA | 0,00 | 1,03 | 28,9 | 0,00 |
| PostgreSQL | 0,03 | 0,79 | 37,8 | 0,00 |
| Kontainer *chaincode* | 0,00 | 30,58 | 25,3 | 0,00 |
| Klien Caliper | 0,69 | 17,98 | 313,5 | 1,84 |

Peer Fabric memakai CPU rata-rata median 1,36% hingga 1,59% dengan puncak 6,99% hingga 91,23%, sedangkan CouchDB mencapai puncak 70,76%. Puncak memori peer berada pada 153,4 hingga 160,8 MiB. Nilai median CPU yang rendah menunjukkan bahwa separuh atau lebih jendela konfirmasi memiliki rata-rata CPU rendah; basis persentase CPU (per core atau per host) belum didefinisikan, dan penyebabnya belum diuji secara terpisah. Kolom delta menunjukkan median delta RX

per jendela konfirmasi (MB); delta I/O disk belum dirangkum pada tabel ini. Nilai ini hanya mewakili pengukuran pada satu host dan tidak dibandingkan dengan pengujian primer.

# BAB V
# KESIMPULAN DAN SARAN

## 5.1 Kesimpulan

Berdasarkan hasil rancang bangun dan pengujian, kesimpulan penelitian adalah sebagai berikut.

1. Berdasarkan rumusan masalah pertama mengenai perancangan sistem Rupiah Digital ritel berbasis Hyperledger Fabric, penelitian ini berhasil merancang dan membangun purwarupa dengan arsitektur dua jenjang yang memisahkan peran Bank Indonesia, bank validator, PJP, dan pengguna ritel. Bank Indonesia menerbitkan Rupiah Digital ke Treasury, kemudian mendistribusikannya secara langsung kepada bank validator atau PJP sebagai kustodian. Implementasi dijalankan pada Hyperledger Fabric 2.5 dengan lima organisasi, *backend* Go, CouchDB, dan PostgreSQL, sehingga alur tersebut dapat direalisasikan sebagai artefak teknis yang dapat diuji.
2. Sebagai eksperimen kontrol terpisah, penelitian ini membangun jaringan empat MSP dan tiga *chaincode* dengan data sintetis. Uji integrasi memverifikasi persetujuan risiko tinggi oleh dua staf berbeda, pembatasan dan pembukaan pembatasan berbukti, pembayaran, pendanaan, pembacaan `ValidatorMSP`, dan penolakan transaksi yang tidak memenuhi *endorsement*. Bukti ini terbatas pada skenario uji; hasilnya tidak menyatakan kepatuhan institusional atau efektivitas AML/CFT di dunia nyata.
3. Sebagai bagian dari rumusan masalah pertama mengenai transfer ritel dan pengawasan, penelitian ini menunjukkan bahwa pemisahan peran organisasi, pemeriksaan MSP kustodian, kepemilikan dompet, status beku, dan kecukupan saldo dapat ditegakkan pada lapisan *backend* dan *chaincode*. Pada pengujian konformitas, oracle otorisasi, batas transaksi, jalur negatif, dan *overspend* memenuhi hasil yang diharapkan. Jalur transfer yang sama melayani pembayaran antarpelanggan dan pelanggan–pedagang, sedangkan laporan serta *event* pengawasan tersedia bagi peran berwenang. Demonstrasi Go/PostgreSQL/KYC terpisah menyelesaikan lima pengulangan alur layanan tanpa digabungkan ke batas latensi Fabric.
4. Berdasarkan rumusan masalah kedua mengenai kinerja sistem, pengujian primer menggunakan satu *worker* yang dipilih setelah kalibrasi satu, dua, empat, dan delapan *worker*. Setiap beban diulang lima kali dengan kontrol laju tetap. Dengan kriteria sedikitnya 99% penyelesaian berhasil dan p95 latensi transaksi berhasil paling tinggi 2,5 detik pada semua pengulangan, beban berkelanjutan tertinggi

mencapai 190 TPS untuk kueri saldo, 50 TPS untuk pembekuan dompet, 65 TPS untuk pembukaan pembekuan, 25 TPS untuk penebusan, serta 20 TPS untuk transfer ritel dan pembayaran pedagang pada kontensi rendah. Penerbitan, distribusi, dan transfer ritel dengan kontensi tinggi tidak memenuhi kriteria tersebut pada laju yang diuji. Pada kontrol berpasangan dengan jumlah Rp100, kontensi rendah mencapai 20,01 TPS dengan tingkat keberhasilan 100% pada 20 TPS, sedangkan kontensi tinggi hanya mencapai 2,01 TPS dan sekitar 10% penyelesaian. Hasil ini berlaku pada satu inang dan topologi lokal, sehingga belum menetapkan kapasitas nasional atau reproduktibilitas lintas mesin.

## 5.2 Saran

1. Menguji sensitivitas kapasitas tulis terhadap `BatchTimeout` orderer. Perbandingan jumlah *worker* merupakan penelitian lanjutan karena pengukuran pada studi ini dibatasi pada satu *worker* dan satu host.
2. Mengulang protokol Caliper pada beberapa perangkat keras dan topologi jaringan, kemudian menjalankan *soak test* berdurasi panjang untuk menguji kestabilan hasil di luar satu inang.
3. Menelusuri kegagalan penerbitan dan distribusi serta konflik MVCC dan batas konkurensi gateway dengan instrumentasi tahap *endorsement*, validasi, dan pengiriman agar penyebab bisnis dan infrastruktur dapat dipisahkan.
4. Memperluas skenario kontensi dompet bersama dan menguji kebijakan *retry* serta *backoff* secara eksplisit terhadap konflik MVCC tanpa mengubah makna idempotensi transaksi.
5. Menguji *private data collection* atau teknik kriptografi penjaga privasi untuk mengurangi keterlihatan metadata transaksi tanpa menghilangkan kemampuan pengawasan [22, 36, 37].
6. Mengembangkan integrasi, interoperabilitas, dan interkoneksi dengan infrastruktur pasar keuangan pada penelitian terpisah setelah kebijakan buku besar ritel tervalidasi [5].
7. Mengembangkan pembayaran tanpa konektivitas sebagai lingkup terpisah yang mencakup perangkat aman, pencegahan *double-spending*, dan rekonsiliasi sesuai Project Polaris [34, 35].
8. Menguji API *backend* secara *end-to-end* dengan basis data studi dan identitas per pengguna, lalu menjalankan Caliper terpisah pada tiga *chaincode* apabila evaluasi kinerjanya diperlukan.

# DAFTAR PUSTAKA


[1] Hyperledger Foundation, "Hyperledger caliper: A blockchain performance benchmark framework," 2024, accessed: Jun. 27, 2026. [Online]. Available: https://hyperledger.github.io/caliper/

[2] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "BLOCKBENCH: A framework for analyzing private blockchains," in *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 2017, pp. 1085–1100.

[3] F. Syarifuddin and T. Bakhtiar, "Monetary policy strategy in the presence of central bank digital currency," Bank Indonesia, Working Paper WP/09/2021, 2021.

[4] Hyperledger Foundation, "Couchdb as the state database," 2024, accessed: Jun. 28, 2026. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/couchdb_as_state_database.html

[5] Bank Indonesia, "Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency," Bank Indonesia, Jakarta, Tech. Rep., 2022.

[6] R. A. Auer and R. Böhme, "The technology of retail central bank digital currency," *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt2003j.htm

[7] D. Urbančok, "Blockchain open-source software comparison," Master's thesis, Masaryk University, Faculty of Informatics, Brno, 2019.

[8] Bank Indonesia, "Blueprint sistem pembayaran indonesia 2025," Bank Indonesia, Jakarta, Tech. Rep., 2019.

[9] R. A. Auer, J. Frost, L. Gambacorta, C. Monnet, T. Rice, and H. Shin, "Central bank digital currencies: motives, economic implications and the research frontier," *SSRN Electronic Journal*, 2022.

[10] L. Dionysopoulos, M. Marra, and A. Urquhart, "Central bank digital currencies: A critical review," *International Review of Financial Analysis*, vol. 91, p. 103031, 2024.

[11] Bank Indonesia, "Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report," Bank Indonesia, Jakarta, Tech. Rep., 2024.

[12] ——, "Consultative paper pengembangan rupiah digital: Tahap immediate state," Bank Indonesia, Jakarta, Tech. Rep., 2023.

[13] T. Zhang and Z. Huang, "Blockchain and central bank digital currency," *ICT Express*, vol. 8, no. 2, pp. 264–270, 2022.

[14] V. Sethaput and S. Innet, "Sok: Blockchain applications in central bank digital currencies (cbdcs)," in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.

[15] Bhawana and S. Kumar, "Permission blockchain network based central bank digital currency," in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.

[16] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Porto, Portugal: ACM, 2018, pp. 1–15.

[17] J. Barrdear and M. Kumhof, "The macroeconomics of central bank digital currencies," *Journal of Economic Dynamics and Control*, p. 104148, 2021.

[18] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, "A survey of research on retail central bank digital currency," *IMF Working Papers*, 2020.

[19] S. Allen, S. Čapkun, I. Eyal, G. Fanti, B. Ford, J. Grimmelmann, A. Juels, K. Kostiainen, S. Meiklejohn, A. K. Miller, E. Prasad, K. Wüst, and F. Zhang, "Design choices for central bank digital currency: Policy and technical considerations," *SSRN Electronic Journal*, 2020.

[20] A. Tsareva and M. Komarov, "Retail central bank digital currency design choices: Guide for policymakers," *IEEE Access*, vol. 12, pp. 66 129–66 146, 2024.

[21] M. M. Islam and H. P. In, "A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.

[22] N. Pocher and A. Veneris, "Privacy and transparency in cbdcs: A regulation-by-design aml/cft scheme," *IEEE Transactions on Network and Service Management*, vol. 19, pp. 1776–1788, 2022.

[23] P. Michalopoulos, O. Olowookere, N. Pocher, J. Sedlmeir, A. G. Veneris, and P. Puri, "Privacy and compliance design options in offline central bank digital currencies," *IEEE Transactions on Network and Service Management*, vol. 22, pp. 3748–3763, 2025.

[24] J. Hans, S. Khan, and D. Svetinovic, "Blockchain cbdc security threats using stride," in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.

[25] H. Sun, H. Mao, X. Bai, Z. Chen, K. Hu, and W. Yu, "Multi-blockchain model for central bank digital currency," in *2017 18th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT)*. IEEE, 2017, pp. 360–367.

[26] B. M. Zams, R. Indrastuti, A. G. Pangersa, N. A. Hasniawati, F. A. Zahra, and I. A. Fauziah, "Designing central bank digital currency for indonesia: The delphi–analytic network process," *Bulletin of Monetary Economics and Banking*, vol. 23, no. 3, pp. 413–440, 2020.

[27] F. Syarifuddin, "Optimal central bank digital currency design for emerging economies," *Journal of Central Banking Law and Institutions*, 2024.

[28] J. N. Aryani, "Designing central bank digital currency in indonesia: A critical literature review," 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.

[29] N. Rohmalia and I. G. B. B. Nugraha, "Designing a central bank digital currency (CBDC) retail model as a payment system," in *2023 10th International Conference on ICT for Smart Society (ICISS)*. IEEE, 2023, pp. 1–6.

[30] J. Ree, "Nigeria's enaira, one year after," International Monetary Fund, IMF Working Paper WP/23/104, 2023.

[31] N. Bhensook, T. Rattanakul, W. Synsatayakul, and P. Tohwisessuk, "Public use and distribution of retail cbdc: An evidence from thailand's retail cbdc pilot program," *Jahrbücher für Nationalökonomie und Statistik*, vol. 245, pp. 367 – 399, 2025.

[32] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, "A high performance payment processing system designed for central bank digital currencies," *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: https://eprint.iacr.org/2022/163

[33] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, "Performance analysis of a hyperledger fabric blockchain framework: Throughput, latency and scalability," in *2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2019, pp. 536–540.

[34] Bank for International Settlements, "Project polaris part 2: A security and resilience framework for cbdc systems," BIS Innovation Hub, Tech. Rep., 2023.

[35] ——, "Project polaris part 1: A handbook for offline payments with cbdc," BIS Innovation Hub, Tech. Rep., 2023.

[36] G. Soana and T. de Arruda, "Central bank digital currencies and financial integrity: finding a new trade-off between privacy and traceability within a changing financial architecture," *Journal of Banking Regulation*, vol. 25, pp. 467 – 486, 2024.

[37] J. E. Said and J. Lánský, "Designing retail central bank digital currencies: A systematic literature review of trade-offs between security, privacy, and financial stability," *International Journal of Financial Studies*, 2026.

[38] Y.-R. Wang, C.-Q. Ma, and Y.-S. Ren, "A model for cbdc audits based on blockchain technology: Learning from the dcep," *Research in International Business and Finance*, vol. 63, p. 101781, 2022.

[39] O. Atangana, L. Khoukhi, M. Barbier, and A. Kokcam, "Block-pad: A blockchain-enabled framework for resilient and flexible cbdc transactions leveraging digital identity," *Computer Networks*, vol. 275, p. 111805, 2026.

[40] K. L. Lukito, J. L. C. Putra, G. W. Saputra, R. A. K. Putra, and R. Wiputra, "Comparative analysis of CBDC technology governance and architecture: Implications for Indonesia's Project Garuda," in *2026 IEEE 5th International*

*Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA)*. IEEE, 2026, pp. 1109–1115.

[41] J. E. Abang, H. Takruri, R. Al-Zaidi, and M. Al-Khalidi, "Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an iot perspective," *Internet of Things*, vol. 26, p. 101217, 2024.

[42] A. Lohachab, S. Garg, B. H. Kang, and M. B. Amin, "Performance evaluation of hyperledger fabric-enabled framework for pervasive peer-to-peer energy trading in smart cyber–physical systems," *Future Generation Computer Systems*, vol. 118, pp. 392–416, 2021.

[43] M. Ajwalia and P. Shah, "Performance comparison of permissioned and permissionless blockchain by varying workload transaction," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 5, no. 4, p. 100251, 2025.

[44] C. Beer, S. Zingg, K. Kostiainen, K. Wüst, V. Capkun, and S. Čapkun, "Payoff: A regulated central bank digital currency with private offline payments," 2024. [Online]. Available: https://arxiv.org/abs/2408.06956

[45] H. Arslanian, "Retail central bank digital currencies," in *The Book of Crypto*. Springer International Publishing, 2022, pp. 203–232.

[46] M. L. Bech and R. Garratt, "Central bank cryptocurrencies," *BIS Quarterly Review*, pp. 55–70, Sep. 2017. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt1709f.htm

[47] W. Xiong, X. Yao, and X. Deng, "Research on the performance analysis of mainstream consensus algorithms of consortium blockchains," in *2024 2nd International Conference on Mobile Internet, Cloud Computing and Information Security (MICCIS)*. IEEE, 2024.

[48] S. Das, M. Mishra, and R. Priyadarshini, "Hyperledger fabric: The enterprise-grade blockchain for modern business applications," in *Fostering Machine Learning and IoT for Blockchain Technology*. Springer Nature Singapore, 2025, pp. 163–188.

[49] Hyperledger Performance and Scale Working Group, "Hyperledger blockchain performance metrics," Hyperledger Foundation, Tech. Rep., 2018.

[50] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[51] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. New York: Addison-Wesley Professional, 2015.

[52] C. Saraf and S. Sabadra, "Blockchain platforms: A compendium," in *2018 IEEE International Conference on Innovative Research and Development (ICIRD)*. IEEE, 2018, pp. 1–6.

[53] I. Systems. (2024) Nosql databases in financial services: Modernizing banking and insurance data management. [Online]. Available: https://www.ispirer.com/blog/nosql-databases-financial-services

[54] PostgreSQL Global Development Group. (2025) Postgresql 18 documentation: Json types, row security policies, and mvcc concurrency control. PostgreSQL Global Development Group. [Online]. Available: https://www.postgresql.org/docs/current/

[55] LogicMonitor. (2025) Postgresql vs. mysql: A comprehensive comparison. [Online]. Available: https://www.logicmonitor.com/blog/postgresql-vs-mysql

[56] Hyperledger Foundation. (2025) Hyperledger fabric documentation: Private data collections. Linux Foundation. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html

[57] P. Thakkar, S. Nathan, and B. Viswanathan, "Performance benchmarking and optimizing hyperledger fabric blockchain platform," in *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2018, pp. 264–276.

[58] C. Gorenflo, S. Lee, L. Golab, and S. Keshav, "FastFabric: Scaling Hyperledger Fabric to 20,000 transactions per second," *International Journal of Network Management*, vol. 30, no. 5, 2020, preprint: arXiv:1901.00910 [cs.DC].

# LAMPIRAN

Lampiran ini memuat kode sumber inti sistem Rupiah Digital ritel yang dibangun pada penelitian ini, meliputi fungsi-fungsi utama *chaincode* `digital-rupiah` serta modul konfigurasi dan *workload* pengujian Hyperledger Caliper. Versi lampiran ini diselaraskan dengan suite benchmark yang digunakan pada pembahasan hasil final, sehingga mencakup konfigurasi transfer dasar 1/2/4 *worker* dan *transfer-repeat*. Lampiran hanya memuat artefak yang langsung mendukung argumentasi tesis. Berkas hasil run (*log*, *HTML report*, dan *JSON summary*), konfigurasi Docker/Fabric standar, serta lapisan *backend* REST API tidak ditampilkan ulang di sini karena bukan bagian inti dari kontribusi benchmark yang dibahas pada Bab III dan Bab IV. Kode sumber lengkap tetap tersedia pada repositori sistem.

## L.1 Source Code Otorisasi MSP *Chaincode*

Bagian ini memuat fungsi pemeriksaan identitas organisasi untuk membatasi mutasi pada batas kepercayaan *chaincode*.

```
  contractapi.Contract
}

const (
  BankIndonesiaMSP  = "BankIndonesiaOrgMSP"
  HimbaraBankMSP    = "HimbaraBankOrgMSP"
  CommercialBankMSP = "CommercialBankOrgMSP"
  OJKObserverMSP    = "OJKObserverOrgMSP"
  PJPMSP            = "PJPOrgMSP"

  schemaStateKey  = "schema::digital_rupiah_v3"
  schemaVersionV3 = "v3"
)

func requireMSP(ctx contractapi.TransactionContextInterface, allowed ...string) error {
  identity := ctx.GetClientIdentity()
  if identity == nil {
    return fmt.Errorf("access denied: client identity is unavailable")
  }
  mspID, err := identity.GetMSPID()
  if err != nil {
    return fmt.Errorf("access denied: resolve client MSP: %w", err)
  }
  for _, allowedMSP := range allowed {
    if mspID == allowedMSP {
      return nil
    }
  }
  return fmt.Errorf("access denied: MSP %s is not authorized", mspID)
}

func requireBankIndonesia(ctx contractapi.TransactionContextInterface) error {
```

## L.2 Source Code Pengawasan dan Rekonsiliasi

Bagian ini memuat kode sumber fungsi laporan rekonsiliasi, metrik, dan emisi *event* pengawasan (berkas `chaincode/digital_rupiah.go`).

```
func (s *SmartContract) GetReconciliationReport(ctx contractapi.
    TransactionContextInterface) (*ReconciliationReport, error) {
  wallets, err := s.GetAllWallets(ctx)
  if err != nil {
    return nil, err
  }
  participants, err := s.getAllParticipants(ctx)
  if err != nil {
    participants = nil
  }
  var totalSupply int64
  for _, w := range wallets {
    totalSupply += w.Balance
  }
```

```
  report := ReconciliationReport{
    ReportID:     fmt.Sprintf("rec_%s", txNow(ctx).Format("20060102150405")),
    TotalSupply:  totalSupply,
    TotalIssued:  0,
    TotalBurned:  0,
    Wallets:      len(wallets),
    Participants: len(participants),
    GeneratedAt:  txNow(ctx).Format(time.RFC3339),
  }
  txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
  if err == nil {
    defer txIter.Close()
    for txIter.HasNext() {
      kv, err := txIter.Next()
      if err != nil {
        continue
      }
      var tr TransactionRecord
      if err := json.Unmarshal(kv.Value, &tr); err != nil {
        continue
      }
      switch tr.TransactionType {
      case TxMint:
        report.TotalIssued += tr.Amount
      case TxBurn:
        report.TotalBurned += tr.Amount
      }
    }
  }
  return &report, nil
}

func (s *SmartContract) GetMetrics(ctx contractapi.TransactionContextInterface) (*
    Metrics, error) {
  wallets, err := s.GetAllWallets(ctx)
  if err != nil {
    return nil, err
  }
  participants, err := s.getAllParticipants(ctx)
  if err != nil {
    participants = nil
  }
  var totalSupply int64
  activeWallets := int64(0)
  for _, w := range wallets {
    totalSupply += w.Balance
    if !w.Frozen {
      activeWallets++
    }
  }
  activeParticipants := int64(0)
  for _, p := range participants {
    if p.Status == ParticipantActive {
      activeParticipants++
    }
  }
  m := Metrics{
    ActiveWallets:     activeWallets,
    ActiveParticipants: activeParticipants,
```

```
    TotalSupply:       totalSupply,
  }
  txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
  if err == nil {
    defer txIter.Close()
    for txIter.HasNext() {
      kv, err := txIter.Next()
      if err != nil {
        continue
      }
      var tr TransactionRecord
      if err := json.Unmarshal(kv.Value, &tr); err != nil {
        continue
      }
      m.TotalTransactions++
      switch tr.Status {
      case TxSettled:
        m.SettledCount++
      case TxRejected:
        m.RejectedCount++
      }
    }
  }
  return &m, nil
}

func (s *SmartContract) emitSupervisionEvent(ctx contractapi.TransactionContextInterface
    , eventType string, entityType string, entityID string, data interface{}) error {
  now := txNow(ctx)
  eventID := fmt.Sprintf("sup_%s_%s", entityID, now.Format("20060102150405"))
  var dataStr string
  if data != nil {
    b, err := json.Marshal(data)
    if err != nil {
      return err
    }
    dataStr = string(b)
  }
  ev := SupervisionEvent{
    EventID:    eventID,
    EventType:  eventType,
    EntityType: entityType,
    EntityID:   entityID,
    Data:       dataStr,
    Timestamp:  now.Format(time.RFC3339),
  }
  key, err := ctx.GetStub().CreateCompositeKey("supervision", []string{entityID, eventID
    })
  if err != nil {
    return err
  }
  evData, err := json.Marshal(ev)
  if err != nil {
    return err
  }
  return ctx.GetStub().PutState(key, evData)
}
```

## L.3 Source Code Konfigurasi Jaringan Caliper

Bagian ini memuat konfigurasi koneksi Caliper ke jaringan Fabric. Nilai *top-level* `version` menyatakan versi skema konfigurasi Caliper, sedangkan versi kontrak dicatat terpisah pada entri kanal. Berkas dimuat langsung dari sumber otoritatif agar lampiran tetap sesuai dengan konfigurasi eksekusi.

```
---
name: PayNet Garuda Network
version: "2.0.0"
caliper:
  blockchain: fabric

channels:
  - channelName: mychannel
    contracts:
      - id: digital-rupiah
        version: "3.0"

organizations:
  - mspid: BankIndonesiaOrgMSP
    identities:
      certificates:
        - name: User1
          clientSignedCert:
            path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.paynet/msp/signcerts/cert.pem
          clientPrivateKey:
            path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.paynet/msp/keystore/key.pem
    connectionProfile:
      path: network/organizations/peerOrganizations/bi.paynet/connection-bi.json
      discover: true

  - mspid: HimbaraBankOrgMSP
    identities:
      certificates:
        - name: User1
          clientSignedCert:
            path: network/organizations/peerOrganizations/himbara.paynet/users/User1@himbara.paynet/msp/signcerts/cert.pem
          clientPrivateKey:
            path: network/organizations/peerOrganizations/himbara.paynet/users/User1@himbara.paynet/msp/keystore/key.pem
    connectionProfile:
      path: network/organizations/peerOrganizations/himbara.paynet/connection-himbara.json
      discover: true

  - mspid: CommercialBankOrgMSP
    identities:
      certificates:
        - name: User1
          clientSignedCert:
            path: network/organizations/peerOrganizations/commercial.paynet/users/User1@commercial.paynet/msp/signcerts/cert.pem
          clientPrivateKey:
            path: network/organizations/peerOrganizations/commercial.paynet/users/
```

```
    User1@commercial.paynet/msp/keystore/key.pem
    connectionProfile:
      path: network/organizations/peerOrganizations/commercial.paynet/connection-commercial.json
      discover: true
```

## L.4 Source Code Suite Konfigurasi *Benchmark* Caliper

Bagian ini memuat konfigurasi benchmark yang digunakan pada versi tesis saat ini. Berkas dimuat langsung dari sumber otoritatif agar lampiran tidak menyimpang dari konfigurasi eksekusi.

```
test:
  name: Digital Rupiah Retail Transfer Benchmark - 1 Worker
  description: Isolated customer-to-customer retail transfer baseline with one Caliper worker
  workers:
    type: local
    number: 1
  rounds:
    - label: retail-warmup-w1
      txDuration: 30
      rateControl:
        type: fixed-rate
        opts:
          tps: 10
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w1
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          fundStandard: 20000000
          fundReceiverStandard: 1000000
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-sustained-w1
      txDuration: 120
      rateControl:
        type: fixed-rate
        opts:
          tps: 30
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w1
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-peak-w1
```

```
      txDuration: 120
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w1
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

observer:
  type: local
  interval: 1

monitors:
  transaction:
    - module: ./benchmark/observers/jsonl-transaction-observer.js
      options: {}
  resource:
    - module: docker
      options:
        interval: 1
        cpuUsageNormalization: true
        containers:
          - /peer0.bi.paynet
          - /peer0.himbara.paynet
          - /peer0.commercial.paynet
          - /peer0.pjp.paynet
          - /peer0.ojk.paynet
          - /couchdb0.bi.paynet
          - /couchdb0.himbara.paynet
          - /couchdb0.commercial.paynet
          - /couchdb0.pjp.paynet
          - /couchdb0.ojk.paynet
          - /orderer.paynet
```

```
test:
  name: Digital Rupiah Retail Transfer Benchmark - 2 Workers
  description: Isolated customer-to-customer retail transfer baseline with two Caliper
    workers
  workers:
    type: local
    number: 2
  rounds:
    - label: retail-warmup-w2
      txDuration: 30
      rateControl:
        type: fixed-rate
        opts:
          tps: 10
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
```

```
          scenario: transfer-w2
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          fundStandard: 20000000
          fundReceiverStandard: 1000000
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-sustained-w2
      txDuration: 120
      rateControl:
        type: fixed-rate
        opts:
          tps: 30
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w2
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-peak-w2
      txDuration: 120
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w2
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

observer:
  type: local
  interval: 1

monitors:
  transaction:
    - module: ./benchmark/observers/jsonl-transaction-observer.js
      options: {}
  resource:
    - module: docker
      options:
        interval: 1
        cpuUsageNormalization: true
        containers:
          - /peer0.bi.paynet
```

```
          - /peer0.himbara.paynet
          - /peer0.commercial.paynet
          - /peer0.pjp.paynet
          - /peer0.ojk.paynet
          - /couchdb0.bi.paynet
          - /couchdb0.himbara.paynet
          - /couchdb0.commercial.paynet
          - /couchdb0.pjp.paynet
          - /couchdb0.ojk.paynet
          - /orderer.paynet
```

```
test:
  name: Digital Rupiah Retail Transfer Benchmark - 4 Workers
  description: Isolated customer-to-customer retail transfer baseline with four Caliper
    workers
  workers:
    type: local
    number: 4
  rounds:
    - label: retail-warmup-w4
      txDuration: 30
      rateControl:
        type: fixed-rate
        opts:
          tps: 10
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w4
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          fundStandard: 20000000
          fundReceiverStandard: 1000000
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-sustained-w4
      txDuration: 120
      rateControl:
        type: fixed-rate
        opts:
          tps: 30
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w4
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

    - label: retail-peak-w4
      txDuration: 120
      rateControl:
        type: fixed-rate
```

```
        opts:
          tps: 60
      workload:
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-w4
          customers: 280
          merchants: 0
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725

observer:
  type: local
  interval: 1

monitors:
  transaction:
    - module: ./benchmark/observers/jsonl-transaction-observer.js
      options: {}
  resource:
    - module: docker
      options:
        interval: 1
        cpuUsageNormalization: true
        containers:
          - /peer0.bi.paynet
          - /peer0.himbara.paynet
          - /peer0.commercial.paynet
          - /peer0.pjp.paynet
          - /peer0.ojk.paynet
          - /couchdb0.bi.paynet
          - /couchdb0.himbara.paynet
          - /couchdb0.commercial.paynet
          - /couchdb0.pjp.paynet
          - /couchdb0.ojk.paynet
          - /orderer.paynet
```

```
test:
  name: Digital Rupiah Retail Transfer - Repetition (mean +/- SD)
  description: Five isolated 60-second peak rounds of the customer-to-customer transfer
    workload for mean and standard-deviation estimation
  workers:
    type: local
    number: 2
  rounds:
    - label: retail-peak-rep-1
      txDuration: 60
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload: &repeat_workload
        module: benchmark/workloads/retail-transfer.js
        arguments:
          scenario: transfer-repeat
          customers: 280
          merchants: 0
```

```
          transactionSlots: 100
          fundedRatio: 1
          maxTransferAmount: 50000
          seed: 20260725
    - label: retail-peak-rep-2
      txDuration: 60
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload: *repeat_workload
    - label: retail-peak-rep-3
      txDuration: 60
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload: *repeat_workload
    - label: retail-peak-rep-4
      txDuration: 60
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload: *repeat_workload
    - label: retail-peak-rep-5
      txDuration: 60
      rateControl:
        type: fixed-rate
        opts:
          tps: 60
      workload: *repeat_workload

observer:
  type: local
  interval: 1

monitors:
  transaction:
    - module: ./benchmark/observers/jsonl-transaction-observer.js
      options: {}
  resource:
    - module: docker
      options:
        interval: 1
        cpuUsageNormalization: true
        containers:
          - /peer0.bi.paynet
          - /peer0.himbara.paynet
          - /peer0.commercial.paynet
          - /peer0.pjp.paynet
          - /peer0.ojk.paynet
          - /couchdb0.bi.paynet
          - /couchdb0.himbara.paynet
          - /couchdb0.commercial.paynet
          - /couchdb0.pjp.paynet
          - /couchdb0.ojk.paynet
          - /orderer.paynet
```

## L.5 Source Code Skrip Pembangkit Aset Jaringan Caliper

Bagian ini memuat kode sumber skrip penyiapan aset jaringan untuk Caliper (berkas `benchmark/gen-network-assets.sh`).

```
#!/bin/bash
# Generates Caliper connection profiles + stable identity files for the garuda network.
# Idempotent; safe to re-run after the network is up. Run from repo root.
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
PROJECT_DIR="$(dirname "$SCRIPT_DIR")"
ORG_ROOT="$PROJECT_DIR/network/organizations/peerOrganizations"

# org | OrgName | MSPID | peerPort
ORGS=(
  "bi|BankIndonesiaOrg|BankIndonesiaOrgMSP|7051"
  "himbara|HimbaraBankOrg|HimbaraBankOrgMSP|9051"
  "commercial|CommercialBankOrg|CommercialBankOrgMSP|11051"
)

for entry in "${ORGS[@]}"; do
  IFS='|' read -r ORG ORGNAME MSPID PORT <<< "$entry"
  DOMAIN="${ORG}.paynet"
  ORG_DIR="$ORG_ROOT/$DOMAIN"
  PEER="peer0.$DOMAIN"
  TLS_CA="$ORG_DIR/peers/$PEER/tls/ca.crt"
  USER_MSP="$ORG_DIR/users/User1@$DOMAIN/msp"

  # Stable identity filenames (cryptogen emits hashed keystore names).
  cp "$USER_MSP/signcerts/User1@$DOMAIN-cert.pem" "$USER_MSP/signcerts/cert.pem"
  cp "$USER_MSP/keystore/"*_sk "$USER_MSP/keystore/key.pem"

  cat > "$ORG_DIR/connection-$ORG.json" <<EOF
{
  "name": "$ORG-paynet",
  "version": "1.0.0",
  "client": { "organization": "$ORGNAME" },
  "channels": {
    "mychannel": { "peers": { "$PEER": {} } }
  },
  "organizations": {
    "$ORGNAME": { "mspid": "$MSPID", "peers": ["$PEER"] }
  },
  "peers": {
    "$PEER": {
      "url": "grpcs://localhost:$PORT",
      "tlsCACerts": { "path": "$TLS_CA" },
      "grpcOptions": {
        "ssl-target-name-override": "$PEER",
        "hostnameOverride": "$PEER"
      }
    }
  }
}
EOF
  echo "Generated connection-$ORG.json + stable identity for $MSPID"
done
```